Knowledge-Based Editor

Volume 4

Programmer's Apprentice Group

MIT AI Lab

January, 1985

Table of Contents: Knowledge Based Editor

Volume 1

OZ:<KBE.DOC>358.TEXT
OZ:<KBE.TEST>358.LISP

OZ:<KBE>KBE.INDEX

OZ:<KBE>LISPM.INIT
OZ:<KBE>SYSDEF.LISP

OZ:<KBE.SOURCE>BASIC.LISP
OZ:<KBE.SOURCE>NOISE.LISP
OZ:<KBE.SOURCE>SETS.LISP
OZ:<KBE.SOURCE>CONDITIONS.LISP
OZ:<KBE.SOURCE>ENGLISH.LISP
OZ:<KBE.SOURCE>SYSTEM-MAGIC.LISP
OZ:<KBE.SOURCE>FLAVOR-MAGIC.LISP
OZ:<KBE.SOURCE>MACRO.LISP
OZ:<KBE.SOURCE>KVARS.LISP
OZ:<KBE.SOURCE>KMAC.LISP
OZ:<KBE.SOURCE>INTERFACE-MACROS.LISP
OZ:<KBE.SOURCE>MBOX.LISP
OZ:<KBE.SOURCE>POST-OFFICE.LISP
OZ:<KBE.SOURCE>BEHAVIOR.LISP
OZ:<KBE.SOURCE>EVENTS.LISP
OZ:<KBE.SOURCE>ACTOR.LISP
OZ:<KBE.SOURCE>COMPLETE.LISP
OZ:<KBE.SOURCE>COMMENT-MACRO.LISP
OZ:<KBE.SOURCE>READMACRO.LISP
OZ:<KBE.SOURCE>OUTIL.LISP
OZ:<KBE.SOURCE>DTRACE.LISP
OZ:<KBE.SOURCE>DUTIL.LISP
OZ:<KBE.SOURCE>DMAC.LISP
OZ:<KBE.SOURCE>DVARS.LISP
OZ:<KBE.SOURCE>DB.LISP
OZ:<KBE.SOURCE>GC.LISP
OZ:<KBE.SOURCE>DDEBUG.LISP

Volume 2

OZ:<KBE.SOURCE>SURFACE.LISP
OZ:<KBE.SOURCE>EXPAND.LISP
OZ:<KBE.SOURCE>PRIMOP.LISP
OZ:<KBE.SOURCE>DFLOW.LISP
OZ:<KBE.SOURCE>UTIL.LISP
OZ:<KBE.SOURCE>SEGTRN.LISP
OZ:<KBE.SOURCE>ANALYZE.LISP
OZ:<KBE.SOURCE>CODER.LISP

Volume 3

OZ:<KBE.SOURCE>PSEUDO.LISP
OZ:<KBE.SOURCE>DICT.LISP
OZ:<KBE.SOURCE>GRAM.LISP
OZ:<KBE.SOURCE>REFER.LISP
OZ:<KBE.SOURCE>VERBS.LISP
OZ:<KBE.SOURCE>IMPLEMENT.LISP
OZ:<KBE.SOURCE>ADA-EVAL.LISP
OZ:<KBE.SOURCE>ADA-ANAL.LISP
OZ:<KBE.SOURCE>ADA-PARSE.LISP
OZ:<KBE.SOURCE>ADA-UNPARSE.LISP
OZ:<KBE.SOURCE>VIEW.LISP
OZ:<KBE.SOURCE>PLI.LISP
OZ:<KBE.SOURCE>DEFLIB.LISP
OZ:<KBE.SOURCE>SYSLIB.LISP
OZ:<KBE.SOURCE>KBELIB.LISP
OZ:<KBE.SOURCE>DEMO-LIB.LISP
OZ:<KBE.SOURCE>DEMO-ADA-LIB.LISP

Volume 4

OZ:<KBE.SOURCE>BRIDGE.LISP
OZ:<KBE.SOURCE>KAOS.LISP
OZ:<KBE.SOURCE>SOCIETY.LISP
OZ:<KBE.SOURCE>STANDARD-ACTOR.LISP
OZ:<KBE.SOURCE>REMOTE.LISP
OZ:<KBE.SOURCE>OVERLAPPING-REGIONS.LISP
OZ:<KBE.SOURCE>MOUSE-SENSITIVE-REGIONS.LISP
OZ:<KBE.SOURCE>OPTIONS.LISP
OZ:<KBE.SOURCE>SIMPLE.LISP
OZ:<KBE.SOURCE>SIMPLE-ACTOR.LISP
OZ:<KBE.SOURCE>MAGIC-ACTOR.LISP
OZ:<KBE.SOURCE>TV-MAGIC.LISP
OZ:<KBE.SOURCE>ZWEI-MAGIC.LISP
OZ:<KBE.SOURCE>POSSIBILITIES.LISP
OZ:<KBE.SOURCE>BELIEF.LISP
OZ:<KBE.SOURCE>EDIT.LISP
OZ:<KBE.SOURCE>EDIT-ACTOR.LISP
OZ:<KBE.SOURCE>CONSISTENCY.LISP
OZ:<KBE.SOURCE>EDIT-COMMANDS.LISP
OZ:<KBE.SOURCE>EDIT-MISC.LISP
OZ:<KBE.SOURCE>EDIT-SIMPLE.LISP
OZ:<KBE.SOURCE>EDIT-DEBUG.LISP
OZ:<KBE.SOURCE>KBE-MODE.LISP
OZ:<KBE.SOURCE>FOCUS.LISP
OZ:<KBE.SOURCE>TEST.LISP
OZ:<KBE.SOURCE>DOC.LISP
OZ:<KBE.SOURCE>PARSE.LISP
OZ:<KBE.SOURCE>EXPLAIN.LISP
OZ:<KBE.SOURCE>DOCUMENT.LISP
OZ:<KBE.SOURCE>PBM-HANDLERS.LISP
OZ:<KBE.SOURCE>INIT.LISP
OZ:<KBE.SOURCE>DEBUG.LISP

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; BRIDGE: Functions to bridge the gap between the KBE internals
;;;         and the interface

;;; ASSOCIATED-INPUT-LINES
;;;
;;;  Making this a DB-PROPERTY means that if the associated seg is killed,
;;;  this property will be killed from its plist. Actually, this is a
;;;  cheat since there out to be a place for it as a database assertion,
;;;  but that really doesn't matter for now.

(def-db-property associated-input-lines)
```

```
;;; (EXECUTE request)
;;;
;;;   Generic request handler. Handles texts and strings.

(defun execute (request)
  (cond ((stringp request)
         (eval-command (parse-command request)))
        ((not (atom request))
         (eval-command request))
        (t
         (barf "~S is not a valid request. -- EXECUTE" request))))
```

```
(defun parsed->code (parsed)
  (let ((things (eval-command parsed)))
    (if things (~code (sel1 (??& (sel1 things) @seg))))))

(defun code->facts (sexp)
  (cond ((not sexp) nil)
        (t (??& * @seg * * (list sexp)))))

(defun fact->parsed (fact)
  (make-ref (~deep fact) t))

(defun fact->seg (fact) (~seg (~deep fact)))

(defun superseg (fact) (?supseg fact))
```

```
(defun name->seg (name)
  (or (~01 ~seg~segcase (??& * @seg 'lambda-exp name))
      (~01 ~seg~segcase (??& * cliche-def name))))

(defun name->segcase (name)
  (let ((x (name->seg name))) (if x (ncons x))))

(defun get-toplevel-name (referent)
  (let ((top-seg (get-toplevel-seg referent)))
    (or (~01 ~cliche-def-name (??& (&* top-seg) cliche-def))
        (~01 ~name (??& (&* top-seg) @seg 'lambda-exp)))))

(defun get-name-from-seg (seg) ;nicer name
  (?@seg-name seg))

(defun get-toplevel-seg (referent)
  (plan-top (~seg (get-segcase referent))))

(defun slot-marker? (sexp)
  (and (not (atom sexp))
       (eq (car sexp) 'kbe-#-comment-macro)))
```

```
(defun code->parsed (sexp)
  (let ((facts (code->facts sexp)))
    (let ((filtered-facts
           (mapcan #'(lambda (x)
                       (if (or (eq (~kind x) 'lambda-exp)
                               (??& * slot (~deep x)))
                           (ncons x)))
                   facts)))
      (cond ((not filtered-facts) nil)        ; no description?
            (t (fact->parsed (car filtered-facts)))))))
```

```
;;; (PARSED->FACTS parsed-form)
;;;
;;;  Gets a list of facts corresponding to what the parsed reference
;;;  refers to if such a reference exists.
;;;
;;;  The second return value says if there was a failure. If the second
;;;  value is NIL, then the first value is the facts list. If the
;;;  second value is non-null, then there was some sort of problem
;;;  evaluating the parsed reference and the second value is an error
;;;  string.

(defun parsed->facts (parsed-form)
  (condition-case (result)
      (eval-command parsed-form)
    (sys:error (values nil "Internal error resolving a parsed reference."))
    (:no-error (values result
                       (if (not result) "That's meaningless to me."))))))
```

```
;;; Things used in VIEW...

(defun plan-top (seg)
  (do ((s           seg           ss )
       (ss (?supseg seg) (?supseg ss)))
      ((null ss) s)))

(defun get-segcase (thing)
  (cond ((not (fact? thing)) thing)
        ((port-fact? thing) (~segcase thing))
        ((cflow-fact? thing) (~from thing))
        (t (~segcase (~from thing)))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp WHEN-ERROR Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; KAOS: A package for communicating via the Chaos net.
;;;
;;; The general shape of this code was originally patterned after
;;; code written by CWH at Symbolics in the file
;;;   PTR:>cwh>hacks>hostls.lisp
;;; Adapted for use with the KBE by KMP, 20-Dec-82.

(defvar *kaos-connect-name* "KAOS"
  "Connect name for use by the KAOS protocol.")

(defvar *kaos-syntax-environment*
        '((PACKAGE  ',(pkg-find-package "USER"))
          (PRINLEVEL NIL)
          (PRINLENGTH NIL)
          (PRINENDLINE NIL)
          (IBASE    10.)
          (BASE     10.)
          (*NOPOINT NIL))
  "Bindings that should be in effect when KAOS is doing I/O")

(defmacro kaos-handler (x) '(get ,x 'kaos-handler))

(defdefmacro define-kaos-handler "KBE Chaos handler" (name bvl . body)
  '(defun (:property ,name kaos-handler) ,bvl
     ,@body))

(defun kaos-server ()
  (ignore-errors
    (let ((conn (chaos:listen *kaos-connect-name*)))
      (chaos:accept conn)
      (with-open-stream (stream (chaos:make-stream conn
                                                   ':direction
                                                   ':bidirectional))
        (multiple-value-bind (message-name arglist)
            (progw *kaos-syntax-environment*
              (values (read stream) (read stream)))
          (send stream ':finish)
          (let ((result (call-kaos-handler message-name arglist)))
            (progw *kaos-syntax-environment*
              (format stream "~S" result)))
          (send stream ':force-output)     ; Get the bits out on the net
          (send stream ':finish)           ; Keep retransmitting until
                                           ; the other side has them
          (send stream ':close ':abort))))))) ; Close without sending an EOF

(define-internal-option *kaos-debug*
                        "Debug KBE network errors"
  nil :boolean)

(defun handle-kaos-error (cond)
  (catch-error-restart ((sys:error sys:abort)
                        "Continue processing this error.")
    (if *kaos-debug* (funcall #'dbg))) ;maybe run the debugger
  (*throw 'kaos-handler-error-return (format nil "~A" cond)))

(defun call-kaos-handler (message-name arglist)
  (let ((handler (kaos-handler message-name)))
    (cond ((not handler)
```

```
          (format nil "No handler for the ~S message." message-name))
         (t
          (*catch 'kaos-handler-error-return
            (condition-bind ((sys:error #'handle-kaos-error))
              (multiple-value-list
                (lexpr-funcall handler arglist))))))))

(add-initialization
  *kaos-connect-name*
  '(process-run-temporary-function "Kaos Server" 'kaos-server)
  nil
  'chaos:server-alist)
```

```
(defun kaos-send-returning-list (host-name message-name &rest args)
  (let ((host (cond ((null host-name) si:local-host)
                    ((stringp host-name) (si:parse-host host-name))
                    (t host-name))))
    (progw *kaos-syntax-environment*
      (with-open-stream (stream
                          (chaos:open-stream host *kaos-connect-name*
                                             ':direction ':bidirectional))
        (format stream "~S ~S" message-name args)
        (send stream ':finish)
        (prog1 (read stream)
               ;; We're done, so close without sending an EOF ...
               (send stream ':close ':abort))))))

(defun kaos-send (host-name message-name &rest args)
  (let ((result (lexpr-funcall #'kaos-send-returning-list
                               host-name message-name args)))
    (cond ((stringp result)
           (barf "~A" result))
          (t (values-list result)))))

(defun kaos-lexpr-send (&rest host-message-and-lexpr-args)
  (lexpr-funcall #'lexpr-funcall #'kaos-send host-message-and-lexpr-args))
```

```
;;;; Debugging Tools

(comment ;This stuff needn't be dumped with the main system...

;;; This page constructs a KAOS user window for debugging purposes.

(defvar *user-window*
        (tv:make-window 'tv:window
                        ':edges (multiple-value-bind (width height)
                                    (send tv:mouse-sheet ':size)
                                  height ;ignored
                                  '(0 0. ,width 100.))
                        ':label "KAOS User Window"
                        ':save-bits t)
  "KAOS user debugging window")

 ; Note: This server test program works only for one-arg handlers.

(defun kaos-server-test (&optional host-name (terminal-io *user-window*))
  (unwind-protect
    (progn (send terminal-io ':expose)
           (do ((message-name (kaos-prompt-for "Message Type")
                              (kaos-prompt-for "Message Type")))
               ((null message-name)
                (send terminal-io ':bury))
             (let ((data (kaos-prompt-for "Message Data")))
               (let ((result
                       (kaos-send-returning-list host-name
                                                 message-name
                                                 data)))
                 (cond ((stringp result)
                        (format t "~&Failed: ~A~%" result))
                       (t
                        (format t "~{~&^Y ~S~}~%" result)))))))))
    (send *user-window* ':bury)))

) ;tnemmoc
```

```
(comment ;This stuff needn't be dumped with the main system...

;;; This page constructs some sample KAOS servers for
;;; debugging purposes.

(defvar *server-window*
        (tv:make-window 'tv:window
                        ':edges (multiple-value-bind (width height)
                                    (send tv:mouse-sheet ':size)
                                  height ;ignored
                                  '(0 100. ,width 200.))
                        ':label "Odd Eval Server Window"
                        ':save-bits t)
  "Server window")

(defvar *kaos-prompt* "")

(defun kaos-prompt (&rest ignore)
  (format t "~A: " *kaos-prompt*))

(defun kaos-prompt-for (*kaos-prompt*)
  (send terminal-io ':rubout-handler '((:prompt kaos-prompt)) #'read))

(define-kaos-handler :EVAL (form)
  (eval form))

(define-kaos-handler :FUNNY-EVAL (form)
  (let ((terminal-io *server-window*))
    (send terminal-io ':select)
    (unwind-protect (eval (kaos-prompt-for
                            (format nil "Return value for ~S" form)))
                    (send *server-window* ':bury))))

) ;tnemmoc
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

(defflavor society (members lock-cell) ()
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :members   nil
    :lock-cell nil))

(defmethod (society :print-self)  (stream level prin1?
                                   &rest ignore)
  level ;ignored
  (cond ((not prin1?)
         (format stream "Society-~O" (%pointer self)))
        (t
         (format stream "#<Society (~{~A~^, ~}) ~O>"
                 members (%pointer self)))))

(defmethod (society :insert) (actor)
  (with-lock (lock-cell :whostate "Await Society")
    (cond ((memq actor members) nil)
          (t
           (push actor members)
           actor))))

(defmethod (society :delete) (actor)
  (with-lock (lock-cell :whostate "Await Society")
    (cond ((memq actor members)
           (setq members (remove actor members))
           actor)
          (t nil))))

(defmethod (society :map) (fn)
  (mapc fn members)
  t)

(compile-flavor-methods society)
```

```
;; This flavor should be mixed into a frame which wants to allow a given
;; actor to be responsible for negotiating replies to that frame.
;; Typically, the actor will have to send the frame a :SET-DELEGATE
;; message identifying itself as the owner of the frame.

(defflavor delegate-mixin ((delegate nil) (delegate-flavor nil)) ()
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables)

(defmethod (delegate-mixin :after :init) (&rest ignore)
  (when (not delegate)
    (setq delegate (make-instance delegate-flavor ':window self))))
```

```
(defvar *society* (make-instance 'society) "Holds data about our society")

(defmacro do-society ((var) &body forms)
  '(dolist (,var (send *society* ':members))
     ,@forms))

;;; (SEND-ALL message arg1 arg2 ... argN)
;;;
;;; Sends a given message to all actors in the KBE society.
;;; Return value is not guaranteed.

(defun send-all (message &rest args)
  (do-society (actor)
    (lexpr-send actor message args)))

(defun lexpr-send-all (message &rest args)
  (lexpr-funcall #'lexpr-funcall #'send-all message args))

;;; (SEND-ALL-THAT-HANDLE message arg1 arg2 ... argN)
;;;
;;; Sends a given message to all actors in the KBE society who claim
;;; to handle that message.

(defun send-all-that-handle (message &rest args)
  (do-society (actor)
    (lexpr-send-if-handles actor message args)))

(defun lexpr-send-all-that-handle (message &rest args)
  (lexpr-funcall #'lexpr-funcall #'send-all-that-handle message args))

;;; (SEND-ANY message arg1 arg2 ... argN)
;;;
;;; Sends a given message to the first actor which claims to handle
;;; the given message, returning that actor.

(defun send-any (message &rest args)
  (do-society (actor)
    (when (handles? actor message)
      (lexpr-send actor message args)
      (return actor))))

(defun lexpr-send-any (message &rest args)
  (lexpr-funcall #'lexpr-funcall #'send-any message args))

;;; (SEND-ANY-THAT-HANDLES message arg1 arg2 ... argN)
;;;
;;; Sends a given message to the first actor which claims to handle
;;; the given message and which does so returning a non-NIL result.
;;; Returns the actor.

(defun send-any-that-handles (message &rest args)
  (do-society (actor)
    (when (lexpr-send-if-handles actor message args)
      (return actor))))

(defun lexpr-send-any-that-handles (message &rest args)
  (lexpr-funcall #'lexpr-funcall #'send-any-that-handles message args))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp WITH-LOCK Indent:1;
;;; Lisp DO-SOCIETY Indent:1;
;;; Lisp WITH-DELEGATE Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; STANDARD-ACTOR: Basic actor definitions.
```

```
;;;; Vocabulary

(define-action report (status) "Requests a status report."
  "I don't know how to compute a status report.")
```

```
;;;; Actor I/O Mixin
;;;
;;; %%% Message Passing %%%
;;; I think most of this mixin can go away when the real message passing
;;; goes in. That should eliminate conflicts of multiple processes
;;; typing asynchronously on same window. The only thing that wants to
;;; stay is the WINDOW, REMARK-WINDOW, and TYPEOUT-WINDOW stuff.
;;; --KMP, 12/31/83

(defflavor actor-io-mixin ((io-lock nil)
                           (window nil)
                           (window-flavor nil)
                           (remark-window nil)
                           (typeout-window nil))
          ()
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables)

(defmethod (actor-io-mixin :after :init) (&rest ignore)
  (setq window (or window
                   (if window-flavor (tv:make-window window-flavor
                                                     ':delegate self))
                   (make-syn-stream 'terminal-io)))
  (setq typeout-window (or typeout-window
                           (send-if-handles window ':typeout-window)
                           window))
  (setq remark-window  (or remark-window
                           (send-if-handles window ':remark-window)
                           typeout-window
                           window)))

;;; :WITH-IO-LOCK message ...message-args...
;;;
;;;  General hook into the IO-LOCK facility.
;;;  Does a given message with given args while holding onto an I/O lock.

(defmethod (actor-io-mixin :with-io-lock) (message &rest args)
  (with-lock (io-lock :whostate "Await IO")
    (lexpr-send self message args)))

;;; :REMARK format-string ...format-args...

(defwhopper (actor-io-mixin :remark) (&rest args)
  (with-lock (io-lock :whostate "Await IO")
    (lexpr-continue-whopper args)))

(defmethod (actor-io-mixin :remark) (format-string &rest format-args)
  (send self ':prepare-remark-window remark-window)
  (lexpr-funcall #'format remark-window format-string format-args))

(defmethod (actor-io-mixin :prepare-remark-window) (w)
  (send w ':clear-screen))

(define-interpretation (actor-io-mixin remark) (string)
  (send self ':remark "~A" string))

(comment ;not needed? -kmp 4/4/84

(define-behavior (actor-io-mixin remark) (string)
  (send self ':remark "~A" string))

);tnemmoc

;;; :TYPEOUT format-string ...format-args...

(defwhopper (actor-io-mixin :typeout) (&rest args)
  (with-lock (io-lock :whostate "Await IO")
```

```
    (lexpr-continue-whopper args)))

(defmethod (actor-io-mixin :typeout) (format-string &rest format-args)
  (send self ':prepare-typeout-window typeout-window)
  (lexpr-funcall #'format typeout-window format-string format-args))

(defmethod (actor-io-mixin :prepare-typeout-window) (w)
  (send w ':clear-screen))

(define-interpretation (actor-io-mixin typeout) (text)
  (send self ':typeout "~A" text))

(comment ;not needed? -kmp 4/8/84

(define-behavior (actor-io-mixin typeout) (string)
  (send self ':typeout "~A" string))

);tnemmoc
```

```
;;;; Actor Process Window Mixin

(defflavor actor-background-process-mixin ((process-window nil)
                                           (process? t)
                                           (priority 0))
          ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

;;; :AFTER :INIT
;;;
;;;  Assures that a window with a process gets created if one
;;;  was not given as an init option.

(defmethod (actor-background-process-mixin :after :init) (&rest ignore)
  (when (and process? (not process-window))
    (setq process-window
          (tv:make-window 'actor-window
                          ':name (text "Background KBE: ~A"
                                       (send self ':short-name))
                          ':delegate self
                          ':priority priority))
    (send process-window ':run-reason self)))

;;; :PROCESS
;;;
;;;  Returns the associtaed process,
;;;   or NIL if there is no associated process.

(defmethod (actor-background-process-mixin :process) ()
  (send (send self ':process-window) ':process))

;;; :WHOSTATE
;;;
;;;  Returns the whostate of the associated process,
;;;   or NIL if there is no associated process.

(defmethod (actor-background-process-mixin :whostate) ()
  (let ((p (send self ':process)))
    (if p (send p ':whostate))))

;;; :PROCESS-STATE
;;;
;;;  Returns two values describing this actor's internal state:
;;;
;;;   (1) A string describing this actor's state.
;;;
;;;   (2) A flag saying whether this string is likely to be interesting
;;;       to the user if he hasn't specifically requested it. This
;;;       flag is NIL if the actor is 'happy' (active or bored) and T
;;;       otherwise (meaning the actor is blocked in some state that may
;;;       require manual intervention to clear).

 ;; This is actually subject to a timing screw since the stack group might
 ;; change between the time we poll for a whostate and the time we check
 ;; the stack group, but I don't know the right primitives to get around
 ;; this problem and since the information is never used by a program,
 ;; (ie, is only intended to be heuristically interesting to human users)
 ;; I'm going to not worry about that for now. --KMP

(defvar *unknown-process-states* nil "For debugging only.")

(defmethod (actor-background-process-mixin :process-state) ()
  (let ((process (send self ':process)))
    (let ((state (send process ':whostate)))
      (cond ((equal state "Bored")
             (values "bored" nil))
```

```
            ((equal state "RUN")
             (values "active" nil))
            ((equal state "Lock")
             (values "blocked awaiting some resource" nil))
            ((or (%string-equal state 0 "Await" 0 5)
                 (%string-equal state 0 "Avoid" 0 5))
             (values (text "blocked to ~A" (string-downcase state))
                     nil))
            ((or (%string-equal state 0 "CONNECT" 0 7)
                 (%string-equal state 0 "NET" 0 3)
                 (%string-equal state 0 "FILE" 0 4))
             (values "negotiating with the Chaosnet" nil))
            ((equal state "TYI")
             (values "blocked awaiting keyboard input" t))
            ((sys:sg-previous-stack-group (send process ':stack-group))
             ;; We assume here that our actor runs in a toplevel stack
             ;; group and if he's in any recursive stack group, it must
             ;; be a debugger stack group. This turns out to be a poor
             ;; assumption, but it's better than nothing...
             (cond ((equal state "Selected")
                    (values "waiting to be debugged" t))
                   (t
                    (values "being interactively debugged" t))))
            (t
             (push state *unknown-process-states*)
             (values (format nil "blocked in state ~S" state) t))))))

;;; :RESET
;;;
;;;  Resets this actor's process.

(defmethod (actor-background-process-mixin :reset) ()
  (send process-window ':reset)
  t)

;;; :SELECT
;;;
;;;  Selects some window associated with this actor.

(defmethod (actor-background-process-mixin :select) ()
  (send process-window ':select))
```

```
;;;; Members of Society

;;; SOCIETY-MEMBER-MIXIN

(defflavor society-member-mixin (society) ()
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :society *society*))

;;; :INIT-SOCIETY
;;;
;;;  This message allows a hook so that an actor can decide to let
;;;  its associated society know that it is a member or not. Most
;;;  kinds of actors will just want to insert themselves into the
;;;  society. Others which like to remain anonymous, might redefine
;;;  this method to be a no-op.

(defmethod (society-member-mixin :after :init) (&rest ignore)
  (send society ':insert self))

;;; :NAME? name
;;;
;;;  Each society member must answer this message so that the whole
;;;  society can be polled for such a member.

(defmethod (society-member-mixin :name?) (test-name)
  (or (string-equal test-name (send self ':name))
      (string-equal test-name (send self ':short-name))))
```

```
;;;; Actors

;;; ESSENTIAL-ACTOR
;;;
;;;  This flavor has a bunch of common things that most actors want.

(defflavor essential-actor () (society-member-mixin actor))

(defflavor basic-actor ()
           (actor-io-mixin
            believer-mixin
            default-vocabulary
            essential-actor))

(define-system-initialization default-delegate
  (setq *delegate* (make-instance 'basic-actor)))

;;; ANSWER

(define-behavior (basic-actor answer) (data)
  (let ((text (cond ((stringp data) data)
                    ((not (atom data))
                     (get data ':text)))))
    (if text (typeout text))))

;;; COMPLAINT

(define-behavior (basic-actor complaint) (data)
  (remark (text "~@[~A: ~]Error! ~A"
                (send self ':current-client)
                data)))

;;; >Report status.
;;;
;;;  Returns a string suitable for display to the user saying what
;;;  this actor's current state is.

(define-interpretation (basic-actor report) ()
  (let ((box (send self ':inbox)))
    (let ((base 10.) (*nopoint t))
      (text "I am ~A; my inbox contains ~A."
            (send self ':process-state)
            (send box ':describe-storage)))))

;;; These used to be events but now are simple functions.

(define-behavior (basic-actor suggest-code-home) (code-name where)
  (if (not (belief ':suggested-code-home code-name))
      (believe ':suggested-code-home code-name where)))

(define-behavior (basic-actor suggested-code-home) (code-name)
  (return (belief ':suggested-code-home code-name)))
```

```
;;; Standard Actor
;;;
;;;  Like Basic Actor, but with a process.

(defflavor standard-actor ()
           (actor-background-process-mixin basic-actor))
```

```
;;;; For Employers

(define-vocabulary employer-vocabulary (&rest deep-structure)
  (send self ':defer-task (unparse-command deep-structure)))

(defflavor employer-mixin ((associated-apprentice nil))
           (employer-vocabulary)
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables)

(define-behavior (employer-mixin associated-apprentice) ()
  (return (send self ':associated-apprentice)))

;;; TASK
;;;
;;;  Handles requests by submitting them to its apprentice.

(define-behavior (employer-mixin task) (request)
  (return (eval-command (parse-command request))))

(defmethod (employer-mixin :submit-by-proxy) (text &optional await?)
  await? ;ignored
  (send (send self ':inbox) ':send-locked self
        ':submit-by-proxy-internal text await?))

(defmethod (employer-mixin :submit-by-proxy-internal) (text await?)
  (let ((id-list (mapcar #'(lambda (command)
                             (send self ':defer-task
                                   (unparse-command command) nil))
                         (read-commands-from-string text))))
    (when await?
      (let ((my-inbox (send self ':inbox)))
        (dolist (id id-list)
          (answer (send my-inbox ':await-message id)))))))

(defmethod (employer-mixin :defer-task) (request &optional (await? t))
  (let ((apprentice (send self ':find-apprentice nil)))
    (cond ((not apprentice)
           (complain "Can't find an apprentice!"))
          ((not await?)
           (tell apprentice task request))
          (t
           (ask apprentice task request)))))

(defun reset-apprentice () ;for debugging
  (let ((obj (parse-actor (associated-apprentice))))
    (cond ((not obj) "No associated apprentice to reset.")
          ((not *debug*)
           (text "Not going to reset ~A because not in debug mode." obj))
          ((send-if-handles obj ':reset) (text "~A has been reset." obj))
          (t (text "~A could not be reset." obj)))))

(defun select-apprentice () ;for debugging
  (let ((obj (parse-actor (associated-apprentice))))
    (cond ((not obj) "No associated apprentice to select.")
          ((not *debug*)
           (text "Not going to select ~A because not in debug mode." obj))
          ((send-if-handles obj ':select) (text "Selecting ~A..." obj))
          (t (text "~A could not be selected." obj)))))

;;; :REMARK
;;; :TYPEOUT
;;;
;;;  %%% Message Passing %%%
;;;  These whoppers have to do with the fact that these methods sometimes
;;;  get called from the wrong process. That's something dirty which
;;;  should be cleaned up one day. --kmp
```

```
(defwhopper (employer-mixin :remark) (&rest args)
  (with-user-package (lexpr-continue-whopper args)))

(defwhopper (employer-mixin :typeout) (&rest args)
  (with-user-package (lexpr-continue-whopper args)))

;;; (EMPLOYER-MIXIN :FIND-APPRENTICE) &rest specs
;;;  Should find an appropriate apprentice, or NIL if none available.
;;;  For now, let's cheat and just know a reasonable name.

(define-KBE-option *default-apprentice*
                   "Name of default apprentice"
  "Apprentice 1" :sexp)

(defmethod (employer-mixin :find-apprentice) (&optional specs)
  specs ;ignored, for now at least
  (or associated-apprentice
      (setq associated-apprentice *default-apprentice*)))

;;; :WARN format-string format-args...
;;;
;;;  This message is sent to warn about an unusual but non-fatal
;;;  condition.

(defmethod (employer-mixin :warn) (from string)
  (send self ':remark "~A: ~A" from string))
```

```
;;;; Alternate Window Type

(defflavor actor-window ((priority 0))
           (delegate-mixin
            associated-process-mixin
            tv:window)
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
     :save-bits t
     :activate-p t
     :deexposed-typeout-action ':permit
     :more-p nil))

(defmethod (actor-window :run-reason) (reason)
  (send (send self ':process) ':run-reason reason))

(defmethod (actor-window :reset) ()
  (send (send self ':process) ':reset))

(defmethod (actor-window :process-toplevel) ()
  (send current-process ':set-priority priority)
  (with-delegate (send self ':delegate)
    (send-delegate ':read-messages)))

(defmethod (actor-window :selectable-windows) ()
  (if *debug* (list (list (send self ':name-for-selection) self))))

(defmethod (actor-window :name-for-selection) ()
  (send self ':name))

(compile-flavor-methods essential-actor
                        basic-actor
                        standard-actor
                        actor-window)
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; Lisp WITH-LOCK Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; REMOTE: Module for hacking apprentices, etc. from afar.
;;;
;;; This isn't the cleanest possible implementation, but it has come
;;; a long way...
```

```
;;;; Remote Actor Support

;;; REMOTE-ACTOR-MIXIN
;;;
;;;  All remote actors want this mixin. It defines the essential
;;;  ingredients that will allow the actor to talk via the Chaos
;;;  net to a remote counterpart.

(defflavor remote-actor-mixin (host local-name) ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)
```

```
;;;; Remote Actors

;;; REMOTE-ACTOR
;;;
;;;  This is a full scale actor acting as a go-between to an actor
;;;  on a remote host. It is part of the local society structure,
;;;  so receives all blast messages sent via the society network.

(defflavor remote-actor () (actor-background-process-mixin
                            remote-actor-mixin
                            essential-actor))

;;;   These are messages that will want to be handled remotely.

(comment ;revised to use different strategy. -kmp 2/22/84

(defmethod (remote-actor :process-message) (message)
  (condition-case (cond)
      (kaos-send (send self ':host) ':local-KBE-dispatch
                 (send self ':local-name) ':queue-message message)
    (sys:error (complain "~A" cond))))

)

(defmethod (remote-actor :process-message) (message)
  (condition-case (cond)
      (kaos-send (send self ':host) ':remote-service
                 (send si:local-host ':name)
                 (send self ':local-name)
                 message)
    (sys:error (complain "~A" cond))))
```

```
;;;; Network Handlers

;;; :DELIVER-MESSAGE
;;;
;;;  This handler puts a given message in a mailbox.

(define-KBE-option *allow-KBE-remote*
                   "KBE remote communication enabled"
  t :boolean)

(defvar *notification-data* '() "Data about who knows about servers.")

(defun remote-service-notify (host)
  (when (or (not user-id) (equal user-id ""))
    (setq user-id "KBE"))
  (when (not *allow-KBE-remote*)
    (complain "Sorry, ~A has disabled remote KBE communication."
              user-id))
  (let ((data (ass #'string-equal user-id *notification-data*)))
    (when (not data)
      (setq data (ncons user-id))
      (push data *notification-data*))
    (when (not (mem #'string-equal host (cdr data)))
      (tv:notify nil "The KBE is being used remotely from ~A." host)
      (push host (cdr data)))))

(define-kaos-handler :remote-service (host destination data)
  (remote-service-notify host)
  (deliver destination (make-message data)))

;;; :LOCAL-KBE-DISPATCH local-name msg &rest args
;;;
;;;  This handler sends the given message with given args to the local
;;;  actor with the given LOCAL-NAME, returning the result(s).

(comment ;no longer needed? -kmp 2/22/84

(define-kaos-handler :local-KBE-dispatch (to-name msg &rest args)
  (let ((actor (parse-actor to-name)))
    (if actor
        (with-delegate actor ;Still needed with message passing?-kmp 2/8/84
          (lexpr-send actor msg args)))))

)
```

```
;;;; Making Remote Actors

;;; (PARSE-REMOTE-ACTOR host name)
;;;
;;;  No remote actors are initially set up, but they can be created
;;;  with this function. eg,
;;;    (PARSE-REMOTE-ACTOR "Magic-1@MIT-LISPM-47")
;;;  finds or creates a remote actor locally which will try to talk
;;;  to an actor named Magic-1 at MIT-LISPM-47.
;;;
;;;  It will not create the actor on that machine. That happens upon
;;;  demand when contacting the machine.
;;;
;;;  If the host component of the name designates the local machine,
;;;  a normal local actor (not a remote-actor) will be returned.
;;;
;;;  This will try to handle local actors, too, if it thinks it is
;;;  not infinitely recursing.

(defun parse-remote-actor (name)
  (multiple-value-bind (actor-name host-name remote?)
      (parse-actor-full-name name)
    (cond (remote?
           (let ((full-name (make-actor-full-name actor-name host-name)))
             (or (send-any-that-handles ':name? full-name)
                 (make-instance 'remote-actor
                                ':name full-name
                                ':host host-name
                                ':local-name actor-name))))
          ;; Iff name seems to be shrinking in size, recurse...
          ((< (string-length actor-name) (string-length name))
           (parse-actor actor-name)))))
```

```
;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods remote-actor)

;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:TV; Base:10; Fonts:MEDFNT; -*-

;;; (c) 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; This file defines an abstraction for dealing with a window with mouse
;;; sensitive regions, some of which may be (but are not constrained to be)
;;; overlapping.

;;; TV:OVERLAPPING-REGIONS-MIXIN (mixin)
;;;
;;;  This is the base mixin which supports the overlapping regions.
;;;  It is built out of TV:ANY-TYI-MIXIN. Here's a description of the
;;;  instance variables...
;;;
;;;  REGIONS
;;;
;;;   By default, this is NIL, meaning the window has no mouse sensitive regions.
;;;   It may be set (with :REGIONS init option or with a :SET-REGIONS
;;;   message later) to be an alist of the form:
;;;    ((coordinates1 . data1) (coordinates2 . data2) (coordinates3 . data3) ...)
;;;   where coordinates have the form:
;;;    (x1 y1 x2 y2)
;;;   and item has the form of a normal item as in other flavors using regions.
;;;
;;;  MOUSE-REGION
;;;
;;;   This variable holds the region currently under the mouse, or NIL if no
;;;   region is under the mouse. You can use the :MOUSE-REGION message to read its
;;;   value. The :MOUSE-REGION init options and :SET-MOUSE-REGION message are
;;;   provided for debugging consistency, but won't have any useful effect, so you
;;;   should probably pretend they don't exist unless you've carefully studied
;;;   the code and understand what they do.
;;;
;;;  MOUSE-REGION-BLINKER
;;;
;;;   Most users will be happy with the default, which is a hollow rectangular
;;;   blinker. This will be used to highlight the selected area. If you want
;;;   another kind of blinker, you can use the :MOUSE-REGION-BLINKER init
;;;   message or :SET-MOUSE-REGION-BLINKER message to set this.
;;;
;;;  REGION-FILTER
;;;
;;;   If not NIL, this should be a function of one argument which takes an
;;;   region which is about to be returned by :MOUSE-SENSITIVE-REGION
;;;   and returns true if acceptable and false if not. This allows one to place
;;;   constraints on which of the possible regions in the REGIONS are active
;;;   at any given time. The default is NIL, meaning no filtering is done.
;;;
;;;  WHO-LINE-DOCUMENTATION-STRING
;;;
;;;   Lisp will check the window under the mouse periodically (actually, quite
;;;   often) to see if this has changed. If it has, it will update the mouse
;;;   documentation line. You can read the window's documentation yourself with
;;;   the :WHO-LINE-DOCUMENTATION-STRING message or set it with the
;;;   :SET-WHO-LINE-DOCUMENTATION-STRING message or :WHO-LINE-DOCUMENTATION-STRING
;;;   init option. In normal use, this isn't necessary. See whe
;;;   DOCUMENTATION-HANDLER function.
;;;
;;;  DOCUMENTATION-HANDLER
;;;
;;;   Readable and settable in the obvious ways. If this variable is NIL, then
;;;   the documentation for the window won't change except by explicit calls to
```

```
;;;   :SET-WHO-LINE-DOCUMENTATION-STRING. However, if it is non-NIL, it can be
;;;   a handler function which is called when the mouse moves, receiving the
;;;   new region (or NIL) as an arg and returning new mouse documentation.
;;;   If you allocate an element of the region list as reserved for documentation,
;;;   this can be very fast. Eg, if your regions are of the form
;;;   (coordinates object doc) then you can use #'CADDR as your
;;;   DOCUMENTATION-HANDLER.
```

```
;;;; Overlapping Regions Mixin

;;; TV:OVERLAPPING-REGIONS-MIXIN
;;;
;;;  Basic definition of how Overlapping Regions work.

(defflavor overlapping-regions-mixin (mouse-region-blinker
                                      mouse-region
                                      regions
                                      region-filter
                                      documentation-handler
                                      who-line-documentation-string)
          (any-tyi-mixin)
  (:required-flavors tv:sheet)
  (:documentation :mixin
   "An abstraction for using possibly-overlapping mouse sensitive regions")
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :who-line-documentation-string nil
    :documentation-handler nil
    :regions              nil
    :mouse-region         nil
    :mouse-region-blinker nil
    :region-filter        nil))

(defmethod (overlapping-regions-mixin :after :init) (&rest ignore)
  (unless mouse-region-blinker
    (setq mouse-region-blinker
          (make-blinker self 'hollow-rectangular-blinker ':visibility nil))))

;;; :MOUSE-CLICK button x y
;;; :MOUSE-MOVES x y
;;;
;;;  These two messages get sent by the window system whenever the mouse is
;;;  played with. This window abstraction just builds on those messages.

(defmethod (overlapping-regions-mixin :mouse-click) (button x y)
  (let ((inside-x (- x (sheet-inside-left)))
        (inside-y (- y (sheet-inside-top))))
    (send self ':select-region
          (send self ':mouse-sensitive-region inside-x inside-y)
          button)))

(defmethod (overlapping-regions-mixin :mouse-moves) (x y)
  (mouse-set-blinker-cursorpos) ;Move mouse blinker
  (let ((inside-x (- x (sheet-inside-left)))
        (inside-y (- y (sheet-inside-top))))
    (multiple-value-bind (new-region)
        (send self ':mouse-sensitive-region inside-x inside-y)
      (when (neq mouse-region new-region)
        (setq mouse-region new-region)
        (send self ':move-mouse-region new-region)))))

;;; :WHOPPER :HANDLE-MOUSE
;;;
;;; This corrects half of a bug that is caused when another window pops up over
;;; the top of the current one, allowing the mouse to "move" off of the current
;;; window without changing its x-y coordinates. The half it fixes is where the
;;; mouse moves off of the window; it does not fix the case of the mouse moving
;;; onto the window because we don't know how to reliably read the mouse x-y
;;; information.

(defwhopper (overlapping-regions-mixin :handle-mouse) (&rest args)
  (values-list (prog1 (multiple-value-list (lexpr-continue-whopper args))
                      (when mouse-region
                        (setq mouse-region nil)
```

```
                    (send self ':move-mouse-region nil)))))

;;; :CLEAR-SCREEN

(defmethod (overlapping-regions-mixin :before :clear-screen) (&rest ignore)
  (setq regions nil)
  (move-blinker mouse-region-blinker nil))

;;; :MOUSE-SENSITIVE-REGION x y
;;;
;;;  Given inside X and Y coordinates, this function returns the active region
;;;  from the REGIONS corresponding to X and Y. That region is the first
;;;  region in the alist which contains the point (X,Y) and which is not filtered
;;;  out by the REGION-FILTER if one exists.

(defmethod (overlapping-regions-mixin :mouse-sensitive-region) (inside-x inside-y)
  (dolist (region regions)
    (let ((coordinates (car region)))
      (when (and (>= inside-x (nth 0 coordinates))
                 (>= inside-y (nth 1 coordinates))
                 (<= inside-x (nth 2 coordinates))
                 (<= inside-y (nth 3 coordinates))
                 (or (not region-filter)
                     (funcall region-filter region)))
        (return region)))))

;;; :SELECT-REGION region button
;;;
;;;  Whenever a button is clicked, this method gets sent the region and
;;;  the button. If there is no associated region, it gets NIL instead.
;;;
;;;  Flavors built on this flavor may wish to redefine this.

(defmethod (overlapping-regions-mixin :select-region) (region button)
  (when region
    (send self ':force-kbd-input '(:mouse-click ,region ,button ,self))))

;;; :MOVE-MOUSE-REGION region
;;;
;;;  Given an region from the REGIONS (or NIL meaning to deselect the
;;;  active region), this sets the mouse blinkers appropriately.
;;;
;;;  This may be a good function to define :BEFORE and :AFTER daemons
;;;  for when building flavors on this mixin. It is called only when the
;;;  mouse-region has actually changed.

(defmethod (overlapping-regions-mixin :move-mouse-region) (region)
  (move-blinker mouse-region-blinker region))

(defun-method move-blinker overlapping-regions-mixin (blinker region)
  (unless (not blinker) ;just in case user kills the blinker
    (cond (region
           (let ((coordinates (car region)))
             (let ((x1 (nth 0 coordinates))
                   (y1 (nth 1 coordinates))
                   (x2 (nth 2 coordinates))
                   (y2 (nth 3 coordinates)))
               (blinker-set-cursorpos blinker x1 y1)
               (blinker-set-size blinker (- x2 x1) (- y2 y1))
               (blinker-set-visibility blinker t))))
          (t
           (blinker-set-visibility blinker nil)))
    (send self ':set-who-line-documentation-string
          (if documentation-handler
              (funcall documentation-handler region)))))
```

```
;;;; Overlapping Regions Window/Pane

;;; TV:OVERLAPPING-REGIONS-WINDOW
;;;
;;;  This is just a TV:WINDOW with TV:OVERLAPPING-REGIONS-MIXIN.

(defflavor overlapping-regions-window () (overlapping-regions-mixin window)
  (:documentation
   "A window which handles possibly-overlapping mouse sensitive regions"))

;;; TV:OVERLAPPING-REGIONS-PANE
;;;
;;;  This is just adds TV:PANE-MIXIN to a TV:OVERLAPPING-REGIONS-WINDOW.

(defflavor overlapping-regions-pane () (pane-mixin overlapping-regions-window)
  (:documentation
   "A window pane which handles possibly-overlapping mouse sensitive regions"))

(compile-flavor-methods overlapping-regions-window overlapping-regions-pane)
```

```
;;; -*- Mode:LISP; Package:TV; Base:10; Fonts:MEDFNT; -*-

;;; (c) 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; This file defines an abstraction for dealing with a window with mouse
;;; sensitive output to a window (built on OVERLAPPING-REGIONS-WINDOW.
;;;
;;; A TV:MOUSE-SENSITIVE-REGIONS window is intended to have the following
;;; style of interface:
;;;
;;;  (SETQ X
;;;    (TV:MAKE-WINDOW 'TV:MOUSE-SENSITIVE-REGIONS-WINDOW ':EDGES-FROM ':MOUSE))
;;;
;;;  (FORMAT X "Hi, ~M is mouse sensitive" "foo")
;;;
;;;  (SEND X ':CHOOSE) ;left click on "foo"
;;;   => "foo", :LEFT
;;;
;;; Here's another example:
;;;
;;;  (FORMAT X "~%~M"
;;;    '(("This whole ~M is ~M and so are some of its ~M."
;;;       ("sentence" SENTENCE "Click here for word 'sentence'")
;;;       ("mouse sensitive" MOUSE-SENSITIVE "Click here for 'mouse sensitive'")
;;;       ("words" WORDS "Click here for 'words'"))
;;;      WHOLE-SENTENCE
;;;      "Click here for whole sentence"))
;;;
;;;  (SEND X ':CHOOSE) ;middle click on "This whole ... its words"
;;;   => WHOLE-SENTENCE, :MIDDLE
;;;
;;; The format command this sets up is ~M. It works on non-mouse-sensitive
;;; windows, but doesn't make its output mouse-sensitive.
;;;
;;; NOTE: Currently, ~M items must not extend over more than one line. This
;;;   is fixable in principle, but I haven't gotten to it yet.
;;;
;;; ~M takes one argument.
;;;
;;;   If that argument is an atom, it is displayed with PRINC.
;;;    The result of :CHOOSE on this thing will be the atom itself.
;;;    The mouse documentation will say something vanilla about clicking to select.
;;;
;;;   If it is a list, its car will be displayed with PRINC.
;;;    The result of :CHOOSE will be the cadr of the list.
;;;    The mouse documentation will be the caddr of the list or something
;;;     vanilla about clicking to select otherwise.
;;;
;;; ~:M also takes one argument. It allows recursive entries.
;;;
;;;   The argument must be a list.
;;;    The car of the list must be a list of form (format-string . format-args).
;;;    The result of :CHOOSE will be the cadr of the list.
;;;    The mouse documentation will be the caddr of the list or something
;;;     vanilla about clicking to select otherwise.
```

```
(defflavor mouse-sensitive-regions-mixin () ()
  (:required-flavors overlapping-regions-mixin minimum-window)
  (:default-init-plist
    :blinker-p nil
    :save-bits t
    :region-filter #'mouse-sensitive-regions-mixin-filter
    :documentation-handler #'caddr))

(defmethod (mouse-sensitive-regions-mixin :after :end-of-page-exception)
           (&rest ignore)
  (let ((new-regions nil))
    (dolist (region (send self ':regions))
      (let ((marker-cell (nthcdr 4 (car region))))
        (when (not (car marker-cell))
          (setf (car marker-cell) t)
          (push region new-regions))))
    (send self ':set-regions new-regions)))

(defwhopper (mouse-sensitive-regions-mixin :mouse-sensitive-region) (&rest stuff)
  (multiple-value-bind (ignore y)
      (send self ':read-cursorpos)
    (let ((*y-pos* (+ y (send self ':line-height))))
      (declare (special *y-pos*))
      (lexpr-continue-whopper stuff))))

(defun mouse-sensitive-regions-mixin-filter (x)
  (declare (special *y-pos*))
  (let ((coordinates (car x)))
    (or (< *y-pos* (nth 3 coordinates))
        (not (nth 4 coordinates)))))

(defflavor mouse-sensitive-regions-window ()
           (mouse-sensitive-regions-mixin
            overlapping-regions-window))

(defflavor mouse-sensitive-regions-pane ()
           (mouse-sensitive-regions-mixin
            overlapping-regions-pane))

(defmethod (mouse-sensitive-regions-mixin :choose) ()
  (do ((blip (send self ':any-tyi) (send self ':any-tyi)))
      (nil) ;loop forever
    (cond ((and (not (atom blip))
                (eq (car blip) ':mouse-click))
           (let ((region (cadr  blip))
                 (button (caddr blip)))
             (caseq button
               ((#\mouse-r-1) (return (cadr region) ':right))
               ((#\mouse-m-1) (return (cadr region) ':middle))
               ((#\mouse-l-1) (return (cadr region) ':left))
               ((#\mouse-r-2) (mouse-call-system-menu))
               (t (beep)))))
          (t (beep)))))

;;;; Interaction using FORMAT.

;;; (WITH-MOUSE-SENSITIVE-REGION window . body)
;;;
;;;  This is a sub-primitive for implementing the format option described below.
;;;  It lets you output to a window with MOUSE-SENSITIVE-REGIONS-MIXIN
;;;  making the region mouse-sensitive. The body of the form should do output
;;;  to STANDARD-OUTPUT.
;;;
;;;  The return values will become the entries in the region. eg,
;;;  if you return the values 3, 4 then (coordinates 3 4) will be the
;;;  region data.
```

```
(defmacro with-mouse-sensitive-region (window &body body)
  (let ((start-x (gensym))
        (start-y (gensym)))
    '(let ((standard-output ,window))
       (multiple-value-bind (,start-x ,start-y)
           (send standard-output ':read-cursorpos)
         (let ((data (multiple-value-list (progn ,@body))))
           (when (typep standard-output 'mouse-sensitive-regions-mixin)
             (multiple-value-bind (end-x end-y)
                 (send standard-output ':read-cursorpos)
               (cond ((> end-y ,start-y) ;output extends over multiple lines?
                      (ferror "We can't handle multi-line mouse things yet."))
                     (t
                      (send standard-output
                            ':set-regions
                            (append
                              (send standard-output ':regions)
                              (ncons
                                (cons (list ,start-x ,start-y end-x
                                            (+ end-y ;bottom of line, not top
                                               (send standard-output
                                                     ':line-height))
                                            nil)
                                      data))))))))))
           (values-list data))))))

(defprop format:m mouse-sensitive-output format:format-ctl-one-arg)

(defun mouse-sensitive-output (obj ignore)
  (with-mouse-sensitive-region standard-output
    (let ((documentation "Any button to select this item.")
          item)
      (cond (format:colon-flag
             (lexpr-funcall #'format t (car obj))
             (setq item (or (cadr obj) item))
             (setq item (cadr obj)
                   documentation (or (caddr obj) documentation)))
            (t
             (cond ((atom obj)
                    (princ obj)
                    (setq item obj))
                   (t
                    (princ (car obj))
                    (setq item (cadr obj)
                          documentation (or (caddr obj) documentation))))))
      (values item documentation))))
```

```
(compile-flavor-methods mouse-sensitive-regions-window
                        mouse-sensitive-regions-pane)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

(defflavor options-window (display-handler-function)
           (associated-process-mixin
            tv:mouse-sensitive-regions-window)
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables
  (:default-init-plist
    :display-handler-function nil))

(defmethod (options-window :redisplay) ()
  (if display-handler-function
      (funcall display-handler-function self)))

(defmethod (options-window :after :refresh) (&rest ignore)
  (send self ':redisplay))

(defmethod (options-window :process-toplevel) ()
  (multiple-value-bind (choice button)
      (send self ':choose)
    (process-run-function (format nil "Option: ~A" choice)
                          #'(lambda (choice window button)
                              (with-user-package
                                (funcall choice window button)))
                          choice
                          terminal-io
                          button)))

(defflavor options-pane () (simple-pane-mixin options-window)
  (:default-init-plist
    :activate-p t
    :deexposed-typeout-action ':permit
    :display-handler-function 'display-options
    :blinker-p nil
    :more-p nil
    :label nil))

(defmethod (options-pane :after :activate) (&rest ignore)
  (send (send self ':process) ':run-reason self))

(defvar *options* '() "List of options for standard options window.")

(defun maybe-display-options-sample (window button)
  (cond ((and *debug* (eq button ':right))
         (debug-options-window window button))
        (t (display-options-sample window button))))

(defun display-options-sample (window button)
  window ;ignored
  (repeat ()
    (return-if
      (tv:menu-choose
        (list (format nil "Signed, ~A" user-id))
        (format nil "I, ~A, do hereby promise that I will faithfully~
                     ~%read and heed the mouse documentation line and~
                     ~%not click ~A on things I have warned not to~
                     ~%click on.~
                     ~%            Click to confirm:"
                user-id (string-downcase button))))))
```

```
(defun debug-options-window (window button)
  button ;ignored
  (unwind-protect (let ((terminal-io window))
                    (send terminal-io ':clear-screen)
                    (send terminal-io ':select)
                    (format t
                            "Remember to reselect another pane manually.")
                    (break "Debug Options"))
                  (send window ':refresh)))

(defun display-options (&optional (standard-output standard-output))
  (send standard-output ':clear-screen)
  (format t "By touching ~M with the mouse, you can ...~
        ~{~% ^@ ~{~:[~A~;~M~]~}.~}"
          '("parts of this window" maybe-display-options-sample
            "Not here! Don't take the words too literally; touch the text below.")
          (mapcar #'cdr *options*)))

(defmacro define-option-line (name &body data)
  '(let ((entry (assq ',name *options*))
         (data  ',(mapcon #'(lambda (x)
                              (list (not (atom (car x)))
                                    (car x)))
                          data)))
     (cond (entry (setf (cdr entry) data))
           (t (setq *options*
                    (nconc *options* (ncons '(,',name . ,data))))))
     ',name))

(defmacro define-option-function (name bvl &body body)
  '(defun ,name ,bvl ,@bvl
          (with-delegate (window-delegate ,(car bvl))
            ,@body)))
```

```
;;;; Apprentice-Related Options

(define-option-function apprentice-select (window button)
  window button ;ignored
  (remark (select-apprentice)))

(define-option-function apprentice-status (window button)
  window button ;ignored
  (let ((apprentice (associated-apprentice)))
    (remark (cond ((not apprentice)
                   "No apprentice yet selected for this window.")
                  (t (ask apprentice task "Report status."))))))

(define-option-function apprentice-reset (window button)
  window button ;ignored
  (remark (reset-apprentice)))

(define-option-line hack-apprentice
  ("Select the associated apprentice's window" apprentice-select) ", "
  ("poll its status"                           apprentice-status) " or "
  ("reset it"                                  apprentice-reset))
```

```
;;;; Debug Options

(define-option-function options-edit-all (window button)
  window button ;ignored
  (edit-KBE-options))

(define-option-function options-toggle-debug (window button)
  window ;ignored
  (caseq button
    ((:LEFT)   (setq *debug* (not *debug*)))
    ((:MIDDLE) (setq *debug* nil))
    ((:RIGHT)  (setq *debug* t)))
  (remark (text "Debug Mode ~:[off~;on~]." *debug*)))

(define-option-line options
  ("Edit KBE options"  options-edit-all)
  " or "
   ("toggle debug mode" options-toggle-debug
   "L:Complement Debug Mode, M:Debug Mode Off, R:Debug Mode On"))
```

```
;;;; Font display

(define-option-function fonts-any (window button)
  (selectq button
    ((:LEFT)   (fonts-small  window button))
    ((:MIDDLE) (fonts-medium window button))
    ((:RIGHT)  (fonts-large  window button))))

(define-option-function fonts-small (window button)
  button ;ignored
  (let ((display-pane (get-any-pane window 'display-pane)))
    (cond ((not display-pane) (beep))
          (t (send display-pane ':set-font-map (list fonts:cptfontb))
             (send display-pane ':refresh)
             (remark "Now using small font (CPTFONTB)."))))))

(define-option-function fonts-medium (window button)
  button ;ignored
  (let ((display-pane (get-any-pane window 'display-pane)))
    (cond ((not display-pane) (beep))
          (t (send display-pane ':set-font-map (list fonts:medfnb))
             (send display-pane ':refresh)
             (remark "Now using medium font (MEDFNB)."))))))

(define-option-function fonts-large (window button)
  button ;ignored
  (let ((display-pane (get-any-pane window 'display-pane)))
     (cond ((not display-pane) (beep))
           (t (send display-pane ':set-font-map (list fonts:bigfnt))
              (send display-pane ':refresh)
              (remark "Now using large font (BIGFNT)."))))))

(define-option-line fonts
  ("Change the display window font" fonts-any
   "L:Small (CPTFONTB), M:Medium (MEDFNB), R:Large (BIGFNT)")
  " to be "
  ("small"  fonts-small  "Select small display font. (CPTFONTB)")  ", "
  ("medium" fonts-medium "Select medium display font. (MEDFNB)")   " or "
  ("large"  fonts-large  "Select large display font. (BIGFNT)"))
```

```
;;;; Self Testing

(define-option-function utility-test (window button)
  button window ;ignored
  (let ((actor (associated-apprentice)))
    (cond ((not actor)
           (remark "No associated apprentice."))
          ((require-mouse-confirmation "Run Standard Test Sequence.")
           (tell actor task "Test yourself."))
          (t
           (remark "No test sequence selected.")))))

(define-option-function utility-init-db (window button)
  button ;ignored
  (cond ((not *debug*)
         (remark "Only works in Debug Mode."))
        ((require-mouse-confirmation "Reset entire database")
         (remark "Re-initializing KBE database...")
         (db-init)
         (remark "KBE Database has been re-initialized."))
        (t
         (remark "Request aborted. KBE Database left intact."))))

(define-option-line utility
  ("Ask the KBE to test itself" utility-test)
  ", or "
  ("re-initialize its database" utility-init-db))
```

```
;;;; Initialization

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods options-window options-pane)

;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; SIMPLE: The "Simple" Frame for the KBE.

;;; The following comment tries to give an overview of the intent of the
;;; Simple Window. Because the code is changing very fast these days, this
;;; documentation is not guaranteed to have been kept up to date. I'll
;;; periodically modify the documentation as major milestones are reached.
;;; For complete details, come talk to me or consult the code.  --KMP

;;; The Simple Frame, which can be created by typing System K
;;; is a frame which looks roughly like:
;;;
;;;    +---------------+
;;;    |               |
;;;    | Display Area  |
;;;    |               |
;;;    +---------------+
;;;    | Status Window |
;;;    +---------------+
;;;    | Command Menu  |
;;;    +---------------+
;;;    | Input Window  |
;;;    +---------------+
;;;    | Lisp Listener |
;;;    +---------------+
;;;
;;;  The purpose of the sections is as follows:
;;;
;;;  Display Area. The large [initially] empty area at the top of the
;;;    screen is reserved for displays of code in focus, and for
;;;    answering various kinds of questions specifically asked by
;;;    the user.
;;;
;;;  Status Window. There is a very small window (defaultly 1 line high)
;;;    which is used to type information of interest to this window's
;;;    session.
;;;
;;;  Command Menu. This is a menu pane filled with useful commands,
;;;    most of which can be gotten by other means, but which are useful
;;;    enough that they warrant having on the screen for easy access via
;;;    the mouse.
;;;
;;;  Input Window. This is an editor window that one can type commands
;;;    to. Typing <End> causes the commands to actually get submitted
;;;    to the apprentice for execution.
;;;
;;;  Lisp Listener. This is a normal lisp listener pane, for debugging.
```

```
;;;; SIMPLE-FRAME

;;; A constraint frame for holding onto the other KBE window
;;; frames in our demo configuration.

(defconst *status-pane-size* 1. "Default status pane height in lines")

(defflavor simple-frame ()
  (delegate-mixin
   tv:bordered-constraint-frame)
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :delegate-flavor 'simple-actor
    :deexposed-typeout-action ':permit
    :x 0 :y 0
    :borders 1
    :blinker-p nil
    :save-bits t
    :edges-from (multiple-value-list (send tv:mouse-sheet ':size))
    :activate-p t
    :panes '((display-pane  display-pane        :borders (2 1 2 1))
             (status-pane   status-pane         :borders (2 1 2 1))
             (options-pane  options-pane        :borders (2 1 2 1))
             (input-pane    editing-input-pane  :borders (6 1 2 1)
                                                :margin-widths (6 1 2 1)
                                                :scroll-bar-width 3
                                                :label nil
                                                :border-margin-width 2
                                                :font-map (fonts:bigfnt))
             (listener-pane listener-pane       :borders (2 1 2 1)))
    :constraints
       '((main . ((display-pane
                   status-pane
                   options-pane
                   input-pane
                   listener-pane)
                  ((display-pane  0.4                                 ))
                  ((status-pane   ,*status-pane-size*      :lines ))
                  ((options-pane  ,(1+ (length *options*)) :lines ))
                  ((input-pane    0.25                                ))
                  ((listener-pane :even                               ))))
         (bare  . ((display-pane
                    status-pane
                    listener-pane)
                   ((display-pane  0.33                          ))
                   ((status-pane   ,*status-pane-size* :lines ))
                   ((listener-pane :even                         )))))))

(defmethod (simple-frame :name-for-selection) ()
  (text "KBE ~A~@[: ~A~]"
        (send self ':name)
        (let ((my-label (send (get-pane self 'display-pane) ':label)))
          (if (listp my-label) ; ignore random font, vsp, etc.
              (setq my-label (nth 5 my-label))) ;ignore font
          (if (equal my-label "") nil my-label))))

(defmethod (simple-frame :remark-window) () (get-pane self 'status-pane))

(defmethod (simple-frame :typeout-window) () (get-pane self 'display-pane))

(defmethod (simple-frame :after :init) (&rest ignore)
  ;; Label this frame (using the Listener's label)
  (send (get-pane self 'listener-pane) ':set-label
        '(:bottom
          :font fonts:medfnb
          :string ,(text "KBE ~A" (send self ':name)))))
```

```
;; Select appropriate pane
(send self ':select-pane (get-pane self 'input-pane)))
```

```
;;;; DISPLAY-PANE

;;; A window which can be used for the KBE's display output.
;;; It allows for character-measured boxes and complemented regions,
;;; instead of pixel-measured ones.

(defflavor display-pane () (simple-pane-mixin code-window)
  (:default-init-plist
    :label '(:bottom :font fonts:medfnb :string "")
    :more-p nil
    :borders nil
    :blinker-p nil
    :font-map (list fonts:bigfnt)))
```

```
;;;; STATUS-PANE

;;; A window which actually controls what the KBE is doing.

(defflavor status-pane ()
   (line-truncating-mixin              ; Line overflows can be discarded
    simple-pane-mixin                  ; Normal pane stuff
    tv:window)                         ; Normal window stuff
  (:default-init-plist
    :label nil
    :more-p nil
    :blinker-p nil))
```

```
;;; EDITING-INPUT-PANE

(defflavor editing-input-pane () (tv:dont-select-with-mouse-mixin
                                  tv:pane-mixin
                                  associated-process-mixin
                                  edit-window-input-commands-mixin
                                  edit-window))

 ;this new version assumes that End is set up with something that submits
 ;buffer for us.
(defmethod (editing-input-pane :process-toplevel) ()
  (with-delegate (window-delegate self)
    (with-user-package ;temp fix
      (send self ':edit)
      (beep)))) ;should never get to here.
```

```
;;;; LISTENER-PANE

;;;  Just a lisp listener with pane-mixin thrown in to make things work.

(defflavor listener-pane ()
  (tv:notification-mixin        ; normal notifications
   tv:listener-mixin            ; listener stuff
   simple-pane-mixin            ; normal pane stuff...
   tv:window)                   ; normal window stuff
  (:default-init-plist
     :selectable-p t))
```

```
;;;; Initialization

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods simple-frame
                        display-pane
                        status-pane
                        editing-input-pane
                        listener-pane)

;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; SIMPLE-ACTOR: A description of the Simple-Actor's behavior.
```

```
;;;; The Simple Actor

;;; SIMPLE-ACTOR
;;;
;;;  A simple-actor is a command interface.
;;;  It has a typein window and a menu window for command input.
;;;  It has remark and presentation windows for display of results.

(defflavor simple-actor ((focus-info nil))
           (employer-mixin standard-actor)
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :window-flavor 'simple-frame))

(define-system-initialization simple-actor
  (make-instance 'simple-actor))
```

```
;;;; Idiosyncratic Behaviors

;;; SUGGEST-FOCUS reference-list paths
;;;
;;;  This tries to change the focus in the window.

(define-behavior (simple-actor suggest-focus) (data)
  (setq focus-info data)              ;cache for later
  (send (my typeout-window) ':change-focus
        (get (locf data) ':reference)
        (cons (get (locf data) ':name)
              (get (locf data) ':paths))))

;;; CODER-CONFIRMATION
;;;
;;;  This handles code generated by the apprentice.
;;;
;;;  If the code is different than previously believed, the
;;;   belief structure is updated and the new code is displayed.
;;;   If this happens, the focus is also invalidated.

(define-behavior (simple-actor coder-confirmation) (fn code)
  (let ((w (my typeout-window))
        (old-code (send self ':belief ':code fn)))
    (when (not (code-equal old-code code))    ; Only if code has changed...
      (if (eq fn (car (send w ':focus)))      ; Invalidate any display...
          (send w ':set-focus nil))
      (send self ':believe ':code fn code))))

(define-interpretation (simple-actor clear) ()
  (send (my typeout-window) ':clear-screen))
```

```
;;;; Initialization

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods simple-actor)

;;; Local Modes:
;;; Mode:LISP
;;; Lisp WHEN Indent:1
;;; Lisp UNLESS Indent:1
;;; End:
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; APPRENTICE: A description of the Apprentice actor's behavior.
```

```
;;;; The Apprentice Actor

;;; APPRENTICE
;;;
;;;  An apprentice can work for several clients.
;;;  In addition to normal actor-world requests, apprentices
;;;  are especially good at TASK requests, which are commands
;;;  written in the KBE's pseudo-English dialect which cause the
;;;  apprentice to analyze, manipulate, or synthesize code
;;;  or documentation.

(defflavor apprentice ((focus-info '()))
           (programmer-vocabulary standard-actor)
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables
  (:default-init-plist
    :priority -1        ; Low priority so as not to interfere with user.
    ))

(defmethod (apprentice :after :init) (&rest ignore)
  (add-to-mailing-list "Programmers" (send self ':short-name) nil))

(define-system-initialization apprentice
  (make-instance 'apprentice))

(defwhopper (apprentice :event) (&rest args)
  (with-kbe-package :sigh
    (lexpr-continue-whopper args)))

;;; :PREPARE-REMARK-WINDOW  remark-window
;;; :PREPARE-TYPEOUT-WINDOW remark-window
;;;
;;;  We just want a new line before remarks and typeout.
;;;  No point in clearing any more of screen than we have to,
;;;   since just about anything might be good for debugging.

(defmethod (apprentice :prepare-remark-window) (w)
  (send w ':fresh-line))

(defmethod (apprentice :prepare-typeout-window) (w)
  (send w ':fresh-line))
```

```
;;;; Apprentice: Idiosyncratic Behaviors

;;; TASK request
;;;
;;;  This handles a text request.

(defun apprentice-eval-command (command myself)
  (bind-focus (send myself ':current-client-focus)
    (with-user-package
      (with-reply-to (send myself ':current-message)
        (eval-command command)))))

(define-behavior (apprentice task) (text)
  (return (apprentice-eval-command (parse-command text) self)))
```

```
;;;; Per-Client Focus Maintenance

;;; An apprentice may be working for more than one client.
;;; In order to keep things straight with regard to their
;;; focus info, it uses the following abstraction internally:
;;;
;;; :CURRENT-CLIENT-FOCUS
;;;
;;;   Reads the focus info for the current client.
;;;
;;; :SET-CURRENT-CLIENT-FOCUS data
;;;
;;;   Sets the focus for the current client to some given DATA.
;;;
;;; Implementation Notes:
;;;
;;;   It is not necessary to specify the amount and format
;;;   of the data. The only we guarantee here is that a bundle of
;;;   per-client data that is saved at one time can be retrieved
;;;   later by someone understanding the saver's format.
;;;
;;;   Normally a plist would be easier to manipulate than an alist
;;;   for this sort of application, but we need an alist here because
;;;   we want the EQUAL lookup of ASSOC, rather than the EQ lookup
;;;   we would get with GET/PUTPROP.

(defmethod (apprentice :current-client-focus) ()
  (cadr (assoc (send self ':current-client) focus-info)))

(defmethod (apprentice :set-current-client-focus) (data)
  (let ((client (send self ':current-client)))
    (let ((entry (assoc client focus-info)))
      (cond ((not entry)
             (push (list client data) focus-info))
            (t
             (setf (cadr entry) data))))))
```

```
;;;; Initialization

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods apprentice)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; TV-MAGIC: Miscellaneous LispM Window system utilities and other magic
```

```
;;; (GET-PANE window pane-name)
;;;
;;;  Gets the pane of window with the given name.

(defun get-pane (window pane-name)
  (send window ':get-pane pane-name))

;;; (GET-ANY-PANE window pane-name)
;;;
;;;  Like GET-PANE but is willing to search upward in the window
;;;  hierarchy if the given window doesn't have such a frame
;;;  in case one of its superiors has the given pane instead.

(defun get-any-pane (window pane-name)
  (send-any-superior-if-handles window ':get-pane pane-name))

;;; (REQUIRE-MOUSE-CONFIRMATION message [window])
;;;
;;;  Does a pop-up type query of a given message, returning T or NIL.
;;;  If window is given, the pop-up window tries to be near that window.

(defun require-mouse-confirmation (message &optional window)
  (cond ((tv:menu-choose (list message)
                         "Confirm:"
                         (cond (window (list ':window window))
                               (t '(:mouse))))
         t)))
```

```
;;;; Simple Pane Mixin

;;; SIMPLE-PANE-MIXIN

;;; This mixin collects useful commonalities of the windows we want in our
;;; simple frame.

(defflavor simple-pane-mixin (selectable-p) (tv:pane-mixin)
  (:documentation :mixin "Pane mixin plus some stylized defaults.")
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :border-margin-width 2
    :selectable-p nil
    :borders 1
    :reverse-video-p nil
    :more-p t
    :activate-p t
    :blinker-p t
    :save-bits t
    :font-map (list fonts:medfnb)
    :deexposed-typeout-action ':permit))

(defmethod (simple-pane-mixin :name-for-selection) () nil)

(defmethod (simple-pane-mixin :mouse-select) (&rest ignore)
  (cond (selectable-p (send self ':select))
        (t (send self ':beep))))
```

```
;;;; Command Menu Mixin

;;; COMMAND-MENU-MIXIN
;;;
;;;  This is a mixin which makes a window into a "command menu".

(defflavor command-menu-mixin () (tv:basic-menu tv:basic-scroll-bar)
  (:documentation :mixin
     "Any mouse input is put in the window's IO buffer")
  (:required-methods :io-buffer :chosen-item :set-chosen-item))

;;; :AFTER :MOUSE-BUTTONS
;;;
;;;  This is a hack that takes any mouse input and arranges to have it
;;;  end up in the io-buffer of the associated command pane. This allows
;;;  a process to simply TYI from the window to get input.
;;;
;;;  TV:COMMAND-MENU uses this tactic, which is where we got the idea.

(defmethod (command-menu-mixin :after :mouse-buttons) (bd ignore ignore)
  (let ((buffer (send self ':io-buffer))
        (item   (send self ':chosen-item)))
    (when item
      (tv:io-buffer-put buffer '(:menu ,item ,bd ,self))
      (send self ':set-chosen-item nil))))
```

```
;;;; Margin Scroll Bar Mixin

;;; MARGIN-SCROLL-BAR-MIXIN
;;;
;;;  This mixin causes a window to have a continuously displayed
;;;  scroll bar imbedded in its margin.
;;;
;;;  The :MARGIN-WIDTHS is used to calculate an appropriate sized
;;;  scroll bar. It does not actually set the size of the borders.
;;;  It is necessary to use this in conjunction with a :BORDERS
;;;  message in order to win totally.

(defflavor margin-scroll-bar-mixin (margin-widths scroll-bar-width) ()
  (:documentation :mixin "Scroll bar is part of margin.")
  (:required-flavors tv:borders-mixin)
  (:required-instance-variables
     tv:scroll-bar tv:scroll-bar-always-displayed)
  :initable-instance-variables
  (:default-init-plist
    :margin-widths '(6 2 2 2)
    :scroll-bar-width 2))

;;; :AFTER :ACTIVATE
;;;
;;;  This is hacked in at a place where it will probably work. It is not
;;;  really the right time, but I don't know what is. Anyway, it seems to
;;;  do the job. The problem seems to be that sending a window a
;;;  :SET-SCROLL-BAR message doesn't do the right thing because we want
;;;  the scroll bar to be inside the margin and something seems to like
;;;  to look at our scroll bar spec and add the width of the margin to
;;;  it because it thinks we couldn't possibly really mean that we want
;;;  to be backed into the margin.

(defmethod (margin-scroll-bar-mixin :after :activate) (&rest ignore)
  (let ((top    (cadr  margin-widths))
        (right  (caddr margin-widths))
        (bottom (cadddr margin-widths)))
    (setq tv:scroll-bar '(,right
                          ,top
                          ,(+ right scroll-bar-width)
                          ,(- bottom)))
    (setq tv:scroll-bar-always-displayed t)))
```

```
;;;; Line Truncating Mixin

;;; LINE-TRUNCATING-MIXIN

(defflavor line-truncating-mixin () (tv:line-truncating-mixin)
  (:documentation :mixin "Sets up to have line overflows be truncated."))

;;; :AFTER :INIT
;;;
;;;  Set up the relevant state data so window will know we intend to have
;;;  it doing truncation.

(defmethod (line-truncating-mixin :after :init) (&rest ignore)
  (setf (tv:sheet-truncate-line-out-flag self) 1))
```

```
;;;; Box Drawing Mixin

;;; BOX-DRAWING-MIXIN

(defflavor box-drawing-mixin () ()
  (:documentation :mixin
   "Methods for drawing outlined or complemented boxes.")
  (:required-methods :draw-rectangle :draw-lines
                     :char-width :line-height))

;;; In this :COMPLEMENT-BOX and :OUTLINE-BOX, give the character position
;;; for the top right character IN the box, and the bottom right character
;;; IN the box. (SEND FOO ':COMPLEMENT-BOX 4 4 4 4) will complement the
;;; character at position 4 4.  Characters are measured from 0.
;;;
;;; Optimized this so it looks good with a single box
;;; even though this may look odd with adjacent boxes due to overlap.
;;; It is intended that this will be better in most cases.

(defmethod (box-drawing-mixin :complement-box) (left top right bottom)
  ;; Convert to pixels
  (let ((char-width  (send self ':char-width))
        (line-height (send self ':line-height)))
    (let ((p-left   (- (* left char-width) 2))
          (p-width  (+ 4 (* (+ 1 (- right left)) char-width)))
          (p-top    (- (* top line-height) 1))
          (p-height (* (+ 1 (- bottom top)) line-height)))
      (send self ':draw-rectangle
            p-width p-height p-left p-top
            tv:alu-xor))))

;;; Draws lines around the area, then complements the corners. (This makes
;;; boxes that are constructed row by row look nicer)

(defmethod (box-drawing-mixin :outline-box) (left top right bottom)
  ;; Convert to pixels
  (let ((char-width  (send self ':char-width))
        (line-height (send self ':line-height)))
    (let ((p-left   (- (* left char-width) 2))
          (p-width  (+ 4 (* (+ 1 (- right left)) char-width)))
          (p-top    (- (* top line-height) 2))
          (p-height (1+ (* (+ 1 (- bottom top)) line-height))))
      (let ((p-right  (+ p-left p-width -1))
            (p-bottom (+ p-top p-height -1)))
        (send self ':draw-lines tv:alu-xor
              p-left  p-top
              p-right p-top
              p-right p-bottom
              p-left  p-bottom
              p-left  p-top)))))
```

```
;;;; Associated Process Mixin

(defflavor associated-process-mixin () (tv:process-mixin)
  (:default-init-plist :process
   ;; Initial parameters for the process stack are smaller than
   ;; for normal Lisp Listeners, so we boost this artificially
   ;; to keep users from pestering for an init option to this flavor
   ;; letting them specify rational values. -kmp
   '(process-toplevel . (:regular-pdl-size 10000.
                         :special-pdl-size  1500.))))

(defun process-toplevel (terminal-io)
  (do () (nil) ;loop forever
    (send terminal-io ':process-toplevel)))

(defmethod (associated-process-mixin :default :process-toplevel) ()
  (process-wait "Wait Forever" #'(lambda () nil)))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WITH-OUTPUT-TO-STRING Indent:1;
;;; Lisp WITH-INPUT-FROM-STRING Indent:1;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; ZMAGIC: Interface to ZWEI internals

;;; This file contains most of the references to ZWEI in the KBE system
;;; so that the rest of the system doesn't get overwhelmed by ZWEI's
;;; crufty interface.
```

```
;;;; Editor command definition

(defvar *editor* nil)

(defvar *initial-buffer* nil
  "In a ZWEI command, the buffer it was invoked from.")

;;;

(defmacro with-buffer-pointer ((variable buffer-pointer type) &body body)
  '(zwei:with-bp (,variable ,buffer-pointer ,type)
     ,@body))

(defmacro with-local-buffer-pointer ((var bp) &body body)
  (let ((val (gensym)))
    '(let ((,val nil))
       (setq ,val ,bp)
       (unwind-protect (let ((,var ,val))
                         ,@body)
                       (zwei:flush-bp ,val)))))

(eval-when (eval load compile)

(defun fix-if-KBE-command-name (command-name)
  (setq command-name (string-copy command-name)) ;lossage avoidance
  (do ((pos (string-search "kbe" command-name)
            (string-search "kbe" command-name (1+ pos))))
      ((not pos) command-name)
    (setf (aref command-name (+ pos 0)) #/K)
    (setf (aref command-name (+ pos 1)) #/B)
    (setf (aref command-name (+ pos 2)) #/E)))

)

;;; Our DEFINE-ZWEI-COMMAND thing does (KM) automatically.
;;; This means the mark stays unless the command goes to work
;;; to make it go away.

(advise zwei:make-command-name :around casification-control nil
  (let ((test-result (get (car arglist) 'zwei:command-name))
        (real-result :do-it))
    (if (string-equal real-result test-result)
        test-result ;prefer old case
        real-result)))

(defmacro define-zwei-command (name bvl key doc &body body)
  '(progn 'compile
          (defprop ,name ,(fix-if-KBE-command-name
                            (zwei:make-command-name name))
                   zwei:command-name)
          (zwei:defcom ,name ,doc (km)
                       (cond (((lambda ,bvl
                                 (let ((*initial-buffer* zwei:*interval*))
                                   ,@body))
                               (and zwei:*numeric-arg-p*
                                    zwei:*numeric-arg*))
                              zwei:dis-text)
                             (t zwei:dis-bps)))
          ,(cond (key
                  '(progn ,@(mapcar #'(lambda (key)
                                        '(zwei:set-comtab
                                          zwei:*standard-comtab*
                                          '(,key ,name)))
                                    (cond ((listp key) key)
                                          (t (list key))))))
                 (t
                  '(zwei:set-comtab zwei:*standard-comtab*
                                    '()
```

```
                                    (zwei:make-command-alist
                                      '(,name)))))))

(defvar *zwei-commands* '() "List of KBE commands available in ZWEI.")

(define-internal-option *debug-zwei-errors*
                        "Debug ZWEI errors in KBE commands"
  nil :boolean)

(defmacro define-editor-command (name bvl key doc &body body)
  '(progn 'compile
          (define-zwei-command ,name ,bvl ,key ,doc
            (with-delegate (window-delegate)
              (condition-case-if (not *debug-zwei-errors*) (cond)
                  (with-automatic-error-recovery-if
                        (or (not *debug*)
                            (not *debug-zwei-errors*))
                    (progn ,@body))
                ((error-condition-mixin)
                 (editor-barf "~A" cond)))))
          (if (not (memq ',name *zwei-commands*))
              (push ',name *zwei-commands*))
          ',name))
```

```
;;;; Editor command documentation

(defun mouse-sensitive-princ (type data &rest format-args)
  (cond ((send standard-output ':operation-handled-p ':item)
         (caseq type
           ((:function)
            (lexpr-send standard-output ':item
                        'zwei:function-name data format-args))
           (otherwise
             (barf "Invalid keyword to MOUSE-SENSITIVE-PRINC: ~S" type))))
        (format-args
         (lexpr-funcall #'format t format-args))
        (t ;for use, eg, outside of ZWEI
         (format t "~A" data))))

(defmacro do-editor-commands ((command) &body body)
  '(dolist (,command (reverse *zwei-commands*)) ;order of definition
     ,@body))

(defun document-editor-commands (&optional (comtab
                                             (if (boundp 'zwei:*comtab*)
                                                 zwei:*comtab*
                                                 zwei:*zmacs-comtab*)))
  (do-editor-commands (command)
    (document-one-editor-command command comtab)))

(defun document-one-editor-command (command
                                    &optional (comtab zwei:*zmacs-comtab*))
  (multiple-value-bind (key-list extended? name)
      (command-key-names command nil comtab)
    (format t "~2&")
    (cond ((not key-list)
           (setq name (or name command))
           (cond (extended?
                  (format t "~A " extended?)
                  (mouse-sensitive-princ ':function command "~A" name))
                 (t
                  (mouse-sensitive-princ ':function command "~A"
                                         name)
                  (format t " (not implemented)"))))
          (t
           (format t "~A (~@[~A ~]"
                   (phrase-append "and" key-list) extended?)
           (mouse-sensitive-princ ':function command "~A" name)
           (format t ")" name)))
    (format t " ... ~&")
    (zwei:print-doc ':full command)
    (format t "~&")))

(defun extended-command-p (symbol
                           &optional (zwei:*comtab* zwei:*zmacs-comtab*))
  (zwei:extended-command-p symbol))

(defun editor-command-name (command) (get command 'zwei:command-name))

(defun command-key-names (command
                          &optional limit (comtab zwei:*zmacs-comtab*))
  (declare (values key-names extended? name))
  (let ((key-names (mapcar #'bucky-char-name
                           (command-keys command limit comtab)))
        (extended? nil)
        (name (zwei:command-name command t)))
    (when (and name (extended-command-p command comtab))
      (setq extended? "m-X"))
    (when (and name
               (not key-names)
               (not extended?)
               (rassq command zwei:*command-alist*))
```

```
      (setq extended? "c-m-X"))
    (values key-names extended? name)))

(defun command-keys (command &optional limit (comtab zwei:*zmacs-comtab*))
  (do ((commands '())
       (i 0 (1+ i)))
      ((and limit (= i limit)) (nreverse commands))
    (multiple-value-bind (found? key)
        (zwei:key-for-command command
                              comtab
                              (1+ (or (car commands) -1)))
      (if (not found?) (return (nreverse commands)))
      (push key commands))))

(defun command-key-name (command)
  (multiple-value-bind (key-names extended? name)
      (command-key-names command)
    (cond (key-names (car key-names))
          (extended? (text "~A ~A" extended? name))
          (t nil))))
```

```
;;;; Buffers

(defun funcall-in-editor (fn &rest args)
  (lexpr-send zwei:*zmacs-command-loop* ':funcall-inside-yourself fn args))

(defun lexpr-funcall-in-editor (fn &rest args)
  (lexpr-funcall #'lexpr-funcall #'funcall-in-editor fn args))

;;; (BIND-EDITOR-BUFFER buffer . body)
;;;
;;;  Executes body with the editor's default buffer bound to the
;;;  given buffer.

(defmacro bind-editor-buffer (buffer &body body)
  '(let ((zwei:*interval* ,buffer))
     ,@body))

;;; (FIND-BUFFER buffer prompt create?)
;;;
;;;  Returns a pointer to the specified buffer.
;;;
;;;  If the buffer name is:
;;;
;;;    NIL, then the CREATE? and PROMPT arguments must have been
;;;              supplied, and the user will be prompted for the name.
;;;
;;;    a string, then the specified buffer will be searched for and
;;;                    created if the user desired. If the buffer was
;;;                    non-existent and creation wasn't requested, a
;;;                    name will be prompted for as if NIL as given.
;;;
;;;    anything else, then that thing will be returned. This is so that
;;;                         actual buffer objects will pass straight
;;;                         through without our having to know what
;;;                         the representation of a buffer is.

(defun find-buffer (buffer-name &optional (prompt nil) (create? t))
  (let ((buffer (cond ((stringp buffer-name)
                       (zwei:find-buffer-named buffer-name create?))
                      (t buffer-name))))
    (cond (buffer buffer)
          ((not prompt)
           (barf "FIND-BUFFER got null buffer but no prompt."))
          (t
           (zwei:read-buffer-name prompt zwei:*interval* create?)))))

;;; (KILL-BUFFER name)

(defun kill-buffer (name)
  (zwei:kill-buffer
    (cond ((stringp name)
           (or (zwei:find-buffer-named name nil)
               (barf "KILL-BUFFER: No such buffer ~S" name)))
          ((null name)
           (zwei:read-buffer-name name zwei:*interval* nil))
          (t name))))

;;; (BUFFER-NAME buffer)

(defun buffer-name (buffer)
  (send buffer ':name))

;;; (BUFFER-NAME-IF-RELEVANT buffer)

(defun buffer-name-if-relevant (buffer)
  (unless (eq buffer *initial-buffer*)
    (buffer-name buffer)))
```

```
;;; (BUFFER-PATHNAME buffer)
;;;
;;;  Get the associated pathname of a buffer.

(defun buffer-pathname (buffer)
  (send-if-handles buffer ':pathname))

;;; (EDITOR-BUFFER-GET buffer propname)
;;; (EDITOR-BUFFER-PUT buffer propname val)

(defun editor-buffer-get (buffer propname)
  (send buffer ':get propname))

(defun editor-buffer-put (buffer propname val)
  (setf (send buffer ':get propname) val))

(eval-when (compile load eval)

(defprop editor-buffer-get
        ((editor-buffer-get flavor-name indicator)
         . (editor-buffer-put flavor-name indicator si:val))
        setf)

) ; NEHW-LAVE

;;; (MAKE-BUFFER-CURRENT buffer)

(defun make-buffer-current (buffer)
  (funcall-in-editor #'zwei:make-buffer-current buffer))

;;; (BUFFER-PACKAGE [buffer])

(defun buffer-package (&optional (buffer zwei:*interval*))
  (send buffer ':get ':package))

;;; (BUFFER-RADIX [buffer])

(defun buffer-radix (&optional (buffer zwei:*interval*))
  (send buffer ':get ':base))

;;; (BUFFER-FONT [buffer])

(defun buffer-font (&optional (buffer zwei:*interval*))
  (or (cdar (or (send buffer ':get ':font-alist)
                (zwei:window-font-alist zwei:*window*))) ;fall-back
      (tv:screen-default-font tv:mouse-sheet)))          ;desperation

;;; (CURRENT-EDITOR-BUFFER)

(defun current-editor-buffer ()
  (if (variable-boundp zwei:*interval*) zwei:*interval* nil))

;;; (BUFFER-END [buffer])

(defun buffer-end (&optional (buffer zwei:*interval*))
  (zwei:interval-last-bp buffer))

;;; (BUFFER-START [buffer])

(defun buffer-start (&optional (buffer zwei:*interval*))
  (zwei:interval-first-bp buffer))

;;; (CONTAINING-BUFFER buffer)

(defun containing-buffer (bp)
  (do ((node (zwei:bp-node bp)))
      (nil)
    (let ((superior-node (zwei:node-superior node)))
      (cond ((not superior-node) (return node))
```

```
        (t (setq node superior-node))))))
```

```
;;;; Editor I/O

;;; These are functions in case we need to funcall them.

;;; (EDITOR-BARF string . args)

(defun editor-barf (&rest args)
  (lexpr-funcall #'zwei:barf args))

(defprop editor-barf t :error-reporter)

;;; (EDITOR-ECHO-OUTPUT string . args)
;;;
;;;  Takes args like FORMAT and types them out to the editor echo area.

(defun editor-echo-output (string &rest args)
  (cond ((boundp 'zwei:*typein-window*)
         (lexpr-funcall #'zwei:typein-line string args))
        (t ; Allow some ZWEI utilities to run outside of ZWEI
         (lexpr-funcall #'format t string args))))

;;; (EDITOR-MAIN-OUTPUT string . args)

(defun editor-main-output (string &rest args)
  (lexpr-funcall #'format t string args))
```

```
;;;; Syntax tables

;;; (SET-EDITOR-LIST-SYNTAX-FROM-CHARACTER char from-char)
;;;
;;;  Sets the list-syntax of CHAR in the editor (ZWEI) to be the same
;;;  as the list syntax of FROM-CHAR.

(defun set-editor-list-syntax-from-character (char from-char)
  (zwei:set-char-syntax (zwei:char-syntax from-char
                                          zwei:*list-syntax-table*)
                        zwei:*list-syntax-table* char))
```

```
;;;; Pathnames

;;; (READ-DEFAULTED-PATHNAME prompt default)
;;;
;;; Prompts (a la mini-buffer) for a filename, which it returns.

(defun read-pathname (prompt default)
  (zwei:read-defaulted-pathname prompt default))

;;; (READ-DEFINITION-NAME prompt [default])

(defun read-definition-name (prompt
                             &optional default must-be-defined-p)
  (zwei:symbol-from-string
    (zwei:read-function-spec-string prompt default must-be-defined-p
          "You are typing the name of a function spec.")))

;;; (CURRENT-EDITOR-DEFINITION-NAME)
;;;
;;; Returns NIL if no current definition name or
;;; two values:
;;;  The name of the current editor definition.
;;;  The kind of definition (the car of the defining form).
;;;
;;; This definition could be much simpler if not for various
;;; sectionization bugs which are known to occur in Release 4.5
;;; and 5.0. Some of the bugs are claimed to be fixed in a later
;;; release...

(defun current-editor-definition-name (&optional (bp (zwei:point))
                                                 (err? nil))
  (declare (values spec type node))
  (let ((spec nil) (type nil) (node nil))
    (cond ((and err? (zwei:window-mark-p zwei:*window*))
           (editor-barf "This command works on definitions, not regions."))
          (t
           (multiple-value-bind (bounding-node start-bp)
               (zwei:definition-interval bp)
             (cond (bounding-node
                    (zwei:check-interval-sections bounding-node)
                    (let ((start-line (zwei:bp-line start-bp)))
                      (setq spec (zwei:get-definition-function-spec
                                   start-bp))
                      (setq type
                            (when (and start-line
                                       (> (string-length start-line) 2)
                                       (char-equal (aref start-line 0)
                                                   #/()))
                              (let ((package
                                      (buffer-package
                                        (containing-buffer bp))))
                                (read-from-string start-line nil 1))))
                      (setq node bounding-node)))
                   ((let ((n (zwei:bp-node bp)))
                      ;; Ugh. At this point, we must rely on sectionization
                      ;; to have been set up right. Not much chance. -kmp
                      (when (setq spec (send-if-handles n ':function-spec))
                        (setq node n)
                        (setq type (send n ':definition-type))
                        t)))
                   (t
                    (when err?
                      (editor-barf
                        "The cursor isn't inside a well-formed definition."
                        )))))))
    (when (not (or (get type 'zwei:definition-function-spec-type)
                   (stringp (si:definition-type-name type))))
      (setq spec nil)
```

```
      (if err?
          (editor-barf "The enclosing~@[ ~S~] form is not a defining form."
                       type)))
    (values spec type node)))

;;; (WITH-CURRENT-EDITOR-DEFINITION-NAME name . body)
;;;
;;;  Binds name to the current editor definition name within the scope
;;;  of body. If no current definition, body is never executed.

(defmacro with-current-editor-definition-name (name-and-maybe-type
                                               &body forms)
  '(multiple-value-bind ,name-and-maybe-type
       (current-editor-definition-name)
     (cond ((not ,(car name-and-maybe-type))
            (zwei:barf "The cursor isn't in a definition."))
           (t
            ,@forms))))

;;; (CURRENT-EDITOR-PATH bp)
;;;
;;;  Returns a path into the current toplevel expression as a list
;;;  (n1 n2 n3 ..) such that the current expression is denoted by
;;;  (nth ... (nth n3 (nth n2 (nth n1 <toplevel>)))).
;;;
;;;  Because it may be slow, it caches its result in case called
;;;  more than once.

(defvar *current-editor-path* nil)
(defvar *current-editor-path-time* nil)
(defvar *current-editor-path-bp* nil)

(defun current-editor-path ()
  (declare (values path toplevel?))
  (cond ((and *current-editor-path-time*
              (zwei:bp-= *current-editor-path-bp* (zwei:point))
              (>= *current-editor-path-time* (zwei:node-tick
                                               (zwei:bp-node
                                                 (zwei:point)))))
         (values-list *current-editor-path*))
        (t
         (let ((fn (get 'current-editor-path (buffer-editing-type))))
           (let ((path (multiple-value-list
                         (if fn (funcall fn (zwei:point) '() fn)))))
             (setq *current-editor-path-time* nil) ;Mark dirty
             (setq *current-editor-path* path)
             (setq *current-editor-path-bp* (copy-buffer-pointer
                                              (zwei:point)
                                              ':normal))
             (setq *current-editor-path-time* (editor-current-time))
             (values-list path))))))

(comment ;obsolete? -kmp

(defun (current-editor-path :ada) (&rest ignore) (values nil t))

);tnemmoc

(defun (current-editor-path :ada) (&rest ignore)
  (condition-case ()
      (multiple-value-bind (start-pt end-pt)
          (bounds-of-definition)
        (cond ((not start-pt) (values nil t))
              (t
               (let ((pt (search-buffer-backward ";")))
                 (cond ((or (not pt)
                            (not (setq pt (forward-char pt)))
                            (zwei:bp-< pt start-pt)
                            (zwei:bp-< end-pt pt))
```

```
                    (values nil t))
                   (t
                    (setq pt (forward-char pt))
                    (let ((part1 (string-buffer-between start-pt pt))
                          (part2 (string-buffer-between pt end-pt)))
                      (let ((magic-text (string-append
                                          part1
                                          "{Zmacs-Cursor};"
                                          part2)))
                        (find-path (ada-parse "{Zmacs-Cursor}")
                                   (ada-parse magic-text)))))))))
    (sys:error (values nil nil))))

(defun find-path (marker sexp)
  (let ((*marker* marker))
    (declare (special *marker*))
    (values (*catch 'find-path (find-path-1 sexp '())))))

(defun find-path-1 (sexp path)
  (declare (special *marker*))
  (cond ((equal *marker* sexp) (*throw 'find-path (nreverse path)))
        ((atom sexp) nil)
        (t
         (do ((i 0 (1+ i))
              (s sexp (cdr s)))
             ((null s))
           (find-path-1 (car s) (cons i path))))))


(defun (current-editor-path :lisp) (start-pt path myself)
  (declare (special *kbe-role-marker*) (values path toplevel?))
  (let ((up (zwei:forward-up-list-or-string start-pt -1))) ;backward-up
    (cond ((not up)
           (ignore-errors
             (check-sectionization start-pt) ;Now while it's cheap...
             (with-buffer-pointer (bp start-pt)
               (check-sectionization (let ((pt (forward-sexp bp)))
                                       (if pt (backward-sexp pt))))
               (check-sectionization (backward-sexp bp))))
           ;needn't reverse it since we worked inside->out
           (cond ((or (not path)
                      (< (car path) 3)
                      (and (= (car path) 3)
                           (not (cdr path))
                           (not (nthcdr 4 (get-definition)))))
                  ;; If at toplevel, say we are actually at the
                  ;; end of the definition we are near. Eg,
                  ;; Toplevel of (DEFUN F (X) ) is the same as
                  ;; being (DEFUN F (X) **HERE**).
                  (let ((sexp (get-definition)))
                    (values (if sexp (list (length sexp)))
                            t))) ;pretend toplevel
                 (t (values path nil))))
          (t (do ((i 0 (1+ i))
                  (pt start-pt))
                 (nil)
               (let ((new-pt (zwei:forward-sexp pt -1 nil 0 nil t t)))
                 (cond (new-pt (setq pt new-pt))
                       (t
                        (setq i (ncons i))
                        ;; Hair to recognize #{...} syntax.
                        ;; Makes rash assumptions about the format
                        ;; of the KBE-#-COMMENT-MACRO expression.
                        (when (cond ((= (character *kbe-role-marker*) #/#)
                                     (and (char-equal (char-after up) #/#)
                                          (char-equal (char-after
                                                        (forward-char up))
                                                      #/{)))
```

```
          (t (char-equal (char-after up) #/{)))
    (cond ((let ((bp (forward-sexp
                       (forward-char
                         (forward-char up)))))
             (zwei:search bp "," nil nil nil
                          (forward-sexp up)))
           (cond ((= (car i) 0)
                  (setq i (list* 1 0 nil)))
                 (t
                  (setq i (list* 2
                                 (1- (car i))
                                 nil)))))
          (t
           (setq i (cons 2 i)))))
  (return
    (funcall myself
             up (nconc i path) myself))))))))))
```

```
;;;; Buffer Insertion/Deletion

;;; (INSERT-INTO-BUFFER-BEFORE-POINT text)
;;;
;;;  Inserts text at point, moving point.

(defun insert-into-buffer-before-point (text)
  (zwei:insert-moving (zwei:point) text))

;;; (INSERT-INTO-BUFFER-AFTER-POINT text)
;;;
;;;  Inserts text after point, leaving point fixed.
;;;
;;;  There's probably a better way to do this...

(defun insert-into-buffer-after-point (text)
  (zwei:insert (zwei:point) text))

;;; (INSERT-INTO-BUFFER-AFTER  pointer text)
;;; (INSERT-INTO-BUFFER-BEFORE pointer text)

(defun insert-into-buffer-before (bp text)
  (let ((cursor (zwei:point)))
    (let ((point-must-move? (zwei:bp-= bp cursor)))
      (zwei:insert-moving bp text)
      (if point-must-move? (move-point bp))
      bp)))

(defun insert-into-buffer-after (bp text)
  (zwei:insert bp text))

;;; (DELETE-CHARS-IN-SET set [pointer])
;;;
;;;  Deletes chars in the given SET from near the given POINTER,
;;;  which defaults to point.

(defun delete-chars-in-set (set &optional (pt (editor-point)))
  (zwei:delete-around set pt))

;;; (DELETE-FROM-BUFFER-BETWEEN pointer1 pointer2 [save?])
;;;
;;;  Deletes the region between pointer1 and pointer2, which need
;;;  not be in order.
;;;
;;;  If SAVE? is non-NIL, then the deleted section will be saved in
;;;  the kill ring.

(defun delete-from-buffer-between (p1 p2 &optional (save? nil))
  (cond (save? (zwei:kill-interval p1 p2 t))
        (t     (zwei:delete-interval p1 p2 t))))

;;; (FLUSH-CHARS char-set [pt])
;;;
;;;  Deletes all the characters in CHAR-SET on both sides of PT.

(defun flush-chars (char-set &optional (pt (zwei:point)))
  (zwei:delete-around char-set pt))

;;; (COPY-FROM-BUFFER-BETWEEN pointer1 pointer2)
;;;
;;;  Returns a copy of the given region.

(defun copy-from-buffer-between (bp1 bp2)
  (zwei:copy-interval bp1 bp2))

;;; (YANK-INTO-BUFFER-AT pointer node)

(defun yank-into-buffer-at (pointer node)
```

```
  (zwei:insert-interval pointer node))

;;; (YANK-INTO-BUFFER-AT-POINT node)
;;;
;;; Returns the bp at the end of inserted text.

(defun yank-into-buffer-at-point (node)
  (zwei:insert-interval (zwei:point) node))

;;; (STRING-BUFFER-BETWEEN point1 point2)

(defun string-buffer-between (point1 point2)
  (zwei:string-interval point1 point2))

;;; (FILL-BUFFER-BETWEEN point1 point2)
;;;
;;;  Does autofilling on a given region of the buffer.

(defun fill-buffer-between (pt1 pt2)
  (zwei:fill-interval pt1 pt2 t))
```

```
;;; (INDENT-THIS-LINE-FOR-LISP)

(defun indent-this-line-for-lisp () (zwei:com-indent-for-lisp))
```

```
;;;; BP manipulation

;;; (create-buffer-pointer line [index])
;;;
;;;  Returns a buffer-pointer to the given line. Index defaults to 0,
;;;  meaning the head of the line.

(defun create-buffer-pointer (line &optional (index 0))
  (zwei:create-bp line index))

;;; (copy-buffer-pointer bp)
;;;
;;;  Returns a copy of the given buffer-pointer.

(defun copy-buffer-pointer (pointer &optional status)
  (zwei:copy-bp pointer status))
```

```
;;;; BP movement

;;; (EDITOR-POINT)

(defun editor-point ()
  (zwei:point))

;;; (EDITOR-MARK)

(defun editor-mark ()
  (zwei:mark))

;;; (MOVE-POINT where)

(defun move-point (where)
  (zwei:move-bp (zwei:point) where))

;;; (MOVE-MARK where)

(defun move-mark (where)
  (zwei:move-bp (zwei:mark) where))

;;; (ESTABLISH-REGION bp1 bp2 [style])
;;;
;;;  Makes a region from BP1 to BP2.
;;;  STYLE can be used to set the style.
;;;   By default, NIL means invisible, T means always visible.
;;;   The editor can be independently extended to
;;;   understand other options without changing this function.

(defun establish-region (bp1 bp2 &optional (region-style t))
  (when (and bp1 bp2)
    (move-point bp1)
    (move-mark  bp2)
    (setf (zwei:window-mark-p zwei:*window*) region-style)
    (setq zwei:*mark-stays* t)
    (zwei:must-redisplay zwei:*window* zwei:dis-bps)))
```

```
;;;; Virtual movement

;;; These commands move without side-effecting. A new BP is returned. You
;;; have to do (MOVE-POINT ...) or (MOVE-MARK ...) to get the cursor
;;; there.

;;; We currently use the ZWEI up/down-list type commands. Note that these
;;; will not work correctly to go into readmacrofied regions. eg, down
;;; from before the ' of '(A B C) goes inside the paren by the A when
;;; conceptually it should go to before the Q in (QUOTE (A B C)), which
;;; is not really a place in the buffer at all. FORTUNATELY, the only
;;; interesting readmacros are #, which mostly introduces atoms and
;;; which we won't want to go inside of or ' which introduces quoted
;;; structure, which the coder never looks inside of. At some point, we
;;; may have to deal with #', but currently we don't, so we'll correct
;;; our up/down-list stuff to be more sensitive to this case when need
;;; presents itself...

;;; (BEGINNING-OF-LINE bp)

(defun beginning-of-line (&optional (pt (zwei:point)))
  (zwei:beg-line pt))

;;; (END-OF-LINE bp)

(defun end-of-line (&optional (pt (zwei:point)))
  (zwei:end-line pt))

;;; (FORWARD-DOWN-LIST bp)

(defun forward-down-list (bp) ;like zwei:com-down-list
  (zwei:forward-list bp 1 nil -1 t))

;;; (BACKWARD-UP-LIST bp)

(defun backward-up-list (bp) ;like zwei:com-backward-up-list
  (zwei:forward-up-list-or-string bp -1))

;;; (FORWARD-DEFUN bp)

(defun forward-defun (bp &optional (count 1))
  (zwei:forward-definition bp count))

;;; (BACKWARD-DEFUN bp)

(defun backward-defun (bp &optional (count 1))
  (zwei:forward-definition bp (- count)))

;;; (FORWARD-WORD bp [count])

(defun forward-word (bp &optional (count 1))
  (zwei:forward-word bp count))

;;; (BACKWARD-WORD bp [count])

(defun backward-word (bp &optional (count 1))
  (zwei:forward-word bp (- count)))

;;; (FORWARD-SEXP bp [count])
;;;
;;;  Goes forward count (default 1) s-expressions.

(defun forward-sexp (bp &optional (count 1))
  (zwei:forward-sexp bp count))

;;; (BACKWARD-SEXP bp [count])
;;;
;;;  Goes backward count (default 1) s-expressions.
```

```
(defun backward-sexp (bp &optional (count 1))
  (zwei:forward-sexp bp (- count)))

;;; (FORWARD-CHAR bp [count])
;;;
;;;  Goes forward (default 1) char(s).

(defun forward-char (bp &optional (count 1))
  (zwei:forward-char bp count))

;;; (BACKWARD-CHAR bp [count])
;;;
;;;  Goes backward (default 1) char(s).

(defun backward-char (bp &optional (count 1))
  (zwei:forward-char bp (- count)))

;;; (FORWARD-LINE pt count)

(defun forward-line (&optional (pt (zwei:point)) (count 1))
  (zwei:beg-line pt count))

;;; (BACKWARD-LINE pt count)

(defun backward-line (&optional (pt (zwei:point)) (count 1))
  (forward-line pt (- count)))
```

```
(defun end-of-definition-comments (&optional
                                   (name (current-editor-definition-name))
                                   (make-room? nil))
  (if (not name) (editor-barf "Not inside a definition."))
  (multiple-value-bind (best-guess string)
      (find-single-definition name)
    (if (not best-guess) (editor-barf string))
    (do ((bp (zwei:beg-line best-guess -1)
             (zwei:beg-line bp -1))
         (flag 0 (1+ flag)))
        (nil)
      (when (or (not bp)
                (neq (zwei:line-type (zwei:bp-line bp)) ':blank))
        (when (and make-room? (< flag 2))
          (insert-into-buffer-before best-guess (text "~V%" (- 3 flag)))
          (return (end-of-definition-comments name)))
        (return best-guess))
      (if (< flag 2) (setq best-guess bp)))))

(defun insert-comment-definition (name explanation &optional syntax)
  (when explanation
    (move-point
     (forward-char
      (insert-into-buffer-before
        (end-of-definition-comments name t)
        (format-explanation-for-comment explanation syntax))))))
```

```
;;;; Ordering

;;; (POINT-PRECEDES? bp1 bp2)

(defun point-precedes? (bp1 bp2)
  (zwei:bp-< bp1 bp2))
```

```
;;;; Search

;;; (SEARCH-BUFFER string [bp])

(defun search-buffer (string &optional (bp (zwei:point)) (times 1))
  (let ((pt bp) (rev? (minusp times)))
    (dotimes (i (abs times))
      (if pt (setq pt (zwei:search pt string rev?))))
    pt))

;;; (SEARCH-BUFFER-FORWARD string [bp])
;;;
;;;  Searches buffer forward for a given string. By default, searches
;;;  from the current point, but an initial bp can be specified.

(defun search-buffer-forward (string &optional (bp (zwei:point)))
  (zwei:search bp string))

;;; (SEARCH-BUFFER-BACKWARD string [bp])
;;;
;;;  Searches buffer backward for a given string. By default, searches
;;;  from the current point, but an initial bp can be specified.

(defun search-buffer-backward (string &optional (bp (zwei:point)))
  (zwei:search bp string t))

;;; (SEARCH-BUFFER-FORWARD-DELIMITED string [pt])
;;;
;;;  Searches the buffer forward for a given string bounded by
;;;  atom delimiters.

(defun search-buffer-forward-delimited (string
                                        &optional (bp (editor-point)))
  (do ((pt (search-buffer string bp) (search-buffer string pt))
       (len (string-length string)))
      ((or (not pt)
           (and (breakchar-p (char-after pt))
                (breakchar-p (char-before (backward-char pt len)))))
       pt)))

;;; (SEARCH-BUFFER-BACKWARD-DELIMITED string [pt])
;;;
;;;  Like SEARCH-BUFFER-FORWARD-DELIMITED, but works backward.

(defun search-buffer-backward-delimited (string
                                         &optional (bp (editor-point)))
  (do ((pt (search-buffer-backward string bp)
           (search-buffer-backward string pt))
       (len (string-length string)))
      ((or (not pt)
           (and (breakchar-p (char-before pt))
                (breakchar-p (char-after (forward-char pt len)))))
       pt)))

;;; (SEARCH-BUFFER-BETWEEN bp1 bp2 string [reverse?])

(defun search-buffer-between (bp1 bp2 string &optional (reverse? nil))
  (zwei:search bp1 string reverse? nil nil bp2))
```

```
;;;; Redisplay

;;;  ZWEI's redisplay level objects are constants bound to
;;;  the following global variables in the ZWEI package:
;;;
;;;     DIS-NONE        No redisplay needed.
;;;
;;;     DIS-MARK-GOES   No redisplay needed except maybe
;;;                     removing region underlining.
;;;
;;;     DIS-BPS         Point and mark may have moved, but text is
;;;                     unchanged.
;;;
;;;     DIS-LINE        Text in one line may have changed.
;;;                     WINDOW-REDISPLAY-LINE is that line.
;;;                     WINDOW-REDISPLAY-INDEX says where in the line
;;;                      changes start.
;;;
;;;     DIS-TEXT        Any text might have changed.
;;;
;;;     DIS-ALL         Global parameters of the window have changed.
;;;                     Clean the window and redisplay all lines from
;;;                     scratch.
;;;
;;;  This information was obtained by using M-. on ZWEI:DIS-NONE.
;;;  It should be periodically checked for correctness until a manual
;;;  is published declaring this sort of thing to be stable.

(defun editor-window-redisplay ()
  (zwei:must-redisplay zwei:*window* zwei:dis-text))

(defun editor-window-must-redisplay-rest-of-line (&optional
                                                  (bp (zwei:point)))
  (zwei:must-redisplay zwei:*window*
                       zwei:dis-line
                       (zwei:bp-line bp)
                       (zwei:bp-index bp)))

(defun redisplay-all-windows () (zwei:redisplay-all-windows t nil))
```

```
;;;; Mode line

(defun set-mode-line (data)
  (setq zwei:*mode-line-list*
        (cond ((atom data) (cons data (cdr zwei:*mode-line-list*)))
              (t data))))

(defmethod (zwei:top-level-editor :change-mode-line-list) (fn)
  (setq zwei:*mode-line-list* (funcall fn zwei:*mode-line-list*)))

(define-internal-option *zmacs-default-mode-line-name*
                        "Editor name for UPDATE-ZMACS-MODE-LINE"
  "KBEmacs" :string)
;  "KBE//ZMACS" :string)

(defun update-zmacs-mode-line ()
  (send zwei:*zmacs-command-loop* ':change-mode-line-list
        #'(lambda (l)
            (let ((new (copylist l)))
              (if (and (stringp (car new))
                       (string-search "MACS" (car new)))
                  (setf (car new)
                        (text "~A " *zmacs-default-mode-line-name*)))
              (setq new (delq 'zwei:*buffer-modified-p* new))
              (let ((tail (or (member ") " new)
                              (member ")" new))))
                (when tail
                  (when (equal (car tail) ") ")
                    (push " " (cdr tail))
                    (push 'zwei:*buffer-modified-p* (cdr tail)))
                  (setf (car tail) ")")))
              new))) )

(define-system-initialization zmacs-mode-line
  (update-zmacs-mode-line))
```

```
;;;; In the name of abstraction

(defflavor vanilla-editor-frame () (zwei:zmacs-frame))

(defmethod (zwei:zmacs-frame :delegate) ()
  (or (send-any-superior-if-handles (send self ':superior) ':delegate)
      (editor-delegate)))

(defun editor-state ()
  (send zwei:*zmacs-window-process* ':whostate))
```

```
;;;; Comment stuff

(defun editor-comment-begin () zwei:*comment-begin*)

(defun editor-comment-end () zwei:*comment-end*)
```

```
;;;; Line validation

;; DWIM stuff for debugging. This should go away some day. --KMP

(define-KBE-option *Automatic-Lisp-Mode*
                   "Automatically enter Lisp Mode for code buffers"
  t :boolean)

(define-KBE-option *Automatic-KBE-Package*
                   "Automatically put code buffers in KBE package"
  t :boolean)

(defvar *known-modes* '(zwei:lisp-mode zwei:ada-mode))

(defun set-buffer-attribute (attribute value
                             &optional (attribute-string
                                        (string-capitalize-words
                                          attribute))
                                       (value-string (string value)))
  (let ((zwei:*set-attribute-updates-list* t))
    (zwei:set-attribute-internal attribute
                                 attribute-string
                                 value-string
                                 value)
    (zwei:com-reparse-attribute-list)))

(defun set-buffer-mode (mode-keyword)
  (selectq mode-keyword
    (:ada (com-ada-mode))
    (otherwise (funcall (intern (text "COM-~A-MODE" mode-keyword) 'ZWEI))))
  (set-buffer-attribute ':major-mode (intern (text "~A-MODE" mode-keyword)
                                             'ZWEI)
                        "Mode" (string mode-keyword)))

(defun validate-buffer-mode (language &optional for-function)
  for-function ;ignored for now
  (set-buffer-mode language))

(defun validate-buffer-package ()
  (when (and *automatic-kbe-package* (not (eq package *kbe-package*)))
    (set-buffer-attribute ':package *kbe-package* "Package" "KBE")
    (zwei:sectionize-buffer zwei:*interval*)))

(defun enter-lisp-mode ()
  (zwei:com-lisp-mode)
  (zwei:com-update-attribute-list))
```

```
(defun check-sectionization (&optional (point (editor-point)))
  (when point
    (zwei:check-interval-sections
      (zwei:line-node (zwei:bp-line point)))))

(defun definition-last-modified-time (fn &optional type)
  (let ((node (find-single-definition-node fn type)))
    (if node (zwei:node-tick node))))

(defun editor-current-time () zwei:*tick*)

(defmacro define-editor-definition-type (name type)
  '(defprop ,name ,type zwei:definition-function-spec-type))

(defun real-editor-buffer? (x) (typep x 'zwei:buffer))

(defun assure-real-editor-buffer ()
  (if (not (real-editor-buffer? (current-editor-buffer)))
      (editor-barf "Not in correct buffer context.")))

(comment ;old definition

(defun place-to-create-dummy-definition (name &optional type)
  type ;ignored
  (let ((data (suggested-code-home name)))
    (let ((buffer-name (get (locf data) ':buffer))
          (near        (get (locf data) ':near)))
      (let ((buffer  (if buffer-name (find-buffer buffer-name)))
            (near-bp (if near (find-single-definition near))))
        (cond ((and near-bp
                    buffer
                    (neq buffer (containing-buffer near-bp)))
               (buffer-end buffer))
              (near-bp
               (zwei:interval-last-bp
                 (zwei:bp-node near-bp)))
              (buffer
               (buffer-end buffer))
              (t
               (buffer-end (current-editor-buffer))))))))

)

(defun place-to-create-dummy-definition (name &optional type)
  type ;ignored
  (let ((data (suggested-code-home name)))
    (let ((buffer-name (get (locf data) ':buffer))
          (near        (get (locf data) ':near)))
      (let ((buffer  (if buffer-name (find-buffer buffer-name))))
        (multiple-value-bind (start-pt end-pt)
            (if near (bounds-of-definition near))
          (copy-buffer-pointer ;be safe...
            (cond ((and start-pt
                        buffer
                        (neq buffer (containing-buffer start-pt)))
                   (buffer-end buffer))
                  ((and end-pt (forward-line end-pt)))
                  (buffer
                   (buffer-end buffer))
                  (t
                   (buffer-end (current-editor-buffer))))))))))

(defun buffer-editing-type (&optional (buffer zwei:*interval*))
  (or (get (send buffer ':get ':major-mode) 'zwei:editing-type)
      ':lisp))

(defun create-dummy-definition (name &optional type)
```

```
  (assure-real-editor-buffer)
  (if (not type) (setq type 'DEFUN))
  (let ((where (place-to-create-dummy-definition name type)))
    (zwei:check-interval-sections
      where
      (insert-into-buffer-before
        where
        (make-dummy-definition
          (buffer-editing-type (containing-buffer where)) name type)))
    (when (not (find-single-definition name))
      (catch-error-restart (sys:error "Continue without sectionization.")
        (cbarf "Try CHECK-INTERVAL-SECTIONS instead."
               "CHECK-INTERVAL-SECTIONS failed.")
        (zwei:sectionize-buffer (containing-buffer where))))))

(defun make-dummy-definition (editing-type name type)
  (let ((fn (get 'make-dummy-definition editing-type)))
    (cond ((not fn)
           (editor-barf "Can't hack ~A definitions." editing-type))
          (t (funcall fn name type)))))

(defun (make-dummy-definition :lisp) (name type)
  (text "~%(~A ~S (...) ...dummy definition...)~%"
        (string-downcase type)
        name))

(defun (make-dummy-definition :ada) (name type)
  type ;ignored
  (text "~%entry ~A;~%" name))
```

```
(defun validate-zmacs-buffers-datum (spec buffer-datum)
  (let (((type node) buffer-datum))
    (let ((definition-line (send node ':definition-line)))
      (when (or (not (stringp definition-line))
                (eq (zwei:line-tick definition-line) 'zwei:deleted))
        (zwei:check-interval-sections node)
        (setq definition-line (send node ':definition-line)))
      (multiple-value-bind (fspec ftype)
          (with-local-buffer-pointer (bp
                                      (zwei:create-bp definition-line 0))
            (zwei:get-definition-function-spec bp))
        (unless (and (equal spec fspec)
                     (equal type ftype))
          (zwei:check-interval-sections (send node ':superior)))))))

(defun validate-zmacs-buffers-data (spec)
  (mapc #'(lambda (datum)
            (validate-zmacs-buffers-datum spec datum))
        (si:function-spec-get spec 'zwei:zmacs-buffers)))

(defun find-single-definition-node (spec &optional type)
  (setq type (or type 'DEFUN)) ;allow NIL to mean DEFUN
  (validate-zmacs-buffers-data spec)
  (let ((result (mem #'(lambda (x y) (eq x (car y)))
                     type
                     (si:function-spec-get spec 'zwei:zmacs-buffers))))
    (setq result (del #'(lambda (ignore y)
                          (eq (zwei:line-tick
                                (send (cadr y) ':definition-line))
                              'zwei:deleted))
                      nil result))
    (cond ((or (not result)
               (assq type (cdr result)))
           (values nil (text "~:[No~;Too many~] ~A ~S's found."
                             result
                             (si:definition-type-name type)
                             spec)))
          (t
           (values (cadr (car result)) nil)))))

(defun find-single-definition (spec &optional type)
  (multiple-value-bind (node error-string)
      (find-single-definition-node spec type)
    (values (if node (zwei:create-bp (send node ':definition-line) 0))
            error-string
            node)))

(defun get-definition (&optional (fn (current-editor-definition-name))
                                 (type nil))
  (let ((bp (find-single-definition fn type)))
    (cond ((not bp) nil)
          (t
           (let ((result (get-possibly-cached-definition fn type bp)))
             (if (atom result) nil result))))))  ;hack to bypass fact
                                                 ; this loses on ADA for now

(defun get-definition-text (&optional (fn (current-editor-definition-name))
                                      (type nil))
  type ;ignored
  (multiple-value-bind (start-bp end-bp)
      (bounds-of-definition fn)
    (if (null start-bp) ""
        (string-buffer-between start-bp end-bp))))

(defun get-possibly-cached-definition (fn type bp)
  (when (not type)
    (setq type 'defun))
```

```
  (let ((db (get fn 'cached-definitions)))
    (when (not db)
      (setq db (ncons nil))
      (setf (get fn 'cached-definitions) db))
    (let ((data (get db type)))
      (when (not data)
        (setq data (ncons nil))
        (setf (get db type) data))
      (let ((last-changed (zwei:node-tick (zwei:bp-node bp))))
        (when (or (not (cdr data))
                  (not (eq (car data) last-changed)))
          (setf (cdr data) (really-get-definition fn type bp))
          (setf (car data) last-changed))
        (cdr data)))))

(define-internal-option *debug-parser* "Debug parser problems"
  nil :boolean)

(defun really-get-definition (fn type bp)
  type ;ignored
  (when-error (let ((zwei:*interval* (containing-buffer bp)))
                (let ((package
                        (buffer-package zwei:*interval*)))
                  (multiple-value-bind (start-bp end-bp)
                      (bounds-of-definition fn bp)
                    (let ((fn (get 'get-definition (buffer-editing-type))))
                      (cond ((not fn) (ferror "Bad editing mode."))
                            (t
                             (condition-bind-if
                               *debug-parser*
                               ((sys:error #'(lambda (cond)
                                               (format debug-io "~&~A"
                                                       cond)
                                               (funcall #'dbg)
                                               nil)))
                               (funcall fn start-bp end-bp))))))))))
    (barf "Unable to parse definition of ~S" fn)))

(defun (get-definition :lisp) (start-bp end-bp)
  (read-hacking-/{ (zwei:interval-stream start-bp end-bp t t)))

(defun (get-definition :ada) (start-bp end-bp)
  (ada-parse (zwei:string-interval start-bp end-bp)))

 ;; We might want to make this use *FILL-COLUMN* sometime, but
 ;; that's too hard to access since the *FILL-COLUMN* for a
 ;; different buffer is only accessible by making that buffer
 ;; current... Also, this mechanism may be useful in addition
 ;; to (not necessarily "instead of") checking fill column.

(define-KBE-option *max-buffer-line-length*
                   "Max line length for grinding code in editor"
  79. :number)

(define-kbe-option *kbe-user-role-marker* "KBE role marker (visible)"
  "{" :assoc '(("{"  . "{") ("#{" . "#{")))

(defun grind-code-for-buffer (code buffer)
  (let ((pp:*print-right-margin*
          (or pp:*print-right-margin*
              (min (1- ;guard against fenceposts
                     (// (send terminal-io ':size)
                         (tv:font-char-width
                           (buffer-font buffer))))
                   *max-buffer-line-length*)))
        (package (buffer-package buffer))
        (radix (buffer-radix   buffer)))
    (let ((base     (or radix 10.))
          (*nopoint (if radix t))
```

```
        (*kbe-role-marker* *kbe-user-role-marker*))
      (declare (special *kbe-role-marker*))
      (with-output-to-string (out)
        (print-code code out)))))

(putprop 'define-cliche 'defun 'zwei:definition-function-spec-type)

(defun infer-definition-type (code)
  (if (not (atom code))
      (or (get (car code)
               'zwei:definition-function-spec-type)
          (car code))))

(defun replace-definition (fn code &optional type)
  (if (not type) (setq type (infer-definition-type code)))
  (multiple-value-bind (pt error-message)
      (find-single-definition fn type)
    (cond ((not pt)
           (values nil
                   (text "Can't update ~S's definition: ~A"
                         fn error-message)))
          ((equal (get-definition fn) code)
           (values ':ignored
                   (text "~S's definition didn't need to be updated." fn)))
          (t
           (let ((zwei:*interval* (containing-buffer pt)))
             (let ((text (grind-code-for-buffer code zwei:*interval*)))
               (multiple-value-bind (start-pt end-pt)
                   (bounds-of-definition fn pt)
                 (with-buffer-pointer (saved-start-pt start-pt ':moves)
                   (with-buffer-pointer (saved-end-pt end-pt ':moves)
                     (with-buffer-pointer (temp-pt
                                           (insert-into-buffer-before
                                             start-pt
                                             text)
                                           ':moves)
                       (if (zwei:bp-= (delete-from-buffer-between
                                        temp-pt
                                        saved-end-pt)
                                      (zwei:point))
                           (zwei:move-bp (zwei:point) saved-start-pt))
                       (check-sectionization temp-pt)
                       (check-sectionization saved-start-pt)
                       (check-sectionization saved-end-pt)
                       (when (not (find-single-definition fn))
                         ;; Should never get here (ha,ha) --kmp
                         (when *debug*
                           (editor-echo-output
                             "Sectionizing buffer to find ~S." fn))
                         (zwei:sectionize-buffer zwei:*interval*))))))
               (editor-window-redisplay)
               (values ':replaced
                       (text "Definition of ~S~@[ (in ~A)~] updated."
                             fn
                             (buffer-name-if-relevant
                               zwei:*interval*)))))))))

(defun bounds-of-definition (&optional
                             (fn (current-editor-definition-name))
                             (pt (find-single-definition fn)))
  (if (null pt)
      (values nil nil)
      (let ((type (buffer-editing-type (containing-buffer pt))))
        (let ((handler (get 'bounds-of-definition type)))
          (cond ((not handler)
                 (editor-barf "Can't hack ~A editing." type))
                (t (funcall handler fn pt)))))))

(defun (bounds-of-definition :lisp) (fn pt)
```

```
  fn ;ignored
  (values pt (forward-sexp pt)))

(defun (bounds-of-definition :ada) (fn pt)
  (let ((prev-end (or (search-buffer-backward '#.(text "~%end") pt)
                      (buffer-start (containing-buffer pt)))))
    (let ((start-pt (search-buffer-between
                      prev-end pt '#.(text "~%with"))))
      (unless start-pt
        (setq start-pt (search-buffer-between
                         prev-end pt '#.(text "~%use"))))
      (setq start-pt (if start-pt (beginning-of-line start-pt) pt))
      (let ((end-pt (if (and (eq pt start-pt)
                             (ignore-errors
                               (let ((pt (zwei:forward-over
                                           '(#\sp #\tab)
                                           (forward-sexp pt 2))))
                                 (or (= (char-after pt) #/;)
                                     (= (char-after (forward-sexp pt))
                                        #/;)))))
                        (end-of-line start-pt)
                        (or (search-buffer-forward (text "end ~A;" fn)
                                                   start-pt)
                            (editor-barf "Missing /"end ~A;/"" fn)))))
        (values start-pt end-pt)))))

(define-editor-command com-mark-this-definition (ignore) ()
                       "Mark the definition at the cursor."
  (multiple-value-bind (start-pt end-pt)
      (bounds-of-definition (current-editor-definition-name))
    (establish-region start-pt end-pt))
  nil) ;hopefully no change to buffer
```

```
;;;; Misc

;;; Terminal interaction

(defun editor-last-command-char ()
  zwei:*last-command-char*)

;;; Self-insertion

(defun editor-insert-last-command-char ()
  (zwei:com-standard))

;;; Enters a mail buffer to a given recipient beginning with given text.

(defun send-mail (rcpt initial-text)
  (zwei:com-mail-internal t rcpt initial-text))
```

```
;;; Utilities supporting completion

;;; Utility macros

;;; (BREAKCHAR-P char)
;;;
;;;  Returns true if char is an atom break character according to
;;;  the active syntax table.

(defun breakchar-p (x)
  (zwei:fsm-atom-delimiter-char-p x))

(defun uppercase-p (x)
  (and (^] x #/A) (^\ x #/Z)))

(defun char-after  (bp) (ldb %%ch-char (zwei:bp-char bp)))
(defun char-before (bp) (ldb %%ch-char (zwei:bp-char-before bp)))

;;; (BOUND-TOKEN)
;;;
;;;  Returns NIL if there is no obvious token which completion should
;;;  apply to. Otherwise, returns three values:
;;;
;;;   1 - A string indicating what is probably right to complete
;;;   2 - The starting bp of that string.
;;;   3 - The ending bp of that string.
;;;
;;;  The actual token chosen is not quite a lisp symbol. It is a lisp
;;;  symbol minus any funny chars on one or the other end. Hence,
;;;  being near "*FOO*" will return info just about the "FOO". This
;;;  heuristic can be toyed with, if need be, by changing the funny
;;;  sequence of forward/backward-sexp/word in this definition.

(defun bound-token ()
  (let ((pt (zwei:point)))
    (if (not (breakchar-p (zwei:bp-char pt)))
        (setq pt (zwei:forward-sexp pt)))
    (cond ((not pt) nil)
          ((breakchar-p (zwei:bp-char-before pt)) nil)
          (t (let ((start-bp (when-error (zwei:forward-sexp
                                           (forward-word
                                             (zwei:forward-sexp pt
                                                                -1
                                                                nil
                                                                0
                                                                nil
                                                                nil
                                                                t))
                                           -1 nil 0 nil nil t)
                                         nil)))
                   (let ((end-bp (when-error (forward-sexp
                                               (backward-word
                                                 (forward-sexp start-bp)))
                                             nil)))
                     (if end-bp
                         (values (zwei:string-interval start-bp end-bp)
                                 start-bp
                                 end-bp))))))))
```

```
;;;; Comtabs

(defmacro define-major-mode (mode-name
                             mode-name-string
                             documentation
                             &body body)
  (let ((mode-function-name (intern (text "COM-~A" mode-name)
                                    (symbol-package mode-name)))
        (zwei-var (intern (get-pname mode-name)
                          "ZWEI"))) ;MUST be on ZWEI package!
    '(progn 'compile
            (zwei:defmajor ,mode-function-name ,zwei-var ,mode-name-string
                           ,documentation ()
              ,@body)
            (zwei:set-comtab zwei:*standard-comtab*
                             '()
                             (zwei:make-command-alist
                               '(,mode-function-name))))))

(defvar *kbe-mode-key-bindings*
        '(#\alt     com-complete-KBE-word
          #\help    com-document-KBE-completions
          #\space   com-auto-complete-KBE-word
          #/,       com-auto-complete-KBE-word
          #/.       com-auto-complete-KBE-word
          #/!       com-auto-complete-KBE-word
          #/?       com-auto-complete-KBE-word))

;;; %%% Release 5 %%%
;;; This copies functionality from the Lisp comtab.
;;; When Rel 5 becomes available, there is a much cleaner way
;;; to do this since modes are finally flavorized.

(define-major-mode kbe-mode "KBE"
  "Sets things up for talking to the Knowledge-Based Editor."
  (setq zwei:*space-indent-flag* t)
  (setq zwei:*paragraph-delimiter-list* nil)
  (setq zwei:*comment-start* 'zwei:lisp-find-comment-start-and-end)
  (zwei:set-comtab zwei:*mode-comtab* *kbe-mode-key-bindings*)
  (zwei:set-comtab
    zwei:*mode-comtab* '(#\tab            zwei:com-indent-for-lisp
                         #\rubout         zwei:com-tab-hacking-rubout
                         #\control-rubout zwei:com-rubout
                         #\meta-Z         zwei:com-compile-and-exit
                         #\control-meta-Z zwei:com-evaluate-and-exit)))

(define-zwei-command com-not-implemented-for-this-mode (ignore) ()
                     "Complain that this key is not defined here."
  (editor-barf "~@:C is not yet implemented for ~A mode."
               (editor-last-command-char)
               (symeval zwei:*major-mode*)))

(define-editor-command com-evaluate-this-definition (ignore) ()
                       "Evaluate the definition at the cursor."
  (let ((name (current-editor-definition-name)))
    (let ((form (get-definition name)))
      (cond ((not form) (beep))
            (t (editor-echo-output "Evaluating definition of ~S ..."
                                   name)
               (eval form)
               (editor-echo-output "Done."))))))
  nil) ;hopefully no change to buffer

(define-editor-command com-eval-and-compile-this-definition (ignore) ()
                       "Eval the definition at the cursor.
If the definition name is fbound when done, compile it as well."
  (let ((name (current-editor-definition-name)))
    (let ((form (get-definition name)))
```

```
      (cond ((not form) (beep))
            (t (editor-echo-output "Compiling definition of ~S ..."
                                   name)
               (let ((result (eval form)))
                 (cond ((symbolp result)
                        (cond ((and (eq result (cadr form))
                                    (fboundp result))
                               (compile result)))
                        (editor-echo-output "Done."))
                       (t
                        (editor-echo-output "Failed.")))))))
  nil) ;hopefully no change to buffer

(defun (com-mode-break :lisp) ()
  (break "in Lisp"))

(comment ;why doesn't this work? --kmp 6/4/84

(define-editor-command com-mode-break (ignore) ()
     "Enter a debugging breakpoint in this buffer's language."
  (unwind-protect (let ((zwei:*inside-break* t))
                    (let ((terminal-io zwei:*typeout-window*))
                      (funcall (or (get 'com-mode-break
                                        (buffer-editing-type))
                                   (get 'com-mode-break ':lisp)))))
                  (send zwei:*zmacs-command-loop*
                        ':expose-mode-line-window))
  (send zwei:*typeout-window* ':make-complete)
  nil) ;No change to buffer (probably)

);tnemmoc

(define-major-mode ada-mode "ADA"
  "Sets things up for editing ADA code."
  (zwei:set-comtab zwei:*mode-comtab*
                   '(#\tab zwei:com-indent-relative
                     #\c-m-H  com-mark-this-definition
                     #\c-sh-E com-evaluate-this-definition
                     #\c-sh-C com-eval-and-compile-this-definition
                     #\m-sh-E com-not-implemented-for-this-mode
                     #\m-sh-C com-not-implemented-for-this-mode))
  (setq zwei:*space-indent-flag* t)
  (setq zwei:*paragraph-delimiter-list* nil)
  (setq zwei:*comment-start* "--")
  (setq zwei:*comment-begin* "-- ")
  (setq zwei:*comment-end* "")
  (setq zwei:*comment-column* (* 60. 6))
  (zwei:set-char-syntax (zwei:char-syntax #\space zwei:*list-syntax-table*)
                        zwei:*list-syntax-table* #/;))

 ;; Set up a canonical file type called KBE for KBE command files.

(fs:define-canonical-type :KBE "KBE"
  ((:tops-20 :tenex :unix :vms) "KBE"))

 ;; Set up to have KBE Mode always entered when a .KBE file is read in.

(push '(:KBE . :KBE) fs:*file-type-mode-alist*)

(defvar *kbe-input-window-comtab*
  (zwei:copy-comtab zwei:*standalone-comtab*))

(zwei:set-comtab *kbe-input-window-comtab*
                 *kbe-mode-key-bindings*)

(zwei:set-comtab *kbe-input-window-comtab*
                 '(#\end            com-submit-and-kill-KBE-requests
                   #\abort          com-kill-KBE-requests
                   #\control-meta-Y com-yank-pop-KBE-request
```

```
                #\control-C      com-yank-pop-KBE-request))

(defvar *kbe-mini-buffer-multi-line-comtab*
  (zwei:copy-comtab zwei:*mini-buffer-multi-line-comtab*))

(zwei:set-comtab *kbe-mini-buffer-multi-line-comtab*
                 *kbe-mode-key-bindings*)

(defvar *kbe-mini-buffer-comtab*
  (zwei:copy-comtab zwei:*mini-buffer-comtab*))

(zwei:set-comtab *kbe-mini-buffer-comtab*
                 *kbe-mode-key-bindings*)
```

```
;;;; Streams

;;; (MINI-BUFFER-STREAM prompt multi-line? initial-contents initial-pos)
;;;
;;;   Prompts for input from some sort of mini-buffer with the given
;;;   string as a prompt. If MULTI-LINE? is non-nil, which it defaultly
;;;   is, the input prompted for may extend over more than one line. The
;;;   initial contents of the input buffer, INITIAL-CONTENTS, and location
;;;   of the cursor in that buffer, INITIAL-POS, may also be specified.
;;;
;;;   The return value is a stream to the user's input.

(defvar *multi-mode-info*
        (list ':right-flush
              (text " (~:C completes, ~:C ends)" #\alt #\end)))

(defvar *single-mode-info*
        (list ':right-flush
              (text " (~:C completes, ~:C ends)" #\alt #\return)))

(defun mini-buffer-stream (prompt
                           &optional
                              (multi-line? t)
                              (initial-contents nil)
                              (initial-pos
                                (if initial-contents
                                    (string-length initial-contents))))
  (multiple-value-bind (nil nil node)
      (zwei:edit-in-mini-buffer
        (cond (multi-line? *kbe-mini-buffer-multi-line-comtab*)
              (t           *kbe-mini-buffer-comtab*))
        initial-contents
        initial-pos
        (list prompt (cond (multi-line? *multi-mode-info*)
                           (t           *single-mode-info*))))
    (zwei:interval-stream node)))

;(rest-of-interval-stream (zwei:interval-first-bp node))?


;;; (BUFFER-STREAM buffer-name prompt create?)
;;;
;;;  Returns a stream to some editor buffer given by a string,
;;;  BUFFER-NAME. See FIND-BUFFER for arg convention info.

(defun buffer-stream (&optional (buffer-name zwei:*interval*)
                                (prompt nil)
                                (create? t))
  (zwei:rest-of-interval-stream
    (zwei:interval-first-bp (find-buffer buffer-name prompt create?))))

;;; (REST-OF-BUFFER-STREAM pt)
;;;
;;;  Returns a stream to the rest of some editor buffer after some
;;;  given point.

(defun rest-of-buffer-stream (&optional (pt (zwei:point)))
  (zwei:rest-of-interval-stream pt))
```

```
;;; (GET-CURRENT-TOKEN-COMPLETION table)
;;;
;;;  The table should be something created by MAKE-COMPLETION-TABLE.
;;;
;;;  If the first value returned is NIL, then the second value will
;;;  be an error message saying why no completion was found.
;;;
;;;  If the first value returned is non-NIL, then it will be a bp
;;;  where the second value, a string, should be inserted. The third
;;;  will be the token originally completed.

(defun get-current-token-completion (table)
  (declare (values bp-or-NIL completion-or-diagnostic initial-string-or-n))
  (multiple-value-bind (token start-bp end-bp)
      (bound-token)
    start-bp ;ignored
    (cond ((not token)
           (values nil
                   "Can't do completion at this buffer position."
                   0))
          (t
           (let ((completion (get-completion-from-table token table)))
             (cond ((> (string-length completion) 0)
                    (values end-bp completion token))
                   (t
                    (let ((options (possible-completions token table)))
                      (cond ((not options)
                             (values
                               nil
                               (text "No valid completions for /"~A/""
                                     token)
                               0))
                            ((not (cdr options))
                             (values
                               nil
                               (text "Unique match: /"~A/"" (car options))
                               1))
                            ((member token options)
                             (values
                               nil
                               (text "Exact match of /"~A/" ~
                                        shadows ~{~A~^, ~}."
                                     token
                                     (remove token options))
                               (- (length options))))
                            (t
                             (values
                               nil
                               (text "Completion ambiguity: ~A => ~
                                        ~{~A~^, ~}."
                                     token options)
                               (length options)))))))))))

;;; (COMPLETE-CURRENT-TOKEN table)
;;;
;;;  Causes modifications to the buffer at the active point inserting
;;;  any completion characters possible. If no completion characters
;;;  can be inserted, an error will result unless an error-p arg of NIL
;;;  is specified. Return values are the number of possible matches and
;;;  an error string (or NIL if no error).

(defun complete-current-token (table &optional (error-p t))
  (declare (values n-matches lose-msg))
  (multiple-value-bind (x y z)
      (get-current-token-completion table)
    (cond (x (insert-into-buffer-before x y)
             (values 1 nil))
```

```
        (error-p (editor-barf y)
                 (values z y)) ;should never get to here, but ...
        (t (values z y)))))

;;; (DOCUMENT-COMPLETIONS table)
;;;
;;;  Types out in the editor echo area what effect completion would
;;;  have at the current buffer position.

(defun document-completions (table)
  (multiple-value-bind (x y z)
      (get-current-token-completion table)
    (cond (x (editor-echo-output "Completion here will change ~A => ~A~A"
                                 z z y))
          (t (editor-echo-output y)))))
```

```
;;; Input window -- a kind of standalone editor

(defflavor edit-window () (margin-scroll-bar-mixin
                           zwei:standalone-editor-window))

(defflavor edit-window-input-commands-mixin () ()
  (:required-flavors zwei:editor)
  (:default-init-plist :*comtab* *kbe-input-window-comtab*))
```

```
;;; Stream into buffer's region if any

(defun region-stream ()
  (cond ((zwei:window-mark-p zwei:*window*)
         (zwei:interval-stream (zwei:point)
                               (zwei:mark)
                               (zwei:bp-< (zwei:point) (zwei:mark))))
        (t nil)))

(defun interval-stream (bp1 bp2)
  (zwei:interval-stream bp1 bp2))
```

```
;;;; Recentering Windows

(defun move-point-to-top-of-window (bp)
  (zwei:recenter-window zwei:*window* ':start bp))
```

```
;;;; Command Type

;;; ZWEI keeps track of our current and previous command type so that
;;; we can find out what kind of command was done last. This is good
;;; for things like finding out when doing a m-Y that we had just done
;;; a c-Y before, or for finding out when doing c-m-Y if we have just
;;; done a c-m-Y already before that.

(defun set-editor-command-type (type)
  (setq zwei:*current-command-type* type))

(defun last-editor-command-type ()
  zwei:*last-command-type*)
```

```
;;;; Support Buffers

(defun push-possibilities-buffer (action type &rest set-options)
  (assure-real-editor-buffer)
  (lexpr-funcall-in-editor #'zwei:push-possibilities-buffer
                           action type set-options))
```

```
(defun initialize-editor-window (window)
  ;; The editor needs to get selected so it can create a buffer
  ;; for the window, etc. It is important in all this to give
  ;; windows enough time to switch so we don't deadlock...
  (send window ':select)                          ;Allow initialization
  (process-wait "Select Editor"                   ;Await editor selection
                #'(lambda (window)
                    (do ((w tv:selected-window (send w ':superior)))
                        ((not w) nil)
                      (if (eq w window) (return t))))
                window)
  (funcall-in-editor #'redisplay-all-windows)   ;Maybe wait some more
  (let ((tw (send tv:selected-window ':send-if-handles ':typeout-window)))
    (if tw (send tw ':set-font-map (list tv:*default-font*))))
  (let* ((buffer (funcall-in-editor #'current-editor-buffer))
         (buffer-name (send buffer ':name)))
    (when (and (string-equal buffer-name "*Buffer-" 0 0 8)
               (not (string-equal buffer-name "*Buffer-1*")))
      (funcall-in-editor #'make-buffer-current
                         (funcall-in-editor #'find-buffer "*Buffer-1*"))
      (funcall-in-editor #'kill-buffer buffer)
      (funcall-in-editor #'redisplay-all-windows)))
  (send terminal-io ':select))                    ;Take back the terminal
```

```
;;;; ADA sectionization support

;;; %%% Release 5 %%%
;;;
;;; The following magic works with <KMP.LISPM.BUGS.FIXES>GET-SECTION-NAME
;;; in order to define a minimal sectionization strategy for ADA-MODE
;;; buffers.

(defprop zwei:ada-mode :ada zwei:editing-type)

(defun (zwei:get-section-name :ada) (line bp)
  (declare (return-list fspec type str error-p))
  (let ((type nil))
    (if (or (%string-equal line 0 "procedure" 0 9) ; ADA procedure
            (%string-equal line 0 "function" 0 8)  ; ADA function
            (%string-equal line 0 "package" 0 7)   ; ADA package
            (%string-equal line 0 "cliche" 0 6)    ; Cliche
            (%string-equal line 0 "entry" 0 5))    ; Dummy definition
        (get-ada-function-spec (zwei:move-bp bp line 0) type)
        (values nil nil nil t))))

(defun get-ada-function-spec (bp &optional type &aux bp1)
  (zwei:atom-word-syntax-bind
    ;; Now get the second word after BP.
    (if (and (setq bp (zwei:forward-word bp))
             (setq bp (zwei:forward-over zwei:*blanks* bp))
             (setq bp1 (zwei:forward-word bp)))
        (let ((string (zwei:string-interval bp bp1)))
          (values (let ((package (buffer-package (containing-buffer bp))))
                    (read-from-string string))
                  (or type 'DEFUN)
                  string))
        (values nil nil nil t))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode: LISP; Base: 10; Package: ZWEI; Fonts: MEDFNB; -*-
```

```
;;;; Unfinished-Edits Subnodes

(defflavor unfinished-edits-subnode (strings) (function-spec-subnode)
  :initable-instance-variables
  :gettable-instance-variables)

(defmethod (unfinished-edits-subnode :description) (stream)
  (format stream "Edit ~A ~S~%~{ ~A~%~}"
          (si:definition-type-name (send self ':definition-type))
          (send self ':function-spec)
          strings))
```

```
;;;; Unfinished-edits Nodes

(define-special-buffer-type
  unfinished-edits-buffer
  ((context nil)
   (tagged-string-list nil))
  (header-description-1-subnode-special-purpose-buffer-mixin
   possibilities-special-purpose-buffer)
  :unfinished-edits "Unfinished-Edits"
  (:buffer-init-options :set-context :set-tagged-string-list)
  :settable-instance-variables)


(defmethod (unfinished-edits-buffer :header-description-1) (stream plural-p)
  (if context
      (format stream "Unfinished edit~P~@[ in ~A~]"
              plural-p context)))

(defmethod (unfinished-edits-buffer :document) (ignore)
  (format t "Complete unfinished edits~@[ in ~A~]." context))

(defmethod (unfinished-edits-buffer :reusable) (ignore) t)

(defmethod (unfinished-edits-buffer :revert) ()
  (do ((l tagged-string-list (cdr l)))
      ((null l))
    (send self ':add-node 'unfinished-edits-subnode
          ':function-spec   (caar l)
          ':definition-type 'defun
          ':strings         (cdar l))))

(defmethod (unfinished-edits-buffer :description) ()
  (or context ""))

(defmethod (unfinished-edits-buffer :list-subnodes) (first-node)
  (do ((node first-node (node-next node)))
      ((not node))
    (format t "~&For the ~A "
            (si:definition-type-name (send node ':definition-type)))
    (send *typeout-window* ':item 'function-name
          (send node ':function-spec))
    (format t "~%~{ ~A~%~}~%" (send node ':strings))))
```

```
;;; This fixes a bug in SUBNODE-SPECIAL-PURPOSE-BUFFER.
;;; It is claimed to be fixed in Release 5, at which time this code
;;; should be discarded if possible.

(defmethod (unfinished-edits-buffer :add-node) (flavor &rest options)
  (let ((node (lexpr-funcall #'make-instance flavor options)))
    (create-interval nil nil node)
    (send interval-stream ':set-bp (interval-last-bp node))
    (send node ':description interval-stream)
    (let* ((infs (last inferiors))
           (inf (car infs)))
      (setf (node-next inf) node)
      (setf (node-previous node) inf)
      (rplacd infs (ncons node))
      (setf (node-superior node) self)
      (let ((inf-last-line   (bp-line (interval-last-bp  inf)))   ;changed
            (node-first-line (bp-line (interval-first-bp node)))) ;changed
        (setf (line-next inf-last-line) node-first-line)          ;changed
        (setf (line-previous node-first-line) inf-last-line))     ;changed
      (move-bp last-bp (interval-last-bp node))                   ;changed
      (move-bp (interval-last-bp inf) (interval-first-bp node)))
;   (setf (node-tick node) ':read-only) ;commented out in the original
    node))
```

```
(compile-flavor-methods unfinished-edits-subnode
                        unfinished-edits-buffer)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1984, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; BELIEF: Belief structure for editing actors, etc.

;;; BELIEF-STRUCTURE
;;;
;;;  A collection of beliefs.

(defflavor belief-structure (storage lock-cell) (name-mixin)
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :name    "Anonymous"
    :storage (ncons nil)
    :lock-cell nil))

(defmethod (belief-structure :reset) ()
  (with-lock (lock-cell :whostate "Await Beliefs")
    (setq storage (ncons nil))))

(defmethod (belief-structure :believe) (belief-type belief-subject val)
  (with-lock (lock-cell :whostate "Await Beliefs")
    (let ((subject-cell (get storage belief-subject)))
      (when (not subject-cell)
        (setq subject-cell (ncons belief-subject))
        (setf (get storage belief-subject) subject-cell))
      (setf (get subject-cell belief-type) val)
      val)))

(defmethod (belief-structure :belief) (belief-type belief-subject)
  (with-lock (lock-cell :whostate "Await Beliefs")
    (get (get storage belief-subject) belief-type)))

(defmethod (belief-structure :disbelieve) (belief-type belief-subject)
  (with-lock (lock-cell :whostate "Await Beliefs")
    (let ((subject-cell (get storage belief-subject)))
      (remprop subject-cell belief-type)
      (when (not (cdr subject-cell))
        (and (remprop storage belief-subject))))
    nil))

(defmethod (belief-structure :map-over-beliefs) (fn)
  (with-lock (lock-cell :whostate "Await Beliefs")
    (do ((b (cdr storage) (cddr b)))
        ((null b))
      (do ((l (cdadr b) (cddr l)))
          ((null l))
        (funcall fn (car l) (car b) (cadr l))))))

(defmethod (belief-structure :map-over-beliefs-about) (belief-subject fn)
  (with-lock (lock-cell :whostate "Await Beliefs")
    (do ((l (cdr (get storage belief-subject)) (cddr l)))
        ((null l))
      (funcall fn (car l) (cadr l)))))

(defmethod (belief-structure :map-over-beliefs-of-type) (belief-type fn)
  (with-lock (lock-cell :whostate "Await Beliefs")
    (do ((b (cdr storage) (cddr b)))
        ((null b))
```

```
(do ((l (cdadr b) (cddr l)))
    ((null l))
  (when (eq (car l) belief-type)
    (funcall fn (car b) (cadr l)))))))
```

```
;;;; Initialization

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods belief-structure)
```

```
;;;; Believer Mixin

(defvar *beliefs* (make-instance 'belief-structure ':name "Main"))

(defflavor believer-mixin (beliefs) ()
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
   :beliefs *beliefs*))

(defmethod (believer-mixin :believe) (belief-type belief-subject val)
  (send beliefs ':believe belief-type belief-subject val))

(defmethod (believer-mixin :belief) (belief-type belief-subject)
  (send beliefs ':belief belief-type belief-subject))

(defmethod (believer-mixin :disbelieve) (belief-type belief-subject)
  (send beliefs ':disbelieve belief-type belief-subject))

(defmethod (believer-mixin :map-over-beliefs-about) (belief-subject fn)
  (send beliefs ':map-over-beliefs-about belief-subject fn))

(defmethod (believer-mixin :map-over-beliefs-of-type) (belief-type fn)
  (send beliefs ':map-over-beliefs-of-type belief-type fn))

(defmethod (believer-mixin :map-over-beliefs) (fn)
  (send beliefs ':map-over-beliefs fn))

(define-behavior (believer-mixin belief) (&rest args)
  (return (lexpr-send self ':belief args)))

(define-behavior (believer-mixin believe) (&rest args)
  (lexpr-send self ':believe args))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; Lisp WITH-LOCK Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;;; EDIT: The "Edit" Frame for the KBE.
```

```
;;;; Edit Frame

;;; EDIT-FRAME
;;;
;;; A constraint frame for holding onto the other KBE window
;;; frames in our demo configuration.

(defflavor edit-frame ()
  (delegate-mixin                       ; We can have an associated actor
   tv:constraint-frame)                 ; We can have sub-windows
  :initable-instance-variables
  :settable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :delegate *initial-editing-actor-secretary*
    :deexposed-typeout-action ':permit
    :blinker-p nil
    :activate-p t
    :panes '((editor-pane   editor-pane      )
             (status-pane   status-pane       :borders (2 0 2 2)))
    :constraints '((edit  . ((editor-pane
                              status-pane)
                             ((status-pane   1     :lines    ))
                             ((editor-pane   :even           )))))
    :selected-pane 'editor-pane))

(defmethod (edit-frame :remark-window) ()
  (get-pane self 'status-pane))

(defmethod (edit-frame :typeout-window) ()
  (make-syn-stream 'terminal-io))

;;; :NAME-FOR-SELECTION
;;;
;;;  The LispM window system will send this message to a window to find
;;;  if it wants to be selected. If it returns NIL, it does not want to
;;;  be selected. If it returns non-NIL, the return value should be a
;;;  string to be used in selecting the window.

(defmethod (edit-frame :name-for-selection) ()
  (format nil "KBE ~A" (send tv:selected-pane ':name-for-selection)))

;;; :SELECTABLE-WINDOWS
;;;
;;;  The LispM window system will send this message to a window to find
;;;  what parts of it can be selected. The default for frames is that they
;;;  and any of their inferiors will be selectable if they answer non-NIL
;;;  to the :NAME-FOR-SELECTION message. The return value expected is an
;;;  alist of the form ((window-name . window) ...)
;;;
;;;  To avoid cluttering the system menu with info about our inferiors,
;;;  KBE edit frames explicitly handle this message, returning information
;;;  only about the frame itself and not its inferiors.

(defmethod (edit-frame :selectable-windows) ()
  (list (list (send self ':name-for-selection) self)))
```

```
;;;; EDITOR-PANE

(defflavor editor-pane ()
     (tv:borders-mixin                    ; We need a border around us
      tv:pane-mixin                       ; Allow us to be a sub-pane
      vanilla-editor-frame)               ; And, of course, an editor!
  (:default-init-plist
    :border-margin-width 0
    :deexposed-typeout-action ':permit
    :save-bits t
    :activate-p t))

;;; :MOUSE-SELECT
;;;
;;;  Handling this message allows us to do something interesting if the
;;;  user does a left click on our window. We do the obvious thing here;
;;;  ie, we select our pane.

(defmethod (editor-pane :mouse-select) (&rest ignore)
  (send self ':select))
```

```
;;;; Methods Edit Actor will prefer

;;; :TYPEOUT
;;;
;;;  We intentionally don't handle this message.
;;;  Let the guy type on our typeout window if he wants to.
;;;  This message must return T or NIL.

(comment ;this doesn't get used, does it? --kmp 1/1/84

(defmethod (editor-pane :typeout) (string)
  string
  nil) ;not handled

)
```

```
;;;; Initialization

;; We have to match the way ZWEI defaults its window size...

(let ((main-screen-info (assq tv:main-screen
                              tv:*default-window-size-alist*)))
  (when main-screen-info
    (let ((zmacs-frame-info (assq 'zwei:zmacs-frame
                                  (cdr main-screen-info))))
      (when zmacs-frame-info
        (lexpr-funcall #'tv:set-default-window-size
                       'edit-frame tv:main-screen nil
                       (cdr zmacs-frame-info))))))

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods edit-frame editor-pane)

;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; EDIT-ACTOR: The "Edit" Frame's associated Actor.

;;;; Option Variables

(define-internal-option *pending-code-notification*
                        "Notification of available code"
  #+3600 "the screen flashes"
  #-3600 "the tone sounds"
  :sexp)

(define-internal-option *pending-typeout-notification*
                        "Notification of pending typeout"
  #+3600 "the screen flashes"
  #-3600 "the tone sounds"
  :sexp)

;;; EDIT-MIXIN

(defflavor edit-mixin () ())

(define-behavior (edit-mixin answer) (the-answer)
  (let ((requests (send self ':pending-requests)))
    (with-lock ((car requests) :whostate "Await Queue")
      (cond ((stringp the-answer)
             (push '(:text :text ,the-answer) (cdr requests)))
            ((not (atom the-answer))
             (push the-answer (cdr requests))))
      (when (and (get (cadr requests) ':text)
                 *pending-typeout-notification*
                 (not (editor-awaiting? self)))
        (make-noise-about 'display-ready)))))
```

```
;;;; EDITING-ACTOR
;;;
;;;  This guy is responsible for managing queues going to/from a normal
;;;  full-editor window such as Zwei.
;;;
;;;  Notes:
;;;
;;;   There is some question about whether the same actor ought to serve
;;;   all edit windows. Do people think of different editor windows as
;;;   different entities? Would they if they could? For now we punt this
;;;   issue and make actors and windows be one-to-one since the simple
;;;   case is of just one edit window. Changing things later probably
;;;   only involves very local changes to this section where messages
;;;   are received and dispatched.
;;;
;;;   Also, PENDING-REQUESTS may get hacked both from the background
;;;   window and the foreground window. Really the foreground
;;;   window should have its own delegate and should interface
;;;   via real message passing also. When this is done, the
;;;   cheap locking strategy used with having PENDING-REQUESTS
;;;   be (lock . queue) can be changed into a simpler lock-free
;;;   strategy, allowing locking to be a side-effect of mailbox
;;;   communication.

(defflavor editing-actor ((suggested-focus nil)
                          (pending-requests (ncons nil)))
           (edit-mixin employer-mixin standard-actor)
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

;;; Possibilities stuff

(defvar *awaiting* (list nil nil))

(defun editor-awaiting? (me)
  me ;ignored
  (car *awaiting*))

(comment
  (let ((state (editor-state)))
    (and (string-equal state "Await " 0 0 6)
         (send me ':name? (substring state 6))))
);ntemmoc

;;; >Report status.

(define-interpretation (editing-actor report) ()
  (let ((base 10.) (*nopoint t))
    (with-output-to-string (str)
      (format str "I have ~A pending edit request~:P to perform.~
                   ~@[~%~A~]"
              (let ((n (length (cdr (send self ':pending-requests)))))
                (cond ((zerop n) "no")
                      (t n)))
              (let ((apprentice (send self ':associated-apprentice)))
                (when apprentice
                  (text "~A: ~A"
                        apprentice
                        (ask apprentice task "Report status."))))))))


;;; CODER-CONFIRMATION name new-code
;;;
;;;  This says that coding has occurred and some new piece of code is now
;;;  available for use.

(define-noise code-ready function-name)
```

```
(define-behavior (editing-actor coder-confirmation) (fn-name code)
  (let ((old-belief (send self ':belief ':code fn-name)))
    (send self ':believe ':code fn-name code)
    (let ((new-definition? (not (equal old-belief code))))
      (when new-definition?
        (send self ':update-definition-info fn-name ':code code)))))

;;; DOCUMENT-CODE name documentation
;;;
;;;  This provides documentation for a given piece of code.

(define-behavior (editing-actor document-code) (fn-name doc)
  (send self ':update-definition-info fn-name ':documentation doc))

(defmethod (editing-actor :update-definition-info) (fn-name marker value)
  (let ((requests (send self ':pending-requests)))
    (with-lock ((car requests) :whostate "Await Queue")
      (let ((entry (dolist (item (cdr requests))
                     (if (eq (get item ':name) fn-name) (return item)))))
        (cond ((not entry) (push (list ':definition
                                       ':name fn-name
                                       marker value)
                                 (cdr requests)))
              (t
               (putprop entry value marker))))))
  (when (and *pending-code-notification*
             (not (editor-awaiting? self)))
    (make-noise-about 'code-ready fn-name)))

;;; >Typeout "...string...".
;;;
;;;  This requests that an actor do literal typeout of the text in
;;;  a given string.

(define-noise display-ready)

(define-interpretation (editing-actor typeout) (string)
  (let ((requests (send self ':pending-requests)))
    (with-lock ((car requests) :whostate "Await Queue")
      (push '(:text :text ,string) (cdr requests))))
  (when (and *pending-typeout-notification*
             (not (editor-awaiting? self)))
    (make-noise-about 'display-ready)))

(comment ;think about this. hint: it doesn't work. -kmp

(define-interpretation (editing-actor analyze) (reference)
  (let ((cmd (cond ((eq (car reference) 'code)
                    (unparse-command '(analyze ,reference)))
                   (t
                    (text "Analyze $~S"
                          (let ((ref-string (unparse-command reference)))
                            (or (get (task (text "How is ~A defined?"
                                                 ref-string))
                                     ':definition)
                                (complain "I don't know about ~A"
                                          ref-string)))))))))
    (send self ':defer-task cmd)))

)

(define-interpretation (editing-actor how) (toplevel-reference)
  (let ((result (send self ':belief ':code
                      (name-symbol (caddr toplevel-reference)))))
    '(:code
      :text ,(cond (result (let (prinlevel prinlength) (text "~S" result)))
                   (t (text "I don't know the definition of ~A"
                            (unparse-command toplevel-reference)))))
```

```
      :definition ,result)))

;;; SUGGEST-FOCUS reference-list path-list
;;;
;;;  This accepts a list of references to focus on. We should define what
;;;  it means to focus on things. Maybe it should just change our local
;;;  version of IT or something. Who knows. Anyway, ignore this for now.

(define-behavior (editing-actor suggest-focus) (data)
  (send self ':set-suggested-focus data))

(define-behavior (editing-actor suggested-focus) ()
  (return (send self ':suggested-focus)))

(defvar *initial-editing-actor* nil)

(define-system-initialization editing-actor
  (setq *initial-editing-actor* (make-instance 'editing-actor)))
```

```
(defflavor editing-actor-secretary ()
          (edit-mixin employer-mixin standard-actor)
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables
  (:default-init-plist
     :associated-apprentice (send *initial-editing-actor* ':name)
     :window-flavor  'edit-frame))

(defwhopper (editing-actor-secretary :submit-by-proxy) (x &optional await?)
  (cond (await?
         (with-lock ((car *awaiting*) :whostate "Await editor")
           (continue-whopper x await?)))
        (t
         (continue-whopper x await?))))

(defmethod (editing-actor-secretary :next-pending-request) ()
  (let ((requests (send self ':pending-requests)))
    (with-lock ((car requests) :whostate "Await Queue")
      (pop (cdr requests)))))

(define-KBE-option *focus-style* "Focus style"
  ':temporary
  :assoc '(("Invisible" . nil)
           ("Temporary" . :temporary)
           ("Permanent" . t)))

(defvar *old-definition* nil)

;these two hook fns assume they run without the user changing
;anything relevant in between.

(defun default-mark-pre-hook (data)
  (let ((fn (get (locf data) ':name)))
    (setq *old-definition* (get-appropriate-definition fn))))

(defun default-mark-post-hook (data)
  (declare (special *input-language*))
  (caseq *input-language*
    (ada (set-cursor-for-ada data))
    (T (set-cursor-for-lisp data))))

(defun get-appropriate-definition (fn)
  (declare (special *input-language*))
  (when fn
    (caseq *input-language*
      (ada (get-definition-text fn))
      (T (get-definition fn)))))
```

```
(defun set-cursor-for-lisp (data)
  (let* ((fn (get (locf data) ':name))
         (path (get (locf data) ':path))
         (highlight? (get (locf data) ':force-highlight))
         (current-definition (get-appropriate-definition fn)))
    (when fn
      (if highlight?
          (if path (mark-sexp-from-path fn path *focus-style*))
          (if (not (equal current-definition *old-definition*))
              (multiple-value-bind (path at-end?)
                  (compare-sexps-for-mark
                    *old-definition* current-definition)
                (mark-sexp-from-path fn path nil at-end?)))))))

;here the traditional path information never makes any sense.
;also we cannot actually highlight a region at this time.
;or do anything based on the underlying sexprs.

(defun set-cursor-for-ada (data)
  (let* ((fn (get (locf data) ':name))
         (current-definition (get-appropriate-definition fn)))
    (when fn
      (if (not (equal current-definition *old-definition*))
          (let (((x y) (compare-texts-for-mark
                         *old-definition* current-definition)))
            (mark-sexp-from-offsets fn x y nil))))))

;the following are minimal comparitors which will be improved soon

(defun compare-sexps-for-mark (old new)
  (first-sexp-difference old new))

(defun compare-texts-for-mark (old new)
  (first-text-difference old new))
```

```
;assumes no dotted cdrs says that two whole lists are
;different if either one has an atomic car and the cars
;are different.

(defun first-sexp-difference (old new)
  (declare (special at-end?))
  (let (at-end?)
    (values (delq T (first-sexp-difference1 old new)) at-end?)))

;Note it will be buggy if we ever have to
;go inside of a {} thing.

(defun first-sexp-difference1 (old new)
  (declare (special at-end?))
  (cond ((equal old new) nil)
        ((and (listp old) (eq (car old) 'kbe-/#-comment-macro)
              (equal new (caadr old)) new)
         nil) ;ignore simple removal of roles
        ((not (and (listp old) (listp new))) (ncons T))
        ((and (not (eq (car old) (car new)))
              (or (symbolp (car old)) (symbolp (car new)))) (ncons T))
        ((and (eq (car old) 'kbe-/#-comment-macro)
              (not (equal (caddr old) (caddr new)))
              (ncons T)))
        (T (let* ((old-len (length old))
                  (new-len (length new))
                  (end (1- (min old-len new-len)))
                  (is-let* (eq (car new) 'let*))
                  (let*-vars-dif))
             (do ((i 0 (1+ i)))
                 ((> i end)
                  (cond (let*-vars-dif)
                        ((> new-len old-len) (ncons i))
                        ((> old-len new-len)
                         (setq at-end? t) (ncons (1- i)))
                        (T (ncons T))))
               (let ((inner-dif
                       (first-sexp-difference1 (nth i old) (nth i new))))
                 (when inner-dif
                   (if (and is-let* (= i 1))
                       (setq let*-vars-dif (cons i inner-dif))
                       (return (cons i inner-dif))))))))))
```

```
;this assumes it is being passed to top level ada functions
;it breaks them up into regions and returns a difference between
;them, prefering the body to the earlier stuff.

(defun first-text-difference (old new)
  (let* (((old-with old-dcls old-body) (ada-text-breakup old))
         ((new-with new-dcls new-body) (ada-text-breakup new)))
    (cond ((string-equal old "") (list 0 0))
          ((first-text-difference1 old-body new-body
             (+ (num-lines new-with) (num-lines new-dcls))))
          ((first-text-difference1 old-dcls new-dcls (num-lines new-with)))
          ((first-text-difference1 old-with new-with))
          (T (list 0 0)))))

(defun num-lines (string)
  (do ((spot -1 (string-search-char #\return string (1+ spot)))
       (lines -1 (1+ lines)))
      ((null spot) (return lines))))

;we assume that there must be a dcls start if anything.
(defun ada-text-breakup (string)
  (let* ((dcls-start
           (1+ (or (string-search #.(format nil "~%cliche") string)
                   (string-search #.(format nil "~%package") string)
                   (string-search #.(format nil "~%procedure") string)
                   (string-search #.(format nil "~%function") string)
                   -1)))
         (body-start
           (1+ (or (string-search #.(format nil "~%begin") string)
                   (1- dcls-start))))
         (last-open (max (or (string-reverse-search
                               "open(" string
                               (string-length string) body-start)
                             0)
                         (or (string-reverse-search
                               "create(" string
                               (string-length string) body-start)
                             0))))
    (if (not (zerop last-open))
        (setq body-start (1+ (or (string-search-char
                                   #\return string last-open)
                                 (1- body-start)))))
    (list (substring string 0 dcls-start)
          (substring string dcls-start body-start)
          (substring string body-start))))

(defun first-text-difference1 (old new &optional initial-y)
  (let ((x 0) (y (or initial-y 0))
        (old-end (1- (string-length old)))
        (new-end (1- (string-length new)))
        (old-i 0) (new-i 0)
        old-c new-c
        (difference? T))
    (do () (nil)
      L (when (<= old-i old-end)
          (setq old-c (aref old old-i))
          (when (blank-stuff old-c)
            (incf old-i)
            (go L)))
      M (when (<= new-i new-end)
          (setq new-c (aref new new-i))
          (caseq new-c
            (#\return (incf y) (setq x 0))
            (T (incf x)))
          (when (blank-stuff new-c)
            (incf new-i)
            (go M)))
      (when (or (> old-i old-end) (> new-i new-end))
```

```
        (if (and (> old-i old-end) (> new-i new-end))
            (setq difference? nil)
          (decf x))
        (return))
      (cond ((= new-c old-c) (incf old-i) (incf new-i))
            (T (do ((j new-i (1- j))) ((minusp j))
                 (if (not (alpha-numeric (aref new j))) (return))
                 (decf x))
               (return))))
    (if difference? (list x y))))

(defun alpha-numeric (c)
  (or (= c #/_)
      (and (<= #/0 c) (>= #/9 c))
      (and (<= #/A c) (>= #/Z c))
      (and (<= #/a c) (>= #/z c))))

(defun blank-stuff (c)
  (member c '#.(list #\space #\tab #\return)))
```

```
(defvar *mark-pre-finish-hook* 'default-mark-pre-hook)

(defvar *mark-post-finish-hook* 'default-mark-post-hook)

(defmethod (editing-actor-secretary :finish-requests) (&optional confirm?)
  (funcall *mark-pre-finish-hook* (send self ':suggested-focus))
  (do ((entry (send self ':next-pending-request)
              (send self ':next-pending-request)))
      ((not entry)
       (funcall *mark-post-finish-hook* (send self ':suggested-focus)))
    (send self ':finish-request entry confirm?)))

(defmethod (editing-actor-secretary :finish-request)
           (data &optional (confirm? nil))
  (when (or (not confirm?)
            (not (eq (car data) ':definition))
            (fquery '() "Update definition of ~A?" (get data ':name)))
    (selectq (car data)
      ((:text)
       (format t "~@[~&~A~&~]" (get data ':text)))
      ((:possibilities)
       (push-possibilities-buffer
         ':list-possibilities     (get data ':type)
         ':set-context            (get data ':context)
         ':set-tagged-string-list (get data ':data)))
      ((:definition)
       (let ((val nil)
             (fn-name (get data ':name)))
         (cond-every
           ((setq val (get data ':code))     ;must precede :document
            (replace-or-create-definition fn-name val))
           ((setq val (get data ':documentation))
            (comment-definition fn-name val)))))
      (otherwise
       (cbarf "Proceed, ignoring this item."
              "Ill-formed entry in KBE's pending-edit queue.")))))

;;; PRESENTATIONAL-FOCUS
;;;
;;;  Eventually we should probably read the focus info in the buffer,
;;;  but for now let's just make it be the whole definition we're in.

(define-behavior (editing-actor-secretary presentational-focus) ()
  (multiple-value-bind (definition-name definition-type)
      (current-editor-definition-name)
    (let ((type-name (selectq definition-type
                       (defun "program")
                       (define-cliche "cliche")
                       (otherwise nil))))
      (return
        (if definition-type
            '(:reference
               ,(if (and type-name
                         (symbolp definition-name))
                    (text "the ~A ~S" type-name definition-name))
               :type ,definition-type
               :name ,definition-name
               :path ,(current-editor-path)))))))
```

```
  ;;; KLUDGE definitions for now...

(define-behavior (editing-actor-secretary suggest-focus) (data)
  (send self ':suggest-focus data))

(define-behavior (editing-actor-secretary suggested-focus) ()
  (return (send self ':suggested-focus)))

(defmethod (editing-actor-secretary :pending-requests) ()
  (send (parse-actor (send self ':associated-apprentice))
        ':pending-requests))

(defmethod (editing-actor-secretary :suggest-focus) (data)
  (send (parse-actor (send self ':associated-apprentice))
        ':set-suggested-focus data))

(defmethod (editing-actor-secretary :suggested-focus) ()
  (send (parse-actor (send self ':associated-apprentice))
        ':suggested-focus))

(defvar *initial-editing-actor-secretary* nil)

(define-system-initialization setup-editor
  (setq *initial-editing-actor-secretary*
        (make-instance 'editing-actor-secretary))
  (initialize-editor-window
    (send *initial-editing-actor-secretary* ':window)))

(defun editor-delegate () *initial-editing-actor-secretary*)
```

```
;;;; Initialization

;;; Request compile-time compilation of flavors. By default, the
;;; methods would not get compiled until fasload-time otherwise.

(compile-flavor-methods editing-actor editing-actor-secretary)

;;; Local Modes:
;;; Mode:LISP
;;; Lisp WHEN Indent:1
;;; Lisp UNLESS Indent:1
;;; Lisp WITH-OUTPUT-TO-STRING Indent:1
;;; Lisp WITH-INPUT-FROM-STRING Indent:1
;;; End:
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; CONSISTENCY: Tools for editor consistency checks

;;; (REMOVE-VANILLA form)
;;;
;;;  This recursively descends form (not recursing into atoms, including
;;;  arrays or structures, and changes references to VANILLA:anything to
;;;  be the active package.
;;;
;;;  This might not be the most obvious place for this definition to be,
;;;  but it will do for now... Note that the reason it's here and not
;;;  in VERBS with 2RENAME is that in fact, we don't want to bind
;;;  ourselves to a package until the very last minute. This gives the
;;;  user the option of changing his package in the editor buffer without
;;;  the KBE having to know and still maximizing the chances that the
;;;  local variables, go tags, etc. come out in the preferred package.

(defun remove-vanilla (form &optional (pkg package))
  (cond ((null form) nil)
        ((symbolp form)
         (cond ((eq (symbol-package form) *Vanilla-package*)
                (intern (string form) pkg))
               (t form)))
        ((atom form) form)
        (t
         (do ((f form (cdr f))
              (l nil (cons (remove-vanilla (car f) pkg) l)))
             ((atom f)
              (nreconc l (cond (f (remove-vanilla f pkg))
                               (t nil))))))))

(defun replace-definition-verbose (name code)
  (multiple-value-bind (success-indicator msg)
      (replace-definition name (remove-vanilla code))
    (when success-indicator
      (code-is-consistent name code))
    (when (not (eq success-indicator ':ignored))
      (editor-echo-output "~A" msg))))

(defun code->language (code)
  (cond ((ada? code) ':ada)
        (t ':lisp)))

(defun replace-or-create-definition (name code &optional type)
  (if (not type) (setq type (infer-definition-type code)))
  (cond ((not (find-single-definition name type))
         (validate-buffer-mode (code->language code) name)
         (validate-buffer-package)
         (create-dummy-definition name type)
         (replace-definition-verbose name code))
        ((not (code-is-consistent? name type))
         (editor-echo-output "~S's definition has changed." name)
         (if (y-or-n-p "Shall I show you how? ")
             (explain-how-code-is-superseded name))
         (cond ((y-or-n-p "Update definition anyway? ")
                (replace-definition-verbose name code))
               (t
                (editor-echo-output "Ok, not updating ~S's definition."
                                    name))))
```

```
(t
 (let ((pointer (find-single-definition name type)))
   (let ((saved-buffer (current-editor-buffer))
         (new-buffer   (containing-buffer pointer)))
     (unwind-protect
       (progn (make-buffer-current new-buffer)
              (validate-buffer-mode (code->language code) name)
              (validate-buffer-package)
              (replace-definition-verbose name code))
       (make-buffer-current saved-buffer)))))))
```

```
;;;; Code consistency checking

(define-KBE-option *check-code-consistency*
                   "Check code consistency during buffer updates"
  t :boolean)

 ;; Yuck! This check for kbe-/#-comment-macro
 ;; seems somehow ill-placed here. It seems
 ;; to violate the current level of abstraction. Probably it should
 ;; go in some other module.

(defun code-equal (x y)
  (cond ((or (and (not (atom x))
                  (eq (car x) 'kbe-/#-comment-macro))
             (and (not (atom y))
                  (eq (car y) 'kbe-/#-comment-macro)))
         (let ((code-x (list x)) (name-x nil)
               (code-y (list y)) (name-y nil))
           (cond ((and (not (atom x)) (eq (car x) 'kbe-/#-comment-macro))
                  (setq code-x (cadr x))
                  (setq name-x (caddr x))))
           (cond ((and (not (atom y)) (eq (car y) 'kbe-/#-comment-macro))
                  (setq code-y (cadr y))
                  (setq name-y (caddr y))))
           (unless (and name-x name-y (not (equal name-x name-y)))
             (code-equal code-x code-y))))
        ((atom x) (equal x y))
        ((atom y) nil)
        (t
         (and (code-equal (car x) (car y))
              (code-equal (cdr x) (cdr y))))))

(define-KBE-option *auto-submit*
                   "Attempt auto-correction of inconsistent code"
  t :boolean)

(defun code-is-consistent? (name &optional type)
  (cond ((not *check-code-consistency*) t)
        (t
         (let ((believed-buffered-code (belief ':buffered-code name)))
           (cond ((not believed-buffered-code) t)
                 (t
                  (let ((buffered-code (get-definition name type))
                        (good-code (belief ':code name)))
                    (or
                      ;; Buffered-code went away?
                      (not buffered-code)
                      ;; This is necessary to avoid problems caused by
                      ;; the auto-submit stuff. If the changes it requests
                      ;; are not inserted atomically, we'll find it
                      ;; complaining where it shouldn't be that the code
                      ;; is inconsistent.
                      (code-equal buffered-code believed-buffered-code)
                      ;; Technically this is the only good reason
                      ;; for assuming code equivalence
                      (code-equal buffered-code good-code)
                      ;; This forces the code to be consistent if it's
                      ;; not viewable as consistent already and if the
                      ;; force feature is enabled...
                      (and *auto-submit*
                           (eq (car buffered-code) 'defun)
                           (make-code-consistent? name
                                                  buffered-code)))))))))))

  ;; The following functionality should probably be in the Apprentice
  ;; internals and should work by actor negotiation, rather than function
  ;; call. In that case, the :BELIEF :CODE (marked with ***) would be a
  ;; reference to the KBE's internal (code-for-defun ...) information and
```

```
;; the EXECUTE-STREAM-CONTENTS of the string-stream would just be
;; a mapc of #'execute ... A context would have to be pushed
;; here so that the whole command sequence could be undone if any of
;; the generated requests failed.

(defun make-code-consistent? (name buffered-code)
  (let ((good-code (belief ':code name))) ;***
    (let ((changes (compare-code name good-code buffered-code)))
      (cond ((not changes) nil)
            (t
             (believe ':buffered-code name buffered-code);!
             (with-input-from-string (string-stream changes)
               (execute-stream-contents string-stream nil)   ;%%%
               t))))))

(defun compare-code (name buffered-code old-code)
  (*catch 'compare-code
    (if (or (atom buffered-code)
            (atom old-code)
            (neq (car buffered-code) 'defun)
            (neq (car old-code) 'defun)
            (neq (cadr buffered-code) (cadr old-code)))
        (*throw 'compare-code nil))
    (let ((arg-changes (compare-args (caddr buffered-code)
                                     (caddr old-code)))
          (result (compare-code-1 (cdddr buffered-code)
                                  (cdddr old-code)
                                  nil nil 0)))
      (if (or arg-changes result)
          (text "~{~A~^~%~}~{~&~A~^~%~}"
                (mapcar #'(lambda (datum)
                            (append-delimiter
                              (text "Append a parameter ~S ~
                                     to the program ~S"
                                    datum name)))
                        arg-changes)
                (nreverse
                  (mapcar #'(lambda (datum)
                              (let (((path . code) datum))
                                (let ((string
                                        (path->string name
                                                      (cons
                                                        (+ 3 (car path))
                                                        (cdr path)))))
                                  (cond (string
                                         (append-delimiter
                                           (text "Implement ~A as $~S"
                                                 string code)))
                                        (t
                                         (*throw 'compare-code nil))))))
                          result)))))))

(defun compare-args (x y)
  (cond ((null x)
         (if (memq '&aux y) (*throw 'compare-code nil))
         y)
        ((null y) (*throw 'compare-code nil))
        ((eq (car x) '&aux)
         (let ((thing (memq '&aux y)))
           (if (equal x thing) (ldiff y thing)
               (*throw 'compare-code nil))))
        ((eq (car x) (car y)) (compare-args (cdr x) (cdr y)))
        (t (*throw 'compare-code nil))))

(defun compare-code-1 (old new data path idx)
  (cond ((atom old)
         (cond ((equal old new) data)
               (t  (*throw 'compare-code nil))))
        ((eq (car old) 'kbe-/#-comment-macro)
```

```
        (cond ((code-equal old new) data)
              (t
               (cons (cons (cdr (reverse (cons idx path))) new) data))))
        ((atom new)  (*throw 'compare-code nil))
        (t
         (do ((i 0 (1+ i))
              (o old (cdr o))
              (n new (cdr n))
              (d data (compare-code-1 (car o) (car n)
                                      d (cons idx path) i)))
             ((or (atom o) (atom n))
              (cond ((equal o n) d)
                    (t (*throw 'compare-code nil))))))))

(define-KBE-option *automatic-reanalysis-when-needed*
                   "Automatically re-analyze inconsistent code"
  t :boolean)

(defmacro with-consistent-code (name &body forms)
  (once-only (name)
    `(cond ((or (code-is-consistent? ,name)
                (when *automatic-reanalysis-when-needed*
                  (editor-echo-output "Code for ~S being re-analyzed."
                                      ,name)
                  (com-KBE-execute-analyze)
                  t))
            ,@forms)
           (t
            (editor-barf "Definition of ~S needs to be re-analyzed."
                         ,name)))))

(defun code-is-consistent (name &optional (code (get-definition name)))
  (believe ':buffered-code name code))

(defun explain-how-code-is-superseded (name
                                       &optional
                                       (code (get-definition name)))
  (let ((old-code (belief ':buffered-code name)))
    (format t "The definition of ")
    (mouse-sensitive-princ ':function name)
    (let ((prinlevel nil)
          (prinlength nil)
          (prinendline nil)
          (base 10.)                    ; For display, force base to ten
          (*nopoint nil))               ;  and make sure user can tell.
      (format t " has changed. It was:~2%~:=~2&but is now:~2%~:=~&"
              old-code code))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1984, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; EDIT-COMMANDS: Editor commands for KBE interaction.
```

```
;;;; Support routines

(defvar *saved-requests* '() "Used by yank stuff later in this file.")

(defun read-command-for-editor (stream)
  (condition-case (cond)
      (read-command stream)
    (error (editor-barf "~A" cond))))

(defun read-commands-from-string (string)
  (with-input-from-string (stream string)
    (read-commands-from-stream-contents stream)))

(defun read-commands-from-stream-contents (stream)
  (do ((l nil (cons r l))
       (r (read-command-for-editor stream)
          (read-command-for-editor stream)))
      ((null r) (nreverse l))))


;;; This PRESCAN-FUNCTION idea will do for now, though in fact it
;;; is probably more a property of the actor involved than of
;;; the function call which invokes us.

(defun assure-buffered-definitions-consistent (requests)
  (let ((referenced-definitions '())
        (requests-to-queue '()))
    (when requests
      (multiple-value-bind (refs refs?)
          (imbedded-references (car requests))
        (cond (refs
               (setq referenced-definitions refs))
              (refs?
               (let ((local-ref (current-editor-definition-name)))
                 (if (symbolp local-ref)
                     (setq referenced-definitions (ncons local-ref))))))))
    (dolist (request (cdr requests))
      (setq referenced-definitions (union
                                     (imbedded-references request)
                                     referenced-definitions)))
    (dolist (ref referenced-definitions)
      (let ((def (get-definition ref)))
        (cond (def
               (send-delegate ':believe ':buffered-code ref def)
               (push (text "Analyze ~A .~%"
                           (unparse-command '(code '',def)))
                     requests-to-queue))
              (t
               (send-delegate ':believe ':suggested-code-home ref
                              (let ((name
                                      (current-editor-definition-name)))
                                (append
                                  (if name '(:near ,name))
                                  '(:buffer
                                    ,(buffer-name
                                       (current-editor-buffer))))))))))
    requests-to-queue))

(defun buffer-command-prescan (requests)
  (let ((focus (presentational-focus)))
    (suggest-focus nil) ;Void old focus info
    (let ((special-requests
            (assure-buffered-definitions-consistent requests)))
      (append special-requests
              (when special-requests
                (cond ((not focus) '())
                      (t
                       (list (text "Locate $~S ." focus)))))))))
```

```
;;; (SUBMIT-STREAM-CONTENTS-TO-KBE stream [await? NIL] [focus NIL])
;;;
;;;  A basic utility function. This does repeated sentence reads from the
;;;  given stream until it is empty, submitting each sentence as it goes.
;;;  Returns as soon as the requests are queued.
;;;
;;;  The return value is the number of commands queued.

(defun submit-stream-contents-to-KBE (stream
                                      &optional
                                      (await? nil)
                                      (prescan-function nil))
  (let ((requests (read-commands-from-stream-contents stream)))
    (let ((p (if prescan-function (funcall prescan-function requests))))
      (let ((text (apply #'string-append
                         (append p
                                 (list #\return)
                                 (mapcar #'unparse-command requests)))))
        (send-delegate ':submit-by-proxy text await?))
      (length requests))))

;; Things to do (KMP, 4/17/84)
;;
;; This should probably catch "a" references such as "Define a program..."
;; since we are conservative about what we do with non-existent definitions
;; anyway. Then we could use the info about what was referred to in order
;; to figure out what we were needing to update.
;;
;; Also, we could use that
;; info in order to suggest code homes for definitions being created if
;; they don't yet exist. Suggesting such homes for things even if they
;; were definite references to things supposed to already exist wouldn't
;; be such a bad thing, I suppose.
;;
;; --- Ideas above this line have been more or less done. ---
;;
;; Editing actor should handle the entire spectrum of things it gets
;; back by just processing them locally and maintaining a queue of things
;; that the interface actor can ask about. The interface should just
;; batch requests to it, prescanning them and appending any relevant
;; LOCATE and ANALYZE requests. Upon return (success or failure) it should
;; get the most accurate data about definitions from the editing actor
;; and insert them in the buffer. It should also check for and perform
;; typeout. It should also maybe offer (query) to update other definitions.
;;
;; Think more about the PROGN nature of the current TASK setup. Perhaps
;; there needs to be a verb to accomplish paragraphing. People do this
;; differently. What do replies to paragraphs look like? Lists? Multiple
;; values? ...
;;
;; Super-Hyper-Control-Meta-Resume should be fixed to query the editing
;; actor about known definitions and use that data.
;;
;; Reinitialize should locally clear the editor buffers of definitions.
;;  Editing actor should clear its database.
;;  Finally any apprentice(s) should reinitialize (if not shared?).

(defun imbedded-references (request)
  (declare (values references references?))
  (selectq (car request)
    ((DEFINE)
     (values (list (caddr (caddr request))) nil))
    (otherwise
     (let ((*any-reference?* nil))
       (declare (special *any-reference?*))
       (values (mapcan #'imbedded-references-1 (cdr request))
               *any-reference?*)))))
```

```
(defun imbedded-references-1 (phrase)
  (declare (special *any-reference?*))
  (if (atom phrase) '()
      (selectq (car phrase)
        ((it this that here top)
           (setq *any-reference?* t)
           '())
        ((root-reference)
           (setq *any-reference?* t)
           (let ((name (caddr (caddr phrase))))
             (if name (ncons name))))
        ((relative-reference)
           (setq *any-reference?* t)
           (imbedded-references-1 (nth 5 phrase)))
        (otherwise
           '()))))

;;; (SUBMIT-STREAM-CONTENTS stream)
;;;
;;;  Submits contents of stream to KBE.

(defun submit-stream-contents (stream)
  (assure-real-editor-buffer)
  (acknowledge-having-queued-stream
    (submit-stream-contents-to-KBE stream nil #'buffer-command-prescan)))

;;; (EXECUTE-STREAM-CONTENTS stream [update-buffer? nil])
;;;
;;;  A basic utility function. This does repeated sentence reads from the
;;;  given stream until it is empty, then submits the bundle of requests
;;;  as a priority request for immediate execution by the KBE. Doesn't
;;;  return until the requests are done.

(defun execute-stream-contents (stream &optional (update-buffer? t))
  (assure-real-editor-buffer)
  (let ((*pending-typeout-notification* nil)
        (*pending-code-notification* nil))
    (submit-stream-contents-to-KBE stream t #'buffer-command-prescan)
    (when update-buffer?
      (resume-KBE-edits nil)
      (editor-echo-output "Done."))))

;;; Common utility

(defun acknowledge-having-queued-stream (n)
  (cond ((zerop n)
         (editor-echo-output "No requests accepted."))
        (t
         (editor-echo-output
           "~D request~:P accepted. Type Hyper-S to see queue status.~
          ~%Type Hyper-Resume to obtain results when ~A."
           n (if (equal *pending-typeout-notification*
                        *pending-code-notification*)
                 *pending-code-notification*
                 (text "~A or ~A"
                       *pending-code-notification*
                       *pending-typeout-notification*))))))
```

```
(defun cursor-location ()
  (when (real-editor-buffer? (current-editor-buffer))
    (multiple-value-bind (name type)
        (current-editor-definition-name)
      (multiple-value-bind (path toplevel?)
          (current-editor-path)
        (let ((code (get-definition name)))
          (or (visible-cursor-location code (if (not toplevel?) path nil))
              (if (and name toplevel?)
                  (text "the ~A ~A"
                        (definition-type-name code)
                        name))))))))

(defun visible-cursor-location (code &optional (path '()))
  (funcall (or (get 'visible-cursor-location (buffer-editing-type))
               (get 'visible-cursor-location ':lisp))
           code path))


(defun (visible-cursor-location :ada) (&rest ignore)
  (multiple-value-bind (start-pt end-pt)
      (bounds-of-definition)
    (when start-pt
      (setq start-pt (editor-point)) ;Don't really care where start is
      (when (zwei:bp-< start-pt end-pt)
        (let ((found-pt (search-buffer-between start-pt end-pt "{the")))
          (ignore-errors
            (trim-cursor-location
              (cadr ;pseudo-call to kbe-comment-name
                    ;new-style format for comment at this point. -kmp,dick
                (read-from-string
                  (string-buffer-between (backward-char found-pt 4)
                                         (search-buffer-forward
                                           "}" found-pt)))))))))))

(defun (visible-cursor-location :lisp) (code path)
  (lisp-cursor-location code path))

(defun lisp-cursor-location (code &optional (path '()))
  (trim-cursor-location (*catch 'found-visible-cursor
                          (visible-cursor-location-1 code path))))

(define-KBE-option *abstract-code-references*
                   "Specificity of mini-buffer references"
  t :assoc '(("Minimal" . T) ("Maximal" . NIL)))

(defun trim-cursor-location (description)
  (when description
    (let ((high-node (cond ((or (not *abstract-code-references*)
                                (not (memq 'of description)))
                            description)
                           (t
                            (do ((d description)) (nil)
                              (setq d (or (cdr (memq 'of d))
                                          (return d))))))))
      (if (string-equal (car high-node) 'the)    (pop high-node))
      (if (string-equal (car high-node) 'input)  (pop high-node))
      (if (string-equal (car high-node) 'output) (pop high-node))
      (text "the ~{~A~^ ~}" (mapcar #'string-downcase high-node)))))

(defun visible-cursor-location-1 (code path)
  (cond ((atom code) nil)
        ((and (kbe-comment? code)
              (kbe-comment-needs-implementation? code))
         (*throw 'found-visible-cursor (caddr code)))
        (t
         (let ((rest-of-path (cdr path)))
```

```
          (dolist (c (nthcdr (or (car path) 0) code))
            (visible-cursor-location-1 c rest-of-path)
            (setq rest-of-path nil))))))

(defun string-for-command (prefix suffix)
  (let ((string (cursor-location)))
    (cond (string (text "~A~A~A" prefix string suffix))
          (t prefix))))
```

```
(defvar *in-command-mini-buffer* nil)

(defvar *going-to-submit* ':undefined ;Global binding can't be relied upon.
  "Used internally by KBE Super- and Hyper- commands.")

(defmacro when-command-setup-needed-for (command &body forms)
  '(cond ((not (real-editor-buffer? (current-editor-buffer)))
          (simple-buffer-insert ,command)
          t)
         (t
          ,@forms)))

(defun simple-buffer-insert (thing)
  (when (not (equal thing ""))
    (with-buffer-pointer (start-pt (editor-point))
      (if (sentence-terminator? (char-before (editor-point)))
          (insert-into-buffer-before-point " "))
      (do ((pt (editor-point) (backward-char pt)))
          ((not pt))
        (let ((c (char-before pt)))
          (when (not (whitespace-char? c))
            (if (not (sentence-terminator? c))
                (insert-into-buffer-before-point
                  (if (OK-to-append-delimiter-directly?
                        (string-buffer-between (beginning-of-line)
                                               (end-of-line)))
                      ". " " . ")))
            (return))))
      (insert-into-buffer-before-point (text "~A" thing))
      (fill-buffer-between start-pt (editor-point)))))

(defun going-to-submit (val)
  (when *in-command-mini-buffer*
    (cond ((and *going-to-submit* (not val))
           (set-mode-line "Execute KBE commands:"))
          ((and (not *going-to-submit*) val)
           (set-mode-line "Submit KBE commands:"))))
  (setq *going-to-submit* val))

(defun send-stream-contents (stream)
  (remark "") ;Make sure we are in a fresh remark context
  (cond (*going-to-submit*
         (submit-stream-contents stream))
        (t
         (execute-stream-contents stream))))

(defun immediate-command? (string)
  (let ((len (string-length string)))
    (and (> len 0) (memq (aref string (1- len)) '(#/. #/! #/?)))))

(defun maybe-send-command (string)
  (cond ((immediate-command? string)
         (let ((text (or (if (> (array-active-length string) 60.)
                             (text "~A ..." (substring string 0 56.)))
                         string)))
           (let ((pos (string-search-char #\return text)))
             (when pos
               (setq text (text "~A ..." (substring text 0 pos)))))
           (editor-echo-output "~&~A ^^ ~A"
                               (bucky-char-name (editor-last-command-char))
                               text))
         (with-input-from-string (str string)
           (send-stream-contents str))
         t)
        (t nil)))

(defun editor-prompt-and-execute (string arg)
```

```
  (going-to-submit nil) ;maybe side-effect an outer binding.
  (when-command-setup-needed-for string
    (let ((*going-to-submit* nil)
          (*in-command-mini-buffer* t))
      (when (or arg (not (maybe-send-command string)))
        (send-stream-contents
          (mini-buffer-stream "Execute KBE commands:" t string))))
    nil))

(defun editor-prompt-and-submit (string arg)
  (going-to-submit t) ;maybe side-effect an outer binding.
  (when-command-setup-needed-for string
    (let ((*going-to-submit* t)
          (*in-command-mini-buffer* t))
      (when (or arg (not (maybe-send-command string)))
        (send-stream-contents
          (mini-buffer-stream "Submit KBE commands:" t string))))
    nil))

(defun request-command-key (name await?)
  (if await?
      (symbolconc 'com-KBE-execute- name)
      (symbolconc 'com-KBE-submit- name)))

(defvar *simple-mini-buffer-command* nil)

(defdefmacro define-editor-request "KBE editor request" (name keys . body)
  (check-arg keys (or (fixp keys) (and (listp keys)
                                       (every keys #'fixp)))
             "a character or list of characters")
  (if (not (stringp (car body)))
      (ferror "First form in body must be a string:~% ~S" (car body)))
  (let ((name-for-execute (symbolconc 'com-KBE-execute- name))
        (name-for-submit  (symbolconc 'com-KBE-submit- name)))
    (if (fixp keys) (setq keys (list keys)))
    '(local-declare ()
       (defprop ,name-for-execute ,(car body) mini-buffer-text)
       (defprop ,name-for-submit  ,(car body) mini-buffer-text)
       (defprop ,name-for-execute nil await?)
       (defprop ,name-for-execute t   await?)
       (defprop ,name-for-submit  ,name-for-execute execute-command)
       (defprop ,name-for-execute ,name-for-submit  submit-command)
       ,(if (equal (car body) "")
            '(setq *simple-mini-buffer-command*
                   (cons ',name-for-submit ',name-for-execute)))
       (define-editor-command ,name-for-execute (arg)
            ,(mapcar #'(lambda (c) (add-bucky-bits c ':super)) keys)
            document-mini-buffer-execute-command
         (check-sectionization)
         (editor-prompt-and-execute (progn ,@body) arg))
       (define-editor-command ,name-for-submit (arg)
            ,(mapcar #'(lambda (c) (add-bucky-bits c ':hyper)) keys)
            document-mini-buffer-submit-command
         (check-sectionization)
         (editor-prompt-and-submit (progn ,@body) arg))
       ',name)))

(defun document-mini-buffer-execute-command (name char mode)
  char ;ignored
  (cond ((eq mode ':name) (editor-command-name name))
        ((neq mode ':full)
         (format t "KBE execute: ~A"
                 (get name 'mini-buffer-text)))
        (t
         (document-mini-buffer (get name 'mini-buffer-text) t))))

(defun document-mini-buffer-submit-command (name char mode)
  char ;ignored
  (cond ((eq mode ':name) (editor-command-name name))
```

```
        (t
         (let* ((execute-name (get name 'execute-command))
                (execute-key
                  (if execute-name (command-key-name execute-name))))
           (cond (execute-key
                  (format t
                          "Like ~A, but to submit rather than to execute."
                          execute-key))
                 ((neq mode ':full)
                  (format t "KBE submit: ~A"
                          (get name 'mini-buffer-text)))
                 (t
                  (document-mini-buffer (get name 'mini-buffer-text)
                                        nil)))))))

(defun simple-command-key (await?)
  (when *simple-mini-buffer-command*
    (command-key-name
      (cond (await? (cdr *simple-mini-buffer-command*))
            (t (car *simple-mini-buffer-command*))))))

(defun document-mini-buffer (text await?)
  (let ((immediate? (immediate-command? text)))
    (cond ((let ((b (current-editor-buffer)))
             (and b (not (real-editor-buffer? b))))
           (format t "Set up to ~:[submit~;execute~] text~:[:  ~A~;.~]~%"
                   await? (equal text "") text))
          ((equal text "")
           (format t "Enters a mini-buffer reading text for the KBE.~
                    ~%When mini-buffer is exited, ~:[submits~;executes~] ~
                      the KBE commands it contains.~%"
                   await?))
          ((not immediate?)
           (let ((key (simple-command-key await?)))
             (if key
                 (format t "Like ~A, but the mini-buffer initially contains:~
                            ~%  ~A~%"
                         key text)
                 (format t "Enters a mini-buffer initially containing:~
                         ~2%  ~A~
                         ~2%When mini-buffer is exited, ~
                            its text is ~:[submitted to~;executed by~] ~
                            the KBE."
                         text await?))))
          (t
           (format t "Immediately ~:[submits~;executes~] KBE command:  ~A~
                    ~%With an explicit arg, allows confirmation//editing ~
                      in mini-buffer."
                   await? text)))))
```

```
(define-editor-request Anything (#/X #\meta-X)
  "")

(define-editor-request Make-Cliche #/C
  "Define a cliche.")

(define-editor-request Make-Defun #/D
  "Define a program ")

(define-editor-request Append-Parameter #/P
  "Append a parameter ")

(define-editor-request What-Needs-To-Be-Done #/W
  "What needs to be done?")

(define-editor-request Finish #\end
  "Finish editing {this definition}"
  (text "Finish editing ~@[~A~]"
        (append-delimiter-if (this-definition-name))))

(define-editor-request Reap-Database #\rubout
  "Reap database.")

(define-editor-request Reinitialize-Database #\clear-input
  "Reinitialize database.")

(define-editor-request Clear-Display #\clear-screen
  "Clear display.")

(define-editor-request Implement #/I
  "Implement {next unfilled role} as "
  (string-for-command "Implement " " as "))

(define-editor-request Move-To (#/. #\meta-/.)
  "Move to ")

(define-editor-request Comment #/;
  "Comment {this definition}."
  (text "Comment ~@[~A~]"
        (append-delimiter-if (this-definition-name))))

(define-editor-request Describe (#/= #\control-=)
  "Describe this.")

(define-editor-request Explain #/E
  "Explain this.")

(define-editor-request Undo #/U
  "Undo.")

(define-editor-request Use-Language #/L
  "Use language ")

(define-editor-request Share #/S
  "Share ")

(define-editor-request Remove #/R
  "Remove {next unfilled role}"
  (string-for-command "Remove " ""))

(define-editor-request Test-Yourself #/T
  "Test yourself.")

(define-editor-request View #/V.
  "View {this definition} in {another language}."
  (text "View ~@[~A in ~A.~]"
        (this-definition-name)
```

```
        (if (neq *display-language* 'lisp)
            "Lisp" "PLI")))

(define-editor-request Add #/A
  "Add ")

(defun definition-type-name (code)
  (selectq (car code)
    (define-cliche "cliche")
    (def-ada-top
     (ada-definition-type-name code))
    (otherwise "function")))

(define-editor-request Analyze #/Z
  "Analyze ${text of current definition}."
  (let ((name (current-editor-definition-name)))
    (let ((code (get-definition name)))
      (ignore-errors ;clumsy, but effective for now -kmp
        (send-delegate ':believe ':buffered-code (cadr code) code))
      (if (not code) (editor-barf "Can't grok context."))
      (append-delimiter
        (text "Analyze ;the ~A ~A~% ~A"
              (definition-type-name code) name
              (unparse-command '(code '',code)))))))
```

```
;;;; Misc foreground-related commands

;;; These should be doing Execute-style requests rather than doing the
;;; stuff in foreground ...

(defun this-definition-name ()
  (multiple-value-bind (name type)
      (current-editor-definition-name)
    (if name (text "the ~A ~S"
                   (definition-type-name (get-definition))
                   name))))
```

```
;;;; Commands for manipulating KBE submit requests

;;; Hyper-Resume                          Resume KBE Edits
;;;
;;;  Completes the edits which have come as replies to any requests
;;;  which were sent to the Apprentice by a 'submit' style request
;;;  (or an aborted 'execute' style request).

(define-editor-command com-resume-KBE-edits (confirm?)
    #\Hyper-Resume
    "Complete any requests which were done in background."
  (resume-KBE-edits confirm?))

(defun resume-KBE-edits (confirm?)
  (assure-real-editor-buffer)
  (send-delegate ':finish-requests confirm?))

;;; Hyper-Status                          KBE Status
;;; Hyper-S
;;;
;;;  Gives information about the status of requests that have
;;;  been sent to the KBE.
;;;
;;;  With an argument, also gives information about the
;;;  apprentice's status.

(define-editor-command com-KBE-status (ignore)
    (#-3600 #\Hyper-Status #\Hyper-S)
    "Inquires about the status of the KBE requests."
  (editor-echo-output "~A: ~A"
                      (send-delegate ':short-name) ;say who we are
                      (task "Report status."))
  nil) ;no changes to buffer
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1984, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; EDIT-MISC: Miscellaneous KBE commands which do not
;;;            work by submiting TASK requests to background.
```

```
;;;; Option Variables

;;; m-X Edit KBE Options
;;;
;;;  Allows the user to edit a table of KBE-related control variables.
;;;
;;;  Iff the KBE is running in Debug Mode, the menu will contain
;;;  internal control variables as well.

(define-editor-command com-Edit-KBE-Options (full?) ()
    "Edit a table of KBE-related control variables.
With an argument, edit KBE internal variables as well."
  (edit-KBE-options (or full? *debug*)))
```

```
;;;; Bug Reporting

;;; m-X Bug KBE
;;;
;;;  Enters a buffer that is set up to send a bug report
;;;  about the KBE.

(define-editor-command com-Bug-KBE (ignore) ()
    "Enter a buffer which is set up to send a KBE bug report."
  (let ((oddities (odd-KBE-options)))
    (send-mail *bug-mail-address*
               (text "In ~A on ~A,~%using ~A in ~A~@[, ~A~]~
                        ~@[~* with special option~P:~2:*~
                        ~{~%* ~A~}~] ...~2%"
                     (loaded-band-string)
                     (site-string)
                     (system-version-string "KBE")
                     (system-version-string ':release)
                     (cond ((not
                              (released-system-version-active? "System"))
                            (system-version-string "System")))
                     oddities
                     (length oddities))))
  nil) ;no changes to this buffer

;;; Hyper-Control-Y                                  Yank KBE Bug Message
;;; Super-Control-Y
;;;
;;;  Inserts the text of the (nth) most recent KBE bug message.
;;;  This is mainly useful when sending KBE bug reports.

(define-editor-command com-yank-KBE-bug-message (arg)
    (#\Super-Control-Y #\Hyper-Control-Y)
    "Inserts the text of the (nth) most-recent KBE bug message"
  (insert-into-buffer-before-point (display-bug-message arg nil))
  t) ;need to redisplay buffer since we probably modified it.
```

```
;;;; Cursor Motion

;;; (MOVE-TO-NEXT-KBE-ROLE n [rev? NIL])
;;;
;;;   Moves to the (nth) next (or with arg, previous) visible role,
;;;   by scanning for KBE role markers.
;;;
;;;   This is a completely textual operation.

(defun move-to-next-KBE-role (arg &optional (rev? nil))
  (declare (special *kbe-user-role-marker*))
  (if (not arg) (setq arg 1))
  (if rev? (setq arg (- arg)))
  (let ((pt (search-buffer *kbe-user-role-marker*
                           (if (plusp arg)
                               (forward-char (editor-point))
                               (editor-point))
                           arg)))
    (cond (pt
           (if (plusp arg)
               (setq pt (backward-char pt
                          (string-length *kbe-user-role-marker*))))
           (move-point pt))
          (t
           (editor-barf "No role~A"
                        (cond ((minusp arg)
                               (format nil "~@[ ~R~] before this."
                                       (if (< arg -1) (minus arg))))
                              ((zerop arg) "to search for?")
                              (t
                               (format nil "~@[ ~R~] after this."
                                       (if (> arg 1) arg)))))))))

;;; Super-F                                  Forward Marked KBE Role
;;; Super-Control-F
;;;
;;;  Goes forward one KBE role. Searches forward for a #{...} marker.
;;;  With an argument, N, goes forward N roles.
;;;  The argument may be negative (to search backward).

(define-editor-command com-forward-marked-KBE-role (arg)
    (#\Super-F #\Super-Control-F)
  "Goes forward one KBE role. Searches forward for a KBE role marker.
With an argument, N, goes forward N roles.
The argument may be negative (to search backward)."
  (move-to-next-KBE-role arg)
  nil) ;no changes to buffer

;;; Super-B                                  Backward Marked KBE Role
;;; Super-Control-B
;;;
;;;  Goes backward one KBE role. Searches backward for a #{...} marker.
;;;  With an argument, N, goes backward N roles.
;;;  The argument may be negative (to search forward).

(define-editor-command com-backward-marked-KBE-role (arg)
   (#\Super-B #\Super-Control-B)
  "Goes back one KBE role. Searches backward for a KBE role marker.
With an argument, N, goes backward N roles.
The argument may be negative (to search forward)."
  (move-to-next-KBE-role arg t)
  nil) ;no changes to buffer
```

```
;;;; Utilities for editing Pseudo-English

;;; Super-Rubout                          Trim KBE Reference
;;;
;;;  Looks backward for the most recent definite reference and
;;;  if it's of the form "the ... of the ... of the ...",
;;;  removes the leading "the ... of" (or arg leading ones, if an
;;;  arg is given) from that phrase. eg, s-2 s-Rubout will change
;;;  "the w of the x of the y of the z" into "the y of the z".

(define-editor-command com-trim-KBE-reference (arg) (#\super-rubout)
  "Delete the first /"the ... of/" in most recent definite reference.
eg, changes /"the x of the y/" to just /"the y/"."
  (let ((mod? nil))
    (dotimes (i (or arg 1))
      (setq mod?
            (or mod? (trim-KBE-reference))))
    (cond (mod?)
          (t (editor-barf "Can't find reference.")))))

(defun trim-KBE-reference ()
  (do ((the-pt (search-buffer-backward-delimited "the" (editor-point))
               (search-buffer-backward-delimited "the" pt))
       (pt nil))
      ((not the-pt) nil)
    (let ((pt1 (backward-word the-pt)))
      (let ((pt2 (if pt1 (forward-word pt1))))
        (cond ((if pt1 (string-equal (string-buffer-between pt1 pt2) "of"))
               (setq pt pt1))
              (pt
               (return (trim-KBE-reference-internal pt)))
              (t
               (return nil)))))))

(defun trim-KBE-reference-internal (of-pt)
  (when of-pt
    (let ((the-pt (search-buffer-backward-delimited "the" of-pt)))
      (with-local-buffer-pointer (end-pt (copy-buffer-pointer
                                           (forward-char of-pt 2)
                                           ':moves))
        (delete-from-buffer-between the-pt end-pt nil)
        (delete-chars-in-set '(#\space #\tab) end-pt)
        (insert-into-buffer-before end-pt " ")
        t))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; EDIT-SIMPLE: Editor commands intended for use in the KBE simple pane.

(defvar *old-submit-requests*      '() "KBE request history list.")
(defvar *old-submit-request-ring* nil "KBE request history ring.")

;;; Abort (in KBE Simple Frame)          Kill KBE Requests
;;;
;;;  Kills the text in the buffer without submitting it.

(define-editor-command com-kill-KBE-requests (ignore) ()
    "Kills text in buffer (aborts submitting it)."
  (delete-from-buffer-between (buffer-start) (buffer-end) t))

;;; End (in KBE Simple Frame)            Submit And Kill KBE Requests
;;;
;;;
;;;  Submits the text in the buffer to the KBE, then kills that text.

(define-editor-command com-submit-and-kill-KBE-requests (ignore) ()
    "Submits text in buffer."
  (let ((buffer (current-editor-buffer)))
    (let ((start (buffer-start buffer))
          (end   (buffer-end buffer)))
      (let ((n (submit-stream-contents-to-KBE
                 (interval-stream start end))))
        (remark (text "~D request~:P accepted." n))
        (push (copy-from-buffer-between start end)
              *old-submit-requests*)
        (delete-from-buffer-between start end t))))
  t) ;buffer changed

;;; Control-Meta-Y (in KBE Simple Frame)        Yank Pop KBE Request
;;;
;;;  Inserts the text of the (nth) most-recent KBE request, killing
;;;  previously yanked text if any.

(define-editor-command com-yank-pop-KBE-request (arg) ()
    "Inserts the text of the (nth) most-recent KBE request, killing
previously yanked text if any."
  (let ((len (length *old-submit-requests*))
        (already-yanking? (eq (last-editor-command-type) 'yank)))
    (set-editor-command-type 'yank)
    (when arg
      (cond ((or (minusp arg) (> arg len))
             (editor-barf "Argument out of range."))
            ((zerop arg)
             (editor-barf "I can't yet handle an argument of zero."))))
    (when (and (or arg (not already-yanking?))
               *old-submit-requests*)
      (setq *old-submit-request-ring*
            (circular-list* *old-submit-requests*)))
    (when already-yanking?
      (delete-from-buffer-between (editor-mark) (editor-point)))
    (dotimes (n (1- (or arg 1))) ;pop specified number of nodes
      (pop *old-submit-request-ring*))
    (move-mark (editor-point))
    (move-point (yank-into-buffer-at-point
                  (pop *old-submit-request-ring*)))))
```

```
t) ;buffer might have changed
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; EDIT-DEBUG: Editor commands used mainly for KBE development/debugging
;;;             rather than intended as a model of what a real KBE
;;;             might have available.
```

```
;;;; For typing convenience

;;; Hyper-Super-Meta-Control-A          Automatic Ada Mode
;;;
;;;  Enters Ada mode and maybe puts buffer in KBE package.

(define-editor-command com-automatic-ada-mode (ignore)
    #\Hyper-Super-Meta-Control-A
    "Enters Ada Mode and updates the Attribute List"
  (validate-buffer-mode ':ada)
  (validate-buffer-package)
  t) ;this command may have modified the buffer

;;; Hyper-Super-Meta-Control-L          Automatic Lisp Mode
;;;
;;;  Enters Lisp mode and maybe puts buffer in KBE package
;;;  (if that's the default for consistent buffers).

(define-editor-command com-automatic-lisp-mode (ignore)
    #\Hyper-Super-Meta-Control-L
    "Enters Lisp Mode and updates the Attribute List"
  (validate-buffer-mode ':lisp)
  (validate-buffer-package)
  t) ;this command may have modified the buffer

;;; Hyper-Super-Meta-Control-N          Name This Expression
;;;
;;;  Name the s-expression (assign some special variable to the
;;;  result of reading it from buffer).

(define-editor-command com-name-this-expression (ignore)
    #\Hyper-Super-Meta-Control-N
    "Name the s-expression at the cursor."
  (let* ((exp (condition-case (cond)
                  (read (rest-of-buffer-stream))
                (sys:read-error
                  (editor-barf "Can't find sexp to name.~%~A" cond))))
         (prinlevel  2)
         (prinlength 3)
         (prompt (text "Name for ~S:" exp)))
    (do ((form (ignore-errors (read (mini-buffer-stream prompt nil)))
               (ignore-errors (read (mini-buffer-stream prompt nil)))))
        ((and form
              (symbolp form)
              (or (not (boundp form))
                  (fquery '() "~S is already bound. Clobber its value? "
                          form)))
         (set form exp)
         (editor-echo-output "~S now set to ~S." form exp))))
  nil) ;no changes have been made to buffer
```

```
;;;; Slide Preparation

;;; Hyper-Super-Meta-Control-Y          Yank Last Several KBE Requests
;;;
;;;  Yanks the text for KBE requests since this command was last used.
;;;  Text of recent requests are yanked onto separate lines.
;;;  If this is not done on a blank line, anything before the cursor is
;;;  repeated before every request yanked. Anything after the cursor is
;;;  repeated after every request yanked.

(define-editor-command com-yank-last-several-KBE-requests (ignore)
    #\Hyper-Super-Meta-Control-Y
    "Yanks the text for KBE requests since this command was last used.
Text of recent requests are yanked onto separate lines.
If this is not done on a blank line, anything before the cursor is
repeated before every request yanked. Anything after the cursor is
repeated after every request yanked."
  (cond (*saved-requests*
         (let ((b (beginning-of-line))
               (e (end-of-line))
               (pt (editor-point)))
           (let ((head (string-buffer-between b pt))
                 (tail (string-buffer-between pt e)))
             (delete-from-buffer-between b
                                         (or (forward-line)
                                             (buffer-end))
                                         t)
             (dolist (request (reverse *saved-requests*))
               (insert-into-buffer-before-point
                 (text "~A~A~A~&" head request tail)))
             (setq *saved-requests* nil)
             t))) ;changes to buffer
        (t
         (editor-echo-output "No recent requests."))))
```

```
;;;; Style reformatting

(define-editor-command com-change-this-expression (ignore)
    #\Hyper-Super-Meta-Control-C
    "Change the style of the enclosing form."
  (do ((pt (editor-point) (backward-up-list pt))
       (error-message nil))
      ((not pt)
       (cond ((not error-message)
              (editor-echo-output "No obvious form to change."))
             (t
              (editor-barf "Syntax error in surrounding expression:~%~A"
                           error-message)))
       nil) ;no changes
    (let ((exp (condition-case (cond)
                    (read (rest-of-buffer-stream pt))
                  (sys:read-error
                    (setq error-message (or error-message
                                            (let ((prinlevel 2)
                                                  (prinlength 2))
                                              (text "~A" cond))))
                    nil))))
      (when (and (not (atom exp)) (atom (car exp)))
        (let ((fn (get (car exp) 'style-macro)))
          (when fn
            (let ((result (funcall fn exp)))
              (cond (result
                     (insert-into-buffer-before
                       pt
                       (let ((prinmargin (- (send terminal-io
                                                  ':size-in-characters)
                                            (cadr pt)))) ;cheat
                         (text "~:=" result)))
                     (delete-from-buffer-between pt (forward-sexp pt))
                     (move-point (backward-sexp pt))
                     (zwei:com-indent-sexp)
                     (move-point (forward-down-list (editor-point)))
                     (let ((prinlevel 1) (prinlength 1))
                       (editor-echo-output "~S rewritten as ~S"
                                           exp result))
                     (return t)) ;changed buffer
                    (t
                     (editor-echo-output "No way to change this ~S form."
                                         (car exp))
                     (return nil)))))))))

(defun (cond style-macro) (form)
  (let ((clauses (cdr form)))
    (cond ((null clauses) nil) ;no idea
          ((null (cdr clauses))
           (let ((clause (car clauses)))
             (cond ((null (cdr clause)) (car clause))
                   ((null (cddr clause)) '(if ,@clause))
                   ((and (not (atom (car clause)))
                         (eq (car clause) 'not))
                    '(unless ,(cadr (car clause)) ,@(cdr clause)))
                   (t '(when ,@clause)))))
          (t nil))))

(defun (when style-macro) (form)
  '(cond ,(cdr form)))

(defun (unless style-macro) (form)
  '(cond ((not ,(cadr form)) ,@(cddr form))))

(defun (if style-macro) (form)
  '(cond (,(cadr form) ,(caddr form))
         ,@(if (cdddr form)
```

```
'((t ,@(cdddr form))))))
```

```
;;;; Debugging

;;; Hyper-Super-Meta-Control-D          Toggle KBE Debug Mode
;;;
;;;  Toggles KBE debug mode.

(define-editor-command com-toggle-kbe-debug-mode (off?)
    #\Hyper-Super-Meta-Control-D
    "Enable debug mode, or with an argument, disable it."
  (debug (not off?))
  nil) ;no changes have been made to buffer

;;; Hyper-Super-Meta-Control-Abort      Reset Apprentice
;;;
;;;  Tries to reset the associated apprentice's process.

(define-editor-command com-reset-apprentice (ignore)
    #\Hyper-Super-Meta-Control-Abort
    "Resets the KBE process. Works in Debug Mode only."
  (editor-echo-output (reset-apprentice))
  nil) ;no changes have been made to buffer

;;; Hyper-Super-Meta-Control-System     Select Apprentice
;;;
;;;  Tries to select the associated apprentice's process.

(define-editor-command com-select-apprentice (ignore)
    #\Hyper-Super-Meta-Control-System
    "Selects the KBE process. Works in Debug Mode only."
  (editor-echo-output (select-apprentice))
  nil) ;no changes have been made to buffer
```

```
;;;; Error Recovery

;;; Hyper-Super-Meta-Control-Resume     Make Buffer Consistent For KBE
;;;
;;;  Tries to recover from some kinds of KBE interface lossage.
;;;  Makes definitions in buffer consistent with definitions known
;;;  to the interface. With an arg, queries for each definition.

(define-editor-command com-Make-buffer-consistent-for-KBE (ignore)
    #\Hyper-Super-Meta-Control-Resume
    "Tries to recover from some kinds of KBE interface lossage.
Makes definitions in buffer consistent with definitions known
to the interface. With an arg, queries for each definition."
  (assure-real-editor-buffer)
  (let ((definitions-to-update nil))
    (declare (special definitions-to-update))
    (send-delegate ':map-over-beliefs-of-type ':code
        #'(lambda (fn code)
            (cond ((not (code-equal code (get-definition fn)))
                   (push (list fn code) definitions-to-update)))))
    (dolist (entry definitions-to-update)
      (let (((fn code) entry))
        (if (fquery '() "Update definition of ~S? " fn)
            (replace-or-create-definition fn code)))))
  t) ;possible mods to buffer
```

```
;;;; Cliche Definitions

;;; Hyper-Super-Meta-Control-;          Compile Cliche Comments
;;;
;;;  Pick up the comments from the cliche definition at the cursor
;;;  and put them in the right place without the overhead of actually
;;;  compiling this definition.

(define-editor-command com-compile-cliche-comments (ignore)
    #\Hyper-Super-Meta-Control-/;
    "Compile cliche comment for current form."
  (cond ((not *debug*)
         (editor-barf "Only works when KBE is in debug mode."))
        (t
         (let ((cliche-name (current-editor-definition-name)))
           (let* ((pgm (get-definition cliche-name))
                  (comments (if (and pgm (eq (car pgm) 'define-cliche))
                                (caddr pgm))))
             (cond (comments
                    (putprop cliche-name
                             (cons nil comments)
                             'cliche-comments)
                    (editor-echo-output "Cliche comments for ~S updated."
                                        cliche-name))
                   (t
                    (editor-barf
                      "Only works for cliche definition forms.")))))))
  nil) ;no changes have been made to buffer
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; KBE-MODE: Editor support for KBE Mode and KBE-style
;;;           command completion.

;;; Complete (when completing)          Complete KBE Word

(define-editor-command com-complete-KBE-word (ignore)
    (#\Super-Altmode #\Hyper-Altmode)
    "Complete current word"
  (complete-current-token *completion-table*)
  t) ;stuff in buffer may have been expanded

;;;                                     Auto Complete KBE Word
;;;
;;; Space, Period, Comma, and Exclam (when completing)
;;;
;;;  Complete the current word and self-insert the character
;;;  used to invoke this command.

(define-editor-command com-auto-complete-KBE-word (ignore)
    (#\Super-Space #\Hyper-Space)
    "Complete and self-insert"
  (multiple-value-bind (ignore start-bp)
      (bound-token)
    (editor-window-must-redisplay-rest-of-line
      (or start-bp (copy-buffer-pointer (editor-point))))
    (when start-bp
      (insert-into-buffer-after-point #\space) ;make room
      (let ((pt (editor-point)))
        (unwind-protect
          (cond ((when-error
                   (and (not (backward-up-list pt)) ;toplevel?
                        (not (backward-up-list start-bp))
                        (not (breakchar-p (char-after (backward-sexp pt))))
                        (not (breakchar-p (char-before pt)))
                        (not (uppercase-p (char-before pt))))
                   nil)
                 (complete-current-token *completion-table* nil)
                 (multiple-value-bind (completion? msg n)
                     (get-current-token-completion *completion-table*)
                   (if (and (not completion?)
                            (> n 1))
                       (editor-barf msg)))))
          (delete-from-buffer-between pt (forward-char pt)))))
    (insert-into-buffer-before-point (editor-last-command-char))
    nil)) ;buffer redisplay attended to elsewhere above

;;; Help (when completing)              Document KBE Completions

(define-editor-command com-document-KBE-completions (ignore)
    (#\super-Help #\hyper-Help)
    "Document what the completion would do at this buffer point."
  (document-completions *completion-table*)
  nil) ;no buffer mods
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; FOCUS: Stuff related to the marking of editor buffers given a
;;;        description, the inverse (giving the name/function
;;;        of a specified region of the buffer), changing the KBE's
;;;        focus, etc.
```

```
;;;; Utilities

(defun code->path (name subpart)
  (declare (special subpart))
  (*catch 'path-seen
    (cond ((eq (car subpart) '*code-is-cdr*)
           (barf "Code format not supported: ~S" subpart))
          ((eq (car subpart) '*code-is-car*)
           (setq subpart (cdr subpart))
           (code->path-1-for-code-is-car (code-for-defun name) nil))
          (t
           (code->path-1 (code-for-defun name) nil)))
    (ferror "(CODE->PATH ~S ~S) failed." name subpart)))

(defun code->path-1-for-code-is-car (code path)
  (declare (special subpart))
  (cond ((not (atom code))
         (do ((l code (cdr l))
              (i 0 (1+ i)))
             ((atom l))
           (if (eq l subpart)
               (*throw 'path-seen (nreverse (cons i path))))
           (code->path-1-for-code-is-car (car l) (cons i path))))))

(defun code->path-1 (code path)
  (declare (special subpart))
  (if (eq code subpart) (*throw 'path-seen (nreverse path)))
  (cond ((not (atom code))
         (do ((l code (cdr l))
              (i 0 (1+ i)))
             ((atom l))
           (code->path-1 (car l) (cons i path))))))
```

```
;;;; Inverse

(defun path->string (name path)
  (declare (values string code))
  (let ((string nil) (code nil))
    (ignore-errors
      (let ((parsed (code->parsed (setq code (path->code name path)))))
        (if parsed (setq string (unparse-command parsed)))))
    (values string (copytree code))))

(defun path->code (name &optional (path '()))
  (let ((code (code-for-defun name)))
    (path->code-1 code path nil)))

(defun path->code-1 (code path backpointer)
  (cond ((null path)
         (cond ((atom code)
                (cond ((null backpointer) nil)
                      (t (cons '*code-is-car* backpointer))))
               (t code)))
        ((atom code)
         ;; this happens when the user's code and the KBE's idea of what
         ;; is in the code differ. probably it means that the definition
         ;; in the buffer has not been analyzed recently enough.

         ; hide icky message for now...
         ;(barf "PATH->CODE-1 saw atomic code: ~S" code)
         ; and replace with something friendlier...

         (editor-barf
           "Sorry, the smarts needed to implement that aren't there yet.~
          ~%Maybe if you re-analyze this function and try again..."))
        (t
         (let ((backpointer (nthcdr (car path) code)))
           (path->code-1 (car backpointer) (cdr path) backpointer)))))
```

```
;;; Note:
;;;  In order for the :TEMPORARY marks to to work at all, we rely
;;;  on a patch to ZWEI which allows WINDOW-MARK-P to take on other
;;;  than boolean values. The patch is not part of the KBE system,
;;;  so this may or may not always win. In the case that the patch
;;;  is not available, the effect of using :TEMPORARY here should be
;;;  the same as using T (ie, all marks should be permanent). --KMP

(defun mark-sexp-from-path (name path &optional (region-style ':temporary)
                                                (at-end? nil))
  (let ((pt (find-single-definition name)))
    (when pt
      (make-buffer-current (containing-buffer pt))
      ;; Ignore possible problems with #{...} syntax.
      (ignore-errors
        (dolist (n path)
          (setq pt (or (forward-down-list pt) (editor-barf)))
          (dotimes (i (1+ n))
            (setq pt (or (forward-sexp pt) (editor-barf))))
          (setq pt (or (backward-sexp pt) (editor-barf)))))
      (if at-end?
          (establish-region (forward-sexp pt) pt region-style)
          (establish-region pt (forward-sexp pt) region-style)))))

(defun compute-offset (bp x y)
  (dotimes (i y)
    (setq bp (or (forward-line bp) bp)))
  (dotimes (i (min (string-length
                     (string-buffer-between bp (end-of-line bp))) x))
    (setq bp (or (forward-char bp) bp)))
  bp)

(defun mark-sexp-from-offsets (&optional
                               (name (current-editor-definition-name))
                               (x1-offset 0)
                               (y1-offset 0)
                               (region-style nil)
                               (x2-offset)
                               (y2-offset))
  (let ((start-pt (bounds-of-definition name)))
    (when start-pt
      (make-buffer-current (containing-buffer start-pt))
      (let* ((offset1 (compute-offset start-pt x1-offset y1-offset))
             (offset2 (if (not x2-offset) offset1
                          (compute-offset start-pt x2-offset y2-offset))))
        (establish-region offset1 offset2 region-style)))))
```

```
;;;; Set up keys...

(defun comment-definition (name text &optional (err-if-no-comment? nil))
  (cond ((and err-if-no-comment? (not text))
         (editor-barf "Unable to provide a comment for ~S." name)
         nil) ;no changes to buffer
        (t
         (insert-comment-definition name text
                                    ;supressing this DICK
                                    ;(syntax-for-definition name)
                                    )
         (editor-window-redisplay)
         t)))

(defun syntax-for-definition (name &optional (code (belief ':code name)))
  (selectq (car code)
    (defun (text "(~A~{ ~A~})"
                 name
                 (mapcar #'string-downcase
                         (parse-argument-list (caddr code)))))
    (define-cliche
     (text "Cliche ~A" name))
    (def-ada-top
     (syntax-for-definition name (nth 4 code)))
    (def-ada-fn
     (text "~:[Procedure~;Function~] ~A~@[(~{~A~^, ~})~]"
           (caddr (cadr code))
           (car  (cadr code))
           (mapcar #'(lambda (x) (string-downcase (car x)))
                   (cadr (cadr code)))))
    (otherwise nil)))  ;no banner line for comment in this case

(defun remove-misplaced-whitespace (string)
  (with-output-to-string (str)
    (do ((i 0 (1+ i))
         (space-flag nil)
         (len (string-length string)))
        ((= i len))
      (let ((c (aref string i)))
        (cond ((or (eq c #\space) (eq c #\tab) (eq c #\return))
               (setq space-flag t))
              (t
               (format str "~@[~* ~]~C"
                       (and space-flag
                            (not (memq c '(#/, #/. #/! #/; #/?))))
                       c)
               (setq space-flag nil)))))))

(defun format-explanation-for-comment (explanation &optional syntax)
  (string-for-comment (text "~@[~A~2%~]~A"
                            syntax
                            (format-explanation explanation))))

(defun format-explanation (explanation)
  (cond ((stringp explanation) explanation)
        (t
         (let ((text (get (locf explanation) ':text)))
           (cond ((get (locf explanation) ':auto-fill)
                  (auto-fill-string (remove-misplaced-whitespace text)))
                 (t text))))))

;;; (LONGEST-LINE-LENGTH string)
;;;
;;;  Returns the length of the longest line in string.
;;;  Lines are defined to end with either a Return or the end of
;;;  the string. For example,
;;;
;;;      (longest-line-length (format nil "123~%1234")) => 4
```

```
(defun longest-line-length (text-string)
  (do ((i 0 (1+ i))
       (j 0)
       (longest-so-far 0)
       (text-length (string-length text-string)))
      ((= i text-length) (max j longest-so-far))
    (cond ((= (aref text-string i) #\return)
           (setq longest-so-far (max j longest-so-far))
           (setq j 0))
          (t
           (incf j)))))

;;; (STRING-FOR-COMMENT string [mode] [leader] [trailer])
;;;
;;;  Takes a multi-line string and puts comment markers in
;;;  appropriate places, returning something in comment syntax.
;;;
;;;  Mode should be one of :TOPLEVEL, :IMBEDDED, or :MARGIN to
;;;  indicate where the comment will appear.
;;;
;;;  The LEADER and TRAILER args are mostly for debugging. In
;;;  practice, they will usually want to just default to
;;;  the globally selected ZWEI options.

(defun string-for-comment (string &optional
                           (mode  ':toplevel)
                           (begin (editor-comment-begin))
                           (end   (editor-comment-end)))
  (let ((trailer-tab nil)
        (leader  (string-right-trim '(#\space #\tab) begin))
        (trailer (if end (string-left-trim  '(#\space #\tab) end))))
    ;; A null trailer is the same as no trailer.
    (when (equal trailer "")
      (setq trailer nil))
    ;; If a single character (eg, ";") and not matchfix, ...
    (when (and (not trailer) (= (string-length leader) 1))
      (setq leader
            (caseq mode
              ((:toplevel) (string-append leader leader leader))
              ((:imbedded) (string-append leader leader))
              ((:margin)   leader)
              (t (barf "Bad MODE argument to STRING-FOR-COMMENT -- ~S~
                        ~%Should be one of :TOPLEVEL, :IMBEDDED, :MARGIN."
                       mode)))))
    ;; Add spaces if necessary
    (setq leader  (string-append leader " "))
    ;; Compute position of right margin
    (setq trailer-tab (if trailer (+ 1
                                     (string-length leader)
                                     (longest-line-length string))))
    ;; Actually do it ...
    (with-output-to-string (str)
      (format str "~A" leader)
      (do ((i 0 (1+ i))
           (len (string-length string)))
          ((= i len))
        (let ((c (aref string i)))
          (cond ((= c #\return)
                 (format str "~@[~VT~]~@[~A~]~%~A"
                         trailer-tab trailer leader))
                (t
                 (format str "~C" c)))))
      (format str "~@[~VT~]~@[~A~]" trailer-tab trailer))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; TEST: Self-testing facilities for the KBE.

;;; Note: The name "sequence" is overloaded in this file. Sometimes
;;;   it means a sequence of commands (which should rightly be called
;;;   a "scenario") and sometimes it means a sequence of scenarios.
;;;   It should usually be obvious by context which was meant. Sometime
;;;   I'll regularize the naming, I hope. --kmp

;;;; Declarations

(defvar *demo-bindings*
        '((PRINLEVEL   NIL)
          (PRINLENGTH  NIL)
          (PRINENDLINE NIL)
          (IBASE       10.)
          (BASE        10.)
          (*NOPOINT    T)
;;; this is a nice idea but it doesn't work well enough, because
;;; even if things get read in right, the get written out again and
;;; then get read in again in the wrong package later.  We
;;; must do a real pkg-goto. --- Dick
          (PACKAGE     ',*KBE-PACKAGE*))
  "Bindings that should be in effect during KBE demo testing.")

(defmacro defun-in-demo-environment (name bvl &body body)
  '(defun ,name ,bvl
     (progw *demo-bindings* ,@body)))

(fs:define-canonical-type :KBE-TEST-SEQUENCE "KBE-TEST-SEQUENCE"
  ((:tops-20) "KBE-TEST-SEQUENCE")
  ((:tenex :unix :vms) "KSQ"))
```

```
;;;; File System Utilities

;;; (MATCH-PATHNAMES wildcard-pathname)
;;;
;;;  Returns a list of all the pathnames that match a given pathname.

(defun match-pathnames (wildcard-pathname)
  (let ((result (fs:directory-list wildcard-pathname
                                   ':fast ':noerror)))
    (cond ((typep result 'sys:error) nil)
          (t (mapcar #'car (cdr result))))))
```

```
;;;; Test and Demo Files

;;; The DEMO directory is allocated for holding onto standard test
;;; sequences and their correct results.

(define-internal-option *demo-file-pathname-defaults*
                        "Self-test input file pathname default"
  "KBE: DEMO; * ^W >" :sexp)

(defun demo-file-pathname (type &optional (name ':wild))
  (send (fs:parse-pathname *demo-file-pathname-defaults*) ':new-pathname
        ':name name ':type type))

;;; The TEST directory is allocated for holding onto the results
;;; of KBE automatic testing.

(define-internal-option *test-file-pathname-defaults*
                        "Self-test output file pathname default"
  "KBE: TEST; * LISP >" :sexp)

(defun test-file-pathname ()
  (send (fs:parse-pathname *test-file-pathname-defaults*) ':new-pathname
        ':name (kbe-version-string-for-pathname)))

(defun all-demo-names ()
  (Rlist (send (Elist (match-pathnames (demo-file-pathname ':KBE)))
               ':name)))

(defun all-demo-sequence-names ()
  (Rlist
    (let ((pathname (Elist (match-pathnames
                             (demo-file-pathname ':KBE-TEST-SEQUENCE)))))
      (let ((sequence (send pathname ':name)))
        (cons (text "~A sequence"
                    (string-capitalize (string-downcase sequence)))
              (with-open-file (stream pathname ':in)
                (read-forms-from-stream stream)))))))

(defun-in-demo-environment demo-sequence (name)
  (error-restart (sys:error "Retry obtaining info for ~A demo sequence."
                            name)
    (values (with-open-file (instream (demo-file-pathname ':KBE name))
              (read-forms-from-stream instream #'maybe-parse-command))
            (with-open-file (instream (demo-file-pathname ':LISP name))
              (read-forms-from-stream instream)))))

(defun maybe-parse-command (stream eof-val)
  (let ((command (read-command stream)))
    (cond ((not command) eof-val)
          (t
           (unparse-command command)))))

(defun read-forms-from-stream (stream &optional (reader #'read))
  (do ((form (funcall reader stream nil) (funcall reader stream nil))
       (l nil (cons form l)))
      ((null form) (nreverse l))))
```

```
;;;;

(defun format-test-file-header (outstream)
  (let ((oddities (odd-KBE-options)))
    (format outstream
            ";;; -*- Mode: LISP; Package: KBE; Base: 10; -*-~
           ~%;;;~
           ~%;;; Knowledge Based Editor.~40TAutomatic self-test results.~
           ~%;;;~
           ~%;;; For ~A, ~Q.~
           ~%;;;~
           ~%;;; In ~A on ~A~
           ~%;;; using ~A in ~A~@[, ~A~].~:[~*~;~
           ~%;;;~
           ~%;;; Special option~P:~2:*~{~
           ~%;;;  * ~A~}~*~]~%"
            user-id
            #'time:print-current-date
            (loaded-band-string)
            (site-string)
            (system-version-string "KBE")
            (system-version-string ':RELEASE)
            (cond ((not (released-system-version-active? "System"))
                   (system-version-string "System")))
            oddities
            (length oddities))))


;;; Returns
;;;  1: a string describing the results
;;;  2: T if all tests were passed, NIL otherwise.

(defun test-KBE (&optional sequence)
  (pkg-goto 'kbe) ;so that tests will work --- Dick
  (declare (values string success?))
  (let ((names (cond ((null sequence) (all-demo-names))
                     ((stringp sequence) (ncons (string sequence)))
                     ((symbolp sequence)
                      (or
                       (mapcar #'string
                               (cdr
                                (ass #'(lambda (x y)
                                         (%string-equal x 0 y 0
                                                        (string-length x)))
                                     (string sequence)
                                     (all-demo-sequence-names))))
                       (ferror "No such sequence: ~S" sequence)))
                     (t (mapcar #'string sequence))))
        ;(*db-check-fact-ports* t) ;Nothing works with this set (yet).
        )
    (let ((results (mapcar #'(lambda (name)
                               (cons name
                                     (multiple-value-list
                                       (run-KBE-test name))))
                           names))
          (string nil)
          (success? nil))
      (with-open-file (outstream (test-file-pathname) ':direction ':out)
        (unwind-protect (format-KBE-test outstream results)
          (close outstream))
        (setq string
              (format nil "KBE ~:[failed~;passed~] the~@[ ~A~] test!~
                           ~@[ Details in ~A~]"
                      (setq success? (every results #'cadr))
                      (if (null (cdr names)) (car names))
                      (send outstream ':send-if-handles ':pathname))))
      (values string success?))))
```

```
(defun format-KBE-test (outstream results)
  (format-test-file-header outstream)
  (format-test-summary outstream results)
  (dolist (result results)
    (lexpr-funcall #'format-test-result outstream result)))

(defun test-KBE-1 (name outstream)
  (lexpr-funcall #'format-test-result
                 outstream name
                 (multiple-value-list (run-KBE-test name))))

(defun format-test-summary (outstream results)
  (format outstream "~%;;;; Summary~%")
  (cond ((and (cdr results) (every results #'cadr))
         (format outstream ";;;  All test sequences were successful.~%"))
        (t
         (dolist (result results)
           (format outstream
                   (cond ((cadr result)
                          ";;;  The ~A sequence was successful.~%")
                         (t
                          ";;;  The ~A sequence failed:~
                          ~%;;;   ~A~%"))
                   (car result) (caddr result))))))

(defun-in-demo-environment format-test-result
                           (outstream name win? errmsg input-list n alist)
  (let ((result nil))
    (format outstream "~/|~%;;;; The ~A Sequence~%~{~
                       ~%;;; >~A~}~%"
            name input-list)
    (cond (win?
           (format outstream "~%;;; No coding errors in this sequence.~%"))
          (t
           (format outstream "~%;;; Test failed~@[ after ~D lines~]!~
                              ~%;;; ~A~%"
                   n errmsg)))
    (dolist (entry alist)
      (let (((fn code good-code explanation) entry))
        (format outstream "~%;;; ~S~%~A~%"
                (cons fn (parse-argument-list (caddr code)))
                (cond (explanation (format-explanation-for-comment
                                    explanation))
                      (t ";;;  Note: No explanation generated by KBE.")))
        (cond ((equal code good-code)
               (format outstream "~%~:=~%" code))
              ((null code)
               (push fn result)
               (format outstream "~% ;; No code produced for ~S.~%" fn))
              (t
               (push fn result)
               (format outstream
                       "~% ;; Faulty code produced for ~S...~%~
                        ~%~:=~%~
                        ~% ;; Correct code would have been...~%~
                        ~%~:=~%"
                       fn code good-code)))))
    result))

;;; Returns
;;;  1: T if successful, NIL if not.
;;;  2: If not successful, an explanation of why not.
;;;  3: Input list
;;;  4: If not all input lines executed, number that were. Otherwise NIL.
;;;  5: An alist of the form ((fn code correct-code explanation) ...)

(defun run-KBE-test (name)
  (declare (values win errmsg input-list n results-alist))
  (multiple-value-bind (input-list good-code-list)
```

```
      (demo-sequence name)
    (let ((fail nil) (codings nil) (n 0))
      (catch-error-restart ((sys:error sys:abort) "Abort test ~S" name)
        (dolist (request input-list)
          (when-abnormal-exit (condition-case-if (not *debug*) (cond)
                                (with-interface-disabled
                                  (eval-command (parse-command request)))
                              (sys:abort ;Typed c-Abort while running
                                (setq fail (format nil
                                                   "Aborted during ~S"
                                                   request))
                                (return))
                              (sys:error
                                cond ;ignored for now...
                                (setq fail (format nil "Error during ~S"
                                                   request))
                                (return)))
            ;; This can happen, for example, when the user
            ;; types Abort from the debugger to find our SYS:ERROR trap
            ;; just outside this do-loop.
            (if (not fail)
                (setq fail
                      (format nil "Aborted during ~S" request))))
          (incf n)))
      (if (not fail) (setq n nil))
      (dolist (good-code good-code-list)
        (let ((fn   (cadr  good-code)))
          (let ((code        (copytree (code-for-defun fn)))
                (explanation (explanation fn)))
            (if (or (not code) (not (equal code good-code)))
                (setq fail (or fail "Bad code generated.")))
            (push (list fn code good-code explanation) codings))))
      (values (not fail) fail input-list n (nreverse codings)))))

(defun choose-KBE-test ()
  (let ((individuals (all-demo-names))
        (sequences   (all-demo-sequence-names)))
    (let ((the-group '("All of the above" :value ,individuals)))
      (tv:menu-choose '(,@individuals
                        ,the-group
                        ,@(mapcar #'(lambda (x)
                                      (list (car x) ':value (cdr x)))
                                  sequences))
                      "Choose a demo sequence:"))))

(define-db-action test (yourself)
                  "Requests that the apprentice test itself."
  (multiple-value-bind (text success?)
      (with-interface-disabled (test-kbe))
    '(:test-results :success ,success? :text ,text)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; DOC: Automatic documentation for the KBE

(define-internal-option *documentation-pathname-default*
                        "Self-documentation pathname default"
  "KBE: DOC; ^W TEXT >" :sexp)

(defun document-KBE-to-file (&optional file)
  (let ((file
          (or file
              (send (fs:parse-pathname *documentation-pathname-default*)
                    ':new-pathname
                    ':name (kbe-version-string-for-pathname)))))
    (with-open-file (stream file ':out)
      (format stream "-*- Mode:Text; Fonts:CPTFONT; -*-~%")
      (document-KBE-to-stream stream t)
      file)))

(defun document-KBE-to-stream (&optional stream print-dictionary?)
  (let ((output-stream (or stream standard-output)))
    (format output-stream "~%Documentation for ~A, produced ~\datime\~%"
            (system-version-string "KBE"))
    (document-KBE output-stream print-dictionary?)))

(defun document-KBE (&optional stream print-dictionary?)
  (let ((standard-output (or stream standard-output)))
    (format t
            "~%The Knowledge-Based-Editor (KBE) is an extension to the~
             ~%normal editing environment developed by the Programmer's~
             ~%Apprentice group at MIT.~
             ~%~
             ~%Interaction with the KBE may occur in different ways:~
             ~%~
             ~%  Some commands are directly available as normal editor~
             ~%  commands which can be invoked by a single keystroke.~
             ~%~
             ~%  Some commands are extended text commands, which can be~
             ~%  done either immediately (an execute request), or in~
             ~%  background (a submit request).~
             ~%~
             ~%Commands related to immediate action tend to be named~
             ~%Super-something. Commands which relate to the background~
             ~%KBE's background server process tend to be called~
             ~%Hyper-something.~
             ~%~
             ~%In particular,...
             ~%  ~A requests immediate execution of a text request~
             ~%  ~A submits a request without waiting for it to be~
             ~%     executed. The status of such requests can be checked~
             ~%     with ~A, and their results can later be~
             ~%     picked up with ~A.~
             ~%~
             ~%Here is a full list of KBE-related Editor commands...~%"
            (simple-command-key t)
            (simple-command-key nil)
            (command-key-name 'com-KBE-status)
            (command-key-name 'com-resume-KBE-edits))
    (document-editor-commands)
    (when print-dictionary?
```

```
      (format t "~%~/|~2%")
      (print-dictionary ':group)
      (format t "~%~/|~2%"))
    (format t "~2&Send bugs, queries, etc. to ~A.~%"
            *bug-mail-address*)))

(define-editor-command com-Document-KBE (arg)
    (#\Super-Control-Help #\Hyper-Control-Help)
    "Gives help information about KBE commands.
With an arg, shows KBE dictionary information instead."
  (cond ((not arg) (document-KBE nil))
        (t (print-dictionary ':group)))
  nil) ;no changes to buffer
```

```
;;; -*- Mode:Lisp; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;;; Declarations

;;; Based on a theory of parsing presented in:
;;;
;;;     Pratt, Vaughan R., ''Top Down Operator Precedence,''
;;;     ACM Symposium on Principles of Programming Languages
;;;     Boston, MA; October, 1973.
;;;
;;; Glossary of useful terms...
;;;
;;;   A NUD operator has NUll left Denotation. It has nothing to its
;;;    left. Nofix, Prefix and Matchfix operators are examples of
;;;    NUD operators.
;;;
;;;   An LED operator has LEft Denotation. They have something to their
;;;    left. Postfix, Infix and n-ary operators are LED operators.
;;;
;;;   The terms LBP and RBP are left and right binding power, respectively.
;;;    They refer to the stickiness toward the right or left of an
;;;    operator. Note that some parsing routines may be liberal with
;;;    the meaning of directionality. eg, for n-ary operators, there
;;;    must be equivalent stickiness in both directions, so the LBP
;;;    is used for stickiness in both directions and the RBP is not used.

;;; Special Variables

(define-KBE-option *input-language*
                   "Input language for use by KBE"
  'LISP :assoc '(("Lisp"  . LISP)
                 ("PL//I" . PLI)
                 ("Ada"   . ADA)))

(defvar *language* nil
  "Dynamically bound to the active parser language structure")

(defvar *casification-hook* #'char-upcase ;this doesn't get used yet
  "Hook is a function called on input chars to adjust case if needed")

(defvar *character-set-size* 256.
  "The size of the set of characters to be seen by the parser")
```

```
;;;; Structure Descriptions

;;; Language Structure

(defmacro def-language (language &body morphemes)
  '(let ((language ',language)
         (language-structure (make-language language ',language)))
     (setf (language-structure language) language-structure)
     ,@(mapcar #'(lambda (x)
                   '(setf (language-morpheme ',(car x) language-structure)
                          (make-morpheme
                            name ',(car x)
                            language ',language
                            ,@(cond ((get x 'nud-lisp-alias) '())
                                    (t '(nud-lisp-alias ',(car x))))
                            ,@(cond ((get x 'led-lisp-alias) '())
                                    (t '(led-lisp-alias ',(car x))))
                            ,@(cdr x))))
               morphemes)))

(defmacro def-operator (operator language &body data)
  '(setf (language-morpheme ',operator (language-structure ',language))
         (make-morpheme
           name ',operator
           language ',language
           ,@(cond ((get (locf data) 'nud-lisp-alias) '())
                   (t '(nud-lisp-alias ',operator)))
           ,@(cond ((get (locf data) 'led-lisp-alias) '())
                   (t '(led-lisp-alias ',operator)))
           ,@data)))

(defstruct (language :named-array :conc-name)
  (language "??")
  (default-lbp 200.)
  (default-rbp 200.)
  (whitespace-characters '(#\tab #\space #\return #\line #\form))
  (syntax-table (make-syntax-table)))

(defmacro active-language ()
  '(language-language *language*))

(defmacro active-default-lbp ()
  '(language-default-lbp *language*))

(defmacro active-default-rbp ()
  '(language-default-rbp *language*))

(defmacro active-whitespace-chars ()
  '(language-whitespace-chars *language*))

(defmacro active-syntax-table ()
  '(language-syntax-table *language*))

(defmacro language-structure (language)
  '(get ,language 'language-structure))

(defun (language :named-structure-invoke) (op self &optional arg1
                                                   &rest ignore)
  (selectq op
    ((:which-operations) '(:print-self))
    ((:print-self) (format arg1 "#<Language ~A ~O>"
                           (language-language self)
                           (%pointer self)))))
```

```
;;; Morpheme Structure

(defmacro semantics (sym) '(language-morpheme ,sym *language*))

(defmacro language-morpheme (sym language)
  '(get ,sym ,language))

(defmacro set-up-morpheme-structure (&body parts)
  '(progn 'compile (defstruct (morpheme :conc-name :named-array)
                     name
                     ,@(mapcar #'(lambda (x) (cond ((atom x) x)
                                                   (t (car x))))
                               parts))
          ,@(mapcar #'(lambda (x)
                        (let ((symbol (cond ((symbolp x) x)
                                            (t (car x))))
                              (default (cond ((symbolp x) nil)
                                             (t (cadr x)))))
                          (let ((y (symbolconc "MORPHEME-" symbol)))
                            '(defmacro ,symbol (data)
                               (let ((morph (gensym)))
                                 '(let ((,morph (semantics ,data)))
                                    (cond (,morph
                                           (or (,',y ,morph)
                                               ,',default))
                                          (t ,',default)))))))))
                    parts)))

(set-up-morpheme-structure
  language                              ; backpointer
  (lbp (active-default-lbp))            ; left binding power
  (rbp (active-default-rbp))            ; right binding power
  led-fn nud-fn                         ; led and nud parse functions
  pos rpos lpos                         ; parts of speech
  led-lisp-alias                        ; lisp alias
  nud-lisp-alias                        ;  ditto
  matchfix separator                    ; matchfix-related info
  )

(defun (morpheme :named-structure-invoke) (op self &optional arg1
                                                   &rest ignore)
  (selectq op
    ((:which-operations) '(:print-self))
    ((:print-self) (format arg1 "#<Morpheme ~A ~A ~O>"
                           (morpheme-language self)
                           (%pointer self)))))
```

```
;;; Syntactic Structure

(defmacro syntax (char) '(aref (active-syntax-table) ,char))

(defmacro syntax-definition (char language)
  '(aref (language-syntax-table ,language) ,char))

(defun make-syntax-table ()
  (make-array *character-set-size*
              ':leader-length 2
              ':named-structure-symbol 'syntax-table))

(defmacro def-syntax (language &body syntax-definitions)
  '(let ((language (language-structure ',language)))
     ,@(mapcar
         #'(lambda (x)
             (cond ((atom (car x))
                    '(setf (syntax-definition ',(car x) language)
                           ,(cadr x)))
                   ((and (atom (cdar x)) (cdar x))
                    '(do ((i ',(caar x) (1+ i))
                          (val ,(cadr x)))
                         ((> i ',(cdar x)))
                       (setf (syntax-definition i language) val)))
                   (t
                    '(let ((val ,(cadr x)))
                       (mapcar #'(lambda (i)
                                   (setf (syntax-definition i language)
                                         val))
                               ',(car x))))))
         syntax-definitions)))
```

```
;;;; Syntax Analysis

(defun char-peek (stream) (tyipeek nil stream #\end))
(defun char-read (stream) (tyi         stream #\end))

(defun tokenize (stream)
  (do ((fn (syntax (char-peek stream))
           (syntax (char-peek stream))))
      (fn (funcall fn stream))
    (char-read stream)))

(defun alphabetp (c) (or (and (^] c #/A) (^\ c #/Z))
                         (and (^] c #/a) (^\ c #/z))))

(defun digitp (c) (and (^] c #/0) (^\ c #/9)))

(defun tokenize-number (stream)
  (let ((ibase 10.))
    (let ((chars (tokenize-number-internal stream)))
      (let ((result (ignore-errors (read-from-string chars))))
        (cond ((not (numberp result))
               (parse-error "Number parse failed: ~S" chars))
              (t result))))))

(defun tokenize-number-internal (stream) ;returns a string
  (with-output-to-string (output)
    (selectq (char-peek stream)
      ((#/+) (char-read stream))
      ((#/-) (tyo (char-read stream) output)))
    (do ((num-flag nil) (dot-flag nil) (e-flag nil)) (nil)
      (cond ((digitp (char-peek stream))
             (setq num-flag t)
             (tyo (char-read stream) output))
            ((eq (char-peek stream) #/.)
             (cond (dot-flag (parse-error "Too many .'s")))
             (setq dot-flag t)
             (tyo (char-read stream) output))
            ((memq (char-peek stream) '(#/E #/e))
             (cond ((not dot-flag)
                    (parse-error "Exponential form requires a dot."))
                   (e-flag
                    (parse-error
                      "Only one E allowed in exponential form."))
                   (t
                    (setq e-flag t)
                    (tyo (char-read stream) output)
                    (cond ((memq (char-peek stream) '(#/- #/+))
                           (tyo (char-read stream) output))))))
            (t
             (cond ((and dot-flag (not e-flag))
                    (cond ((not num-flag)
                           (parse-error "No digits in number.")))
                    (tyo #/0 output))) ;force floating point!
             (return nil))))))

(defun casify (x) (funcall *casification-hook* x))

(defun tokenize-symbol (stream)
  (intern-KBE (with-output-to-string (output)
                (do () (nil)
                  (cond ((or (digitp (char-peek stream))
                             (alphabetp (char-peek stream)))
                         (tyo (casify (char-read stream)) output))
                        (t (return nil)))))))

(defun tokenize-single-char (stream)
  (intern-KBE (string (char-read stream))))
```

```
(defun tokenize-stars (stream)
  (char-read stream) ;the first * is there for sure
  (cond ((= (char-peek stream) #/*)
         (char-read stream)
         '**)
        (t '*)))

(defun tokenize-dash (stream)
  (char-read stream)
  (cond ((= (char-peek stream) #/>)
         (char-read stream)
         '->)
        (t '-)))
```

```
;;;; Semantic Analysis

(defun parse-error (&rest stuff)
  (lexpr-funcall #'complain stuff))

(defprop parse-error t :error-reporter)

;;; type-checking stuff

(defmacro def-coercion (name arglist &body body)
  '(progn 'compile
          (defvar ,name nil
            ,(format nil
                     "Parser-datatype marker/function for type ~A" name))
          (defun ,name ,arglist ,@body)
          (if (or (not (boundp ',name)) (not ,name))
              (setq ,name #',name))))

(defun typed-parse (type item) (cons type item))

(defun coerce-parsed-form (pos item) (funcall pos item))

(defmacro type-field (item) '(car ,item))
(defmacro data-field (item) '(cdr ,item))

(def-coercion *any* (item) (data-field item))

(def-coercion *pointer* (item)
  (local-declare ((special *expression*)) ;forward reference
    (cond ((or (eq (type-field item) *any*)
               (eq (type-field item) *expression*))
           (data-field item))
          (t (parse-error "Odd pointer expression seen")))))

(def-coercion *expression* (item)
  (cond ((or (eq (type-field item) *any*)
             (eq (type-field item) *expression*)
             (eq (type-field item) *pointer*))
         (data-field item))
        (t (parse-error "~&Expression not seen where expected.~
                         ~% Data: ~S" (type-field item)))))

(def-coercion *statement* (item)
  (let ((result (data-field item)))
    (cond ((atom result) result)
          ((memq (car result) '(= eq)) ;good ol' infix overloading!
           (cond ((symbolp (cadr result))
                  '(setq ,@(cdr result)))
                 (t
                  '(setf ,@(cdr result)))))
          (t result)))) ;be lax about error checking.

;;; functions to handle nud/led operators

(defun nud (op)
  (let ((fn (nud-fn op)))
    (cond ((not fn) (parse-error "not a prefix operator: ~A" op))
          (t (funcall fn op)))))

(defun led (op left)
  (let ((fn (led-fn op)))
    (cond ((not fn) (parse-error "not an infix operator: ~A" op))
          (t (funcall fn op left)))))

;;; buffer abstraction

(defvar *buffer* nil)
(defvar *buffer-storage* nil)
```

```
(defun token-peek-2 ()
  (cond ((not *buffer-storage*) (token-peek) (token-peek-2))
        ((cdr *buffer-storage*) (cadr *buffer-storage*))
        (t (setq *buffer-storage*
                 (nconc *buffer-storage* (ncons (tokenize *buffer*))))
           (token-peek-2))))

(defun token-peek ()
  (cond (*buffer-storage* (car *buffer-storage*))
        (t (car (setq *buffer-storage* (ncons (tokenize *buffer*)))))))

(defun token-read ()
  (cond (*buffer-storage* (pop *buffer-storage*))
        (t (tokenize *buffer*))))

;;; interface to real world

(defun parser (&rest args)
  (lexpr-send terminal-io ':rubout-handler nil #'parser-internal args))

(defun string-parser (string &rest args)
  (with-input-from-string (stream string)
    (lexpr-funcall #'parser-internal stream args)))

(defun parser-internal (&optional
                        (stream terminal-io)
                        (language *input-language*))
  (let ((*language* (language-structure language))
        (*buffer-storage* nil)
        (*buffer* stream))
    (parse *any* 0)))

;;; driver loop

(defun parse (part-of-speech rbp) ; right binding power
  (do ((left (cond ((semantics (token-peek))
                    (nud (token-read)))
                   (t (typed-parse *any* (token-read))))
             (led (token-read) left)))
      ((^] rbp (lbp (token-peek)))
       (coerce-parsed-form part-of-speech left))))

;;; canned routines for easy parses

(defun parse-prefix (op)
  (typed-parse (pos op)
          (list (nud-lisp-alias op) (parse (rpos op) (rbp op)))))

(defun parse-infix (op left)
  (typed-parse (pos op)
          (list (led-lisp-alias op)
                (coerce-parsed-form (lpos op) left)
                (parse (rpos op) (rbp op)))))

(defun parse-postfix (op left)
  (typed-parse (pos op)
          (list (led-lisp-alias op)
                (coerce-parsed-form (lpos op) left))))

(defun parse-nary (op left)
  (typed-parse (pos op)
               (list* (led-lisp-alias op)
                      (coerce-parsed-form (lpos op) left)
                      (parse-nary-internal op (lpos op) (lbp op)))))

(defun parse-nary-internal (op pos rbp)
  (do ((result (list (parse pos rbp))
               (cons (parse pos rbp) result)))
```

```
      ((not (eq op (token-peek)))
       (reverse result))
    (token-read)))

(defun parse-matchfix (op)
  (typed-parse (pos op)
          (cons (nud-lisp-alias op)
                (parse-matchfix-internal
                  (matchfix op)
                  (separator op)
                  (rpos op)
                  (rbp op)))))

;;; m1 obj1 sep obj2 sep obj3 ... sep objn m2
;;;

(defun parse-matchfix-internal (match separator pos rbp)
  (do ((result nil (cons (parse pos rbp) result)))
      ((eq (token-peek) match)
       (token-read)
       (reverse result))
    (cond ((not result) (comment do nothing))
          ((eq separator (token-peek)) (token-read))
          (t (parse-error "missing ~A" match)))))

(defun parse-nofix (op)
  (typed-parse (pos op) (list (nud-lisp-alias op))))

(defun parse-noop (op)
  (typed-parse (pos op) nil))

(defun parse-left (op left)
  (typed-parse (pos op) (coerce-parsed-form *statement* left)))

;;; definitions of actual morphemes

;(defun parse-sign (op left)
;  (setq left (coerce-parsed-form *expression* left))
;  (do ((result (list (cond ((eq op '/-)
;                            (list (nud-lisp-alias op)
;                                  (parse *expression* (lbp op))))
;                           (t (parse *expression* (lbp op))))
;                     left)
;               (cons (parse *expression* (lbp op)) result)))
;      ((not (memq (token-peek) '(/+ /-)))
;       (typed-parse *expression*
;                    (cons (led-lisp-alias '/+) (reverse result))))
;    (cond ((eq (token-peek) '/+) (token-read)))))

(defun parse-nud-lparen (op)
  (cond ((eq (token-peek) (matchfix op))
         (parse-error "Invalid parenthesized expression"))
        (t
         (let ((right (parse-matchfix-internal
                        (matchfix op)
                        (separator op) ;shouldn't get used
                        (rpos op)
                        (rbp op))))
           (cond ((or (null right) (cdr right))
                  (parse-error "Too many expressions?")) ;shouldn't happen
                 (t
                  (typed-parse (pos op) (car right)))))))))

(defun parse-led-lparen (op left)
  (setq left (coerce-parsed-form (lpos op) left))
  (cond ((not (symbolp left))
         (parse-error "Invalid expression in operator position"))
        (t
         (let ((right (parse-matchfix-internal
```

```
                (matchfix op)
                (separator op)
                (rpos op)
                (rbp op))))
        (typed-parse (pos op) (cons left right))))))
```

```
;;;; Algol

(def-language algol

  (/^       lbp             140.
            rbp             139.
            pos             *expression*
            lpos            *expression*
            rpos            *expression*
            led-fn          #'parse-infix)

  (/*       lbp             120.
            pos             *expression*
            lpos            *expression*
            led-fn          #'parse-nary)

  (/+       lbp             100.
            rbp             100.
            pos             *expression*
            rpos            *expression*
            lpos            *expression*
            led-fn          #'parse-nary
            nud-fn          #'parse-prefix)

  (/-       lbp             100.
            rbp             134.
            pos             *expression*
            rpos            *expression*
            lpos            *expression*
            led-fn          #'parse-nary
            nud-fn          #'parse-prefix)

  (begin    rbp             10.
            pos             *any*
            rpos            *expression*
            nud-fn          #'parse-matchfix
            nud-lisp-alias  'algol-block
            matchfix        'end
            separator       '/;)

  (/;       lbp             5)

  (end      lbp             5)

  (#.(values (intern (string #\end)))
            lbp             -1.) )

(def-syntax algol
  ((#/A . #/Z)                      #'tokenize-symbol       )
  ((#/a . #/z)                      #'tokenize-symbol       )
  ((#/0 . #/9)                      #'tokenize-number       )
  ((#/+ #/- #/* #// #/^ #\END #/;) #'tokenize-single-char ))
```

```
;;;; PL/1

(defun parse-PL/1-do (op)
  (typed-parse (pos op)
               (parse-PL/1-do-internal op (separator op) (matchfix op))))

(defun parse-PL/1-do-internal (op separator match)
  (do ((result nil (cons statement result))
       (statement (parse-PL/1-statement op) (parse-PL/1-statement op)))
      ((eq statement match)
       (let ((forms (reverse result)))
         (cond ((null (car forms))
                '(PROGN ,@(cdr forms)))
               (t
                '(PL1-ITER ,@forms)))))
    (cond ((eq (token-peek) separator)
           (token-read))
          (t (parse-error "Missing ~A" separator)))))

(defun parse-PL/1-statement (op)
  (let ((statement (cond ((eq (token-peek) '/;) NIL)
                         (t (parse *statement* (rbp op))))))
    statement))

(defun parse-PL/1-end (op)
  (cond ((not (eq (token-peek) '/;)) (token-read))) ;optional tag
  (typed-parse (pos op) op))

(defun parse-PL/1-if (op)
  (let ((condition (parse *expression* (rbp op))))
    (cond ((not (eq (token-peek) 'then))
           (parse-error "Missing THEN in ~A statement" op)))
    (let ((consequent (parse-PL/1-statement (token-read))))
      (typed-parse (pos op)
                   '(cond (,condition ,consequent)
                          ,@(cond ((eq (token-peek-2) 'else)
                                   (token-read) ;flush the ";"
                                   '((t ,(parse-PL/1-statement
                                           (token-read))))))))))))

(defun parse-PL/1-pointer (op left)
  (setq left (coerce-parsed-form (lpos op) left))
  (cond ((not (symbolp left))
         (parse-error "Unrecognized pointer expression seen"))
        (t
         (do ((result left) (flag nil t))
             ((and flag (not (eq (token-peek) '->)))
              (typed-parse (pos op) result))
           (cond (flag (token-read))) ;pop arrows
           (setq result
                 (list (caseq (token-peek)
                         (next  'cdr)
                         (value 'car)
                         (t
                           (parse-error "Unrecognized pointer usage.~
                                         ~% Data: ~A" (token-peek))))
                       result))
           (token-read) ;pop the used token
           ))))

(defun parse-PL/1-= (op left)
  (typed-parse (pos op)
               (list (cond ((eq (type-field left) *pointer*) 'EQ)
                           (T '=))
                     (coerce-parsed-form (lpos op) left)
                     (parse (rpos op) (rbp op)))))
```

```
(def-language PL/1

  (->       lbp             180.
            rbp             180.
            pos             *pointer*
            lpos            *expression*
            rpos            *expression*
            led-fn          #'parse-PL/1-pointer)

  (**       lbp             140.
            rbp             139.
            pos             *expression*
            lpos            *expression*
            rpos            *expression*
            led-lisp-alias  '/^
            led-fn          #'parse-infix)

  (//       lbp             120.
            pos             *expression*
            lpos            *expression*
            led-fn          #'parse-nary)

  (*        lbp             120.
            pos             *expression*
            lpos            *expression*
            led-fn          #'parse-nary)

  (+        lbp             100.
            rbp             100.
            pos             *expression*
            rpos            *expression*
            lpos            *expression*
            led-fn          #'parse-nary
            nud-fn          #'parse-prefix)

  (-        lbp             100.
            rbp             134.
            pos             *expression*
            rpos            *expression*
            lpos            *expression*
            led-fn          #'parse-nary
            nud-fn          #'parse-prefix)

  (do       lbp             30
            rbp             25.
            pos             *statement*
            rpos            *statement*
            nud-fn          #'parse-PL/1-do
            matchfix        'end
            separator       '/;)

  (if       rbp             45.
            ;no lbp (default as high as possible)
            pos             *any*
            rpos            *any*
            nud-fn          #'parse-PL/1-if
            )

  (then     lbp             5.
            rbp             25.)

  (else     lbp             5.
            rbp             25.)

  (>        lbp             80.
            rbp             80.
            pos             *expression*
            rpos            *expression*
            lpos            *expression*
```

```
          led-fn          #'parse-infix)

 (<       lbp             80.
          rbp             80.
          pos             *expression*
          rpos            *expression*
          lpos            *expression*
          led-fn          #'parse-infix)

 (=       lbp             80.
          rbp             80.
          pos             *expression*
          rpos            *expression*
          lpos            *expression*
          led-fn          #'parse-PL/1-=)

 (/(      lbp             200.
          rbp             10.
          pos             *expression*
          lpos            *expression*
          rpos            *expression*
          led-fn          #'parse-led-lparen
          nud-fn          #'parse-nud-lparen
          matchfix        '/)
          separator       '/,)

 (/,      lbp             10.) ;probably a nud-err/led-err should be here

 (/)      lbp             5.) ;probably a nud-err/led-err should be here

 (/;      lbp             5.
          rbp             10.
          pos             *statement*
          led-fn          #'parse-left)

 (end     lbp             5
          pos             *statement*
          nud-fn          #'parse-PL/1-end)

 (#.(values (intern (string #\end)))
          lbp             -1.) )

(def-syntax PL/1
  ((#/A . #/Z)                    #'tokenize-symbol       )
  ((#/a . #/z)                    #'tokenize-symbol       )
  ((#/0 . #/9)                    #'tokenize-number       )
  (#/.                            #'tokenize-number       )
  (#/*                            #'tokenize-stars        )
  (#/-                            #'tokenize-dash         )
  ((#/+ #// #/^
    #/( #/) #/,
    #/= #/< #/>
    #\END #/; #/:)                #'tokenize-single-char ))


;;; Temporary cross-file compatibility measure

(setf (language-structure 'PLI) (language-structure 'PL/1))
```

```
;;;; LISP

(def-language LISP
  (#.(values (intern (string #\end   ))) lbp -1)
  (#.(values (intern (string #\tab   ))) lbp -1)
  (#.(values (intern (string #\space ))) lbp -1)
  (#.(values (intern (string #\return))) lbp -1)
  (#.(values (intern (string #\line  ))) lbp -1)
  (#.(values (intern (string #/.     ))) lbp -1)
  (#.(values (intern (string #/,     ))) lbp -1)
  (#.(values (intern (string #/)     ))) lbp -1) )

(def-syntax LISP
  ((0 . 255) #'read-hacking-/{) ; 255 = #.(1- *character-set-size*)
  ((#/. #/, #/) #\END #\TAB #\SPACE #\RETURN #\LINE)
   #'tokenize-single-char))
```

```
;;;; ada

(def-language ada
  (#.(values (intern (string #\end   ))) lbp -1) )

(def-syntax ada
  ((0 . 255) #'ada-parse-to-end) ; 255 = #.(1- *character-set-size*)
  ((#\END) #'tokenize-single-char))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; EXPLAIN: Module for generation of automatic explanations of code.

;;; This code was originally written by Scott Cyphers in Spring, 1982.
;;; See MIT-AI-WP-237, "Automatic Program Description" for details.
;;; On 6-Dec-82, the file OZ:PS:<KBE.EXPLANATION>EXPLAIN.LISP.4 was
;;; snarfed and merged into the KBE sources with some minor modifications
;;; by KMP.

;;; Debugging Utilities

(defvar *trace-explanations* nil "Flag: type out debugging info?")

(defmacro trace-explanation (&rest args)
  '(cond (*trace-explanations* (format t ,@args))))

(define-internal-option *EXPLAIN-debug* "Debug explanation errors"
  nil :boolean)

;; this sets up a gard to detect recursive calls to a function.

(defvar recursion-stack nil "used by recursion-check")

(defmacro recursion-check (call &body body)
  '(let ((recursion-stack (cons ,call recursion-stack)))
     (cond ((member (car recursion-stack) (cdr recursion-stack))
            (when *EXPLAIN-debug*
              (cbarf "Allow this explanation request to fail."
                     "Explain: Recursive call ~:N"
                     (car recursion-stack)))
            nil)
           (t
            ,@body))))
```

```
;;;; Abstraction from KBE internals

(defun cliche? (segcase)
  (~01 (??& segcase CLICHE)))

(defun cliche-name (cliche)
  (~01 ~clichename (??& cliche CLICHE)))

(defun plan-type (segcase)
  (~01 ~type (??& segcase PLANTYPE)))

(defun has-plan-type (segcase type)
  (~01 ~type (??& segcase PLANTYPE type)))

(defun has-slot (segcase thing)
  (and (not (fact? thing))
       (??& segcase SLOT thing)))

(defun slot-name (segcase thing)
  (and (not (fact? thing))
       (~01 ~slotname (??& segcase SLOT thing))))

(defun segment-of (thing &aux (type (thing-type thing)))
  (selectq type
    (seg thing)
    (@io (~segcase (~deep thing)))
    (t (~segcase thing))))

(defun subseg* (seg1 seg2)
  (cond ((equal seg1 seg2) t)
        ((null seg1) nil)
        (t (subseg* (& (?supseg (~seg seg1))) seg2))))

(defun from (port &aux (data-flow (??& * DFLOW * port)))
  (cond (data-flow (~from (car data-flow)))
        (t (~01 (??& * TINPUT * * port)))))

(defun to (port &aux (data-flow (??& * DFLOW port *)))
  (cond (data-flow (~nset ~to data-flow))
        (t (~nset (??& * TOUTPUT * * port)))))

(defun dest-is-term (port)
  (do ((ports (to port) (mapcan #'to losers))
       (winners nil)
       (losers nil nil))
      ((null ports) winners)
    (dolist (port ports)
      (cond ((and (eq (thing-type port) 'tinput)
                  (??& * ROLE (segment-of port) 'term))
             (push (segment-of port) winners))
            (t (push port losers))))))

(defun find (cliche-name)
  (~nset ~segcase (??& * CLICHE cliche-name)))
```

```
;;;; Automatic Explanations

(defun explain-fn (fn)
  (let ((sc (name->segcase fn)))
    (when sc
      (let ((exp (condition-case-if (not *EXPLAIN-debug*) (cond)
                     (explain sc)
                   (sys:pdl-overflow
                     (text "Got a ~A pdl overflow generating explanation!"
                           (send cond ':pdl-name)))
                   (sys:error
                     (text "Got an error generating explanation!~@[~%~A~]"
                           (if *debug* cond))))))
        (when exp
          '(:text ,exp :auto-fill t))))))

(defun explain1 (thing relative-to implementation)
  (cond ((has-slot relative-to thing)
         (cond (implementation
                (format nil "~A, implemented as ~A"
                        (explain thing relative-to ':outer)
                        implementation))
               (t (explain thing relative-to ':outer))))
        (t (let ((part (and (not (fact? thing))
                            (~01 (??& * SLOT thing)))))
             (cond (part (explain thing (segment-of part) ':outer))
                   (t implementation)))))))
```

```
(defun explain (thing &optional (relative-to thing) (i-level ':detail)
                      &aux (level i-level))
  (trace-explanation "~%Explaining ~S: ~S-- ~A" thing relative-to level)
  (recursion-check (list 'explain thing relative-to i-level)
    (setq relative-to (segment-of relative-to))
    (let ((explanation-segment (selectq i-level
                                 ((:DETAIL)  relative-to)
                                 ((:SUMMARY) (segment-of thing))
                                 ((:OUTER)   relative-to)))
          type)
      (if (eq i-level ':outer) (setq level ':summary))
      (setq type
            (cond ((symbolp thing) (prog1 thing (setq thing nil)))
                  ((slot-name explanation-segment thing))
                  ((thing-type thing))))
      (cond ((and (eq type 'seg)
                  (not (equal thing explanation-segment)))
             (explain thing relative-to ':summary))
            ((and (eq type 'toutput)
                  (not (equal (segment-of thing) relative-to)))
             (or (explain thing (segment-of thing) ':summary)
                 (explain (segment-of thing) relative-to ':outer)))
            ((and (eq type 'output)
                  (not (equal (segment-of thing) relative-to)))
             (or (explain (segment-of thing) relative-to ':outer)
                 (explain thing (segment-of thing) ':summary)))
            (t
             (let ((cliche-name (cliche-name explanation-segment))
                   explanation-handler
                   plantype)
               (cond ((get cliche-name 'cliche-comments)
                      (handle cliche-name level
                              relative-to explanation-segment type thing))
                     (t
                      (setq plantype (plan-type explanation-segment))
                      (setq explanation-handler (get plantype level))
                      (cond (explanation-handler
                             (funcall explanation-handler
                                      relative-to type thing))
                            ((and (null plantype)
                                  (neq i-level ':outer))
                             (explain thing relative-to ':outer)))))))))))
```

```
(defun explain-input (input default)
  (let ((name (~01 ~name (??& input @io))))
    (cond (name (format nil "~A" name))
          (default)
          (t (or (explain-input1 (from input))
                 (format nil "the input {~A}" (~obj input)))))))

(defun explain-input1 (input)
  (cond ((memq (thing-type input) '(output toutput))
         nil)
        ((null input) nil)
        (t (let ((name (~01 ~name (??& input @io))))
             (cond (name (format nil "~A" name))
                   (t (explain-input1 (from input))))))))

(defun find-inputs (segment relative-to)
  (mapcar #'(lambda (x) (find-input x relative-to))
          (??& segment INPUT)))

(defun find-input (input relative-to)
  (trace-explanation "~&FIND-INPUT ~S~S" input relative-to)
  (cond ((equal (segment-of input) relative-to)
         (explain-input input nil))
        (t (or (find-input1 (from input) relative-to)
               (explain-input input nil)))))

(defun find-input1 (input relative-to)
  (trace-explanation "~&FIND-INPUT1 ~S~S" input relative-to)
  (cond ((null input) nil)
        ((or (equal (segment-of input) relative-to)
             (??& relative-to SLOT (segment-of input)))
         (explain input relative-to ':outer))
        ((memq (thing-type input) '(output toutput))
         (explain input relative-to ':summary))
        (t (find-input1 (from input) relative-to))))

(defun explain-output (output &aux (kind (~kind (?& output @IO))))
  (cond ((eq kind 'returnval)
         (format nil "the returned value, {~A}," (~obj output)))
        (t (format nil "the output {~A}" (~obj output)))))
```

```
(defun explain-parts (relative-to partname)
  (let ((parts (??& relative-to SLOT * partname)))
    (cond (parts
           (phrase-append "and"
                          (mapcar #'(lambda (master)
                                      (explain-part relative-to
                                                    master
                                                    partname))
                                  parts)))
          (t (or (explain partname relative-to ':outer)
                 (format nil "{MISSING PART ~A IN ~A}"
                         partname relative-to))))))

(defun explain-part (relative-to part partname)
  (cond ((and (eq (thing-type (~slotseg part)) 'seg)
              (not (cliche? (~slotseg part)))
              (has-plan-type (~slotseg part) 'pred))
         (explain-termination relative-to partname))
        (part
         (or (explain (~slotseg part) relative-to ':summary)
             (explain (~slotseg part) relative-to ':outer)))
        (t (format nil "{MISSING PART ~A IN ~A}" partname relative-to))))

(defun explain-termination (relative-to partname)
  (let ((part (~01 ~slotseg (??& relative-to slot * partname))))
    (cond (part (explain-wff part relative-to))
          (t (format nil "{MISSING PART ~A IN ~A}"
                     partname relative-to)))))

(defun explain-wff (segment relative-to)
  (phrase-append "or" (make-termination-wff
                       (~segcase (?& * ROLE segment))
                       relative-to)))
```

```
(defun make-termination-wff (ssr relative-to)
  (let* ((body (~from (?& ssr CFLOW * ssr)))
         (join (~from (?& body CFLOW * body)))
         (other-terms
           (mapcan #'(lambda (gronk)
                       (make-termination-wff gronk relative-to))
                   (mapcan #'dest-is-term (??& ssr TOUTPUT)))))
    (cons
      (do ((term-segcase
             (~from (?& body CFLOW * (& (~seg join) 'pred)))
             (~from (?& (& (~seg term-segcase)) CFLOW * term-segcase)))
           (wff nil nil)
           (term-part nil))
          ((or (null term-segcase)
               (setq wff (~01 ~wff (??& term-segcase WHEN))))
           (explain1
             (or term-part
                 (& (~seg term-segcase)))
             relative-to
             (cond ((null wff) nil)
                   ((and (listp wff)
                         (eq (car wff) 'not))
                    (format nil "not ~A"
                            (find-input
                              (?& (& (~seg term-segcase)) INPUT
                                  * (cadr wff))
                              relative-to)))
                   (t (find-input (?& (& (~seg term-segcase)) INPUT
                                      * wff)
                                  relative-to)))))
        (if (and (null term-part)
                 (??& * slot (& (~seg term-segcase))))
            (setq term-part (& (~seg term-segcase)))))
      other-terms)))
```

```
(defun handle (cliche level relative-to explanation-segment type thing)
  (let ((exp (handler-format relative-to cliche type level)))
    (selectq level
      ((:DETAIL)
       (cond ((eq type 'seg)
              (handle-format exp relative-to thing level))
             (exp
              (format nil "~A, implemented as ~A"
                      (handle-format exp relative-to thing level)
                      (explain thing relative-to ':summary)))
             (t "Don't know.")))
      ((:SUMMARY)
       (cond (exp (cond
                   ((memq (thing-type thing) '(input tinput))
                    (explain-input thing
                                   (handle-format
                                     exp relative-to thing level)))
                   (t (let ((explanation
                              (handle-format
                                exp explanation-segment thing level)))
                        (selectq type
                          (seg (a-or-an explanation))
                          (t explanation))))))
             (t (selectq type
                  ((input tinput) (find-input thing relative-to))
                  (t (explain thing relative-to ':summary)))))))))

(defun handle-format (format relative-to thing level)
  level thing ;ignored? --KMP
  (or (listp format) (setq format (list format)))
  (do ((string "" (format nil "~A~A~A" string space next-string))
       (space "" " ")
       (next-string)
       (format format (cdr format)))
      ((null format) string)
    (setq next-string
          (selectq (typep (car format))
            (:symbol (explain-parts relative-to (car format)))
            (:string (car format))))))
```

```
(defun handler-format (relative-to cliche type level &aux handler)
  (setq type
        (cond ((eq type 'seg)
               (selectq level
                 (:detail 'detail)
                 (:summary 'main)))
              (t type)))
  (setq handler
        (or (get (get cliche 'cliche-comments) type)
            (if (not (memq type
                           '(detail main input output tinput toutput)))
                (de-hyphenate type))))
  (cond ((and (neq type 'detail)
              (neq type 'main))
         (let ((parts (or (??& relative-to slot * type)
                          (if (memq type '(action init pred join
                                           rec op body dfjoin
                                           bssr gen acc trn
                                           term filter))
                              (??& relative-to ROLE * type)))))
           (if handler
               (setq handler
                     (list
                       (selectq (length parts)
                         (0 handler)
                         (1 (format nil "the ~A" handler))
                         (t (a-or-an handler)))))))))
  handler)

(defun constant-handler? (format)
  (and (= (length format) 1)
       (stringp (car format))))
```

```
;;; -*- Mode:LISP; Package: KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

(defun document-fn (fn)
  (let* ((sc (~01 ~segcase (??& * @seg 'lambda-exp fn)))
         (cliche (and sc (??& sc cliche))))
    (when cliche
      '(:text ,(document-fn1 (~seg sc) fn) :auto-fill nil))))

(declare (special *2focus-seg*))  ;see definition of kbe-/#-comment-macro

(defun document-fn1 (seg name)
  (let* ((*2focus-seg* seg)
         (cliche-name (~clichename (?& (&* seg) cliche)))
         (comments (get cliche-name 'cliche-comments))
         (role-tree (get comments 'role-tree))
         (described-roles (get comments 'described-roles))
         (main-sentence '(+doc-sentence "The" "program" ,name "is"
                          ,(select-a-or-an cliche-name)
                          ,(string-downcase cliche-name)))
         (role-doc-sentences))
    (prog (role-name roles-to-do)
          (setq roles-to-do described-roles)
        L (when (null roles-to-do)
            (setq role-doc-sentences (nreverse role-doc-sentences))
            (return))
          (setq role-name (pop roles-to-do))
          (let* ((segcase
                   (eval (make-kbe-comment nil (list 'the role-name))))
                 (cliche (if segcase
                             (~01 ~clichename (??& segcase cliche)))))
            (cond ((null segcase)
                   (push '(+doc-sentences
                            (+doc-sentence "There"
                             ,(select-is-or-are role-name)
                             "no" ,(string-downcase role-name)))
                         role-doc-sentences))
                  ((?empty-slot (~seg segcase))
                   (push '(+doc-sentences
                            (+doc-sentence "The"
                             ,(string-downcase role-name)
                             ,(select-is-or-are role-name)
                             "not" "implemented"))
                         role-doc-sentences))
                  ((null cliche)
                   (let ((code (or (possible-embedded-code segcase 60)
                                   "an idiosyncratic computation")))
                     (if code
                         (push '(+doc-sentences
                                  (+doc-sentence "The"
                                   ,(string-downcase role-name)
                                   ,(select-is-or-are role-name) ,code))
                               role-doc-sentences))))
                  ((eq cliche 'pseudo-subcliche)
                   (setq roles-to-do
                         (append (reverse (cdr (assq role-name role-tree)))
                                 roles-to-do)))
                  (T (push '(+doc-sentences
                              (+doc-sentence "The"
                               ,(string-downcase role-name)
                               ,(select-is-or-are role-name)
```

```
                              ,(select-a-or-an cliche)
                              ,(string-downcase cliche))
                             ,(process-comment-string
                                (~seg segcase) cliche))
                           role-doc-sentences))))
      (go L))
(let ((pp:*print-right-margin* 70.))
  (format nil "~:="
          (list* '+doc-whole main-sentence role-doc-sentences)))))
```

```
(pp:define-list-print-function +doc-whole (sentences s)
  (format s "~@!~2I~{~=~^~%~}~." (cdr sentences)))

(pp:define-list-print-function +doc-sentences (sentences s)
  (format s "~@!~3I~{~=~^  ~:_~}~." (cdr sentences))) ;odd if >2

(pp:define-list-print-function +doc-sentence (sentence s)
  (let ((*print-escape* nil))
    (format s "~@!~{~=~^ ~:_~}.~." (cdr sentence))))

(defun process-comment-string (seg cliche)
  (let* ((*2focus-seg* seg)
         (comments (get cliche 'cliche-comments))
         (string (get comments 'comment))
         (words))
    (if (and (listp string) (eq (car string) 'str-lit)) ;for ada
        (setq string (cadr string)))
    (setq words (fix-up-semi-kbe-/#-comment-macros
                  (read-from-string (string-append "(" string ")"))))
    (list* '+doc-sentence
           '"It"
           (mapcan #'(lambda (w)
                       (if (symbolp w) (ncons (string-downcase (string w)))
                           (let ((code (possible-embedded-code
                                         (eval w)
                                         20)))
                             (if code (ncons code)
                               (make-a-phrase (kbe-comment-name w))))))
                   words))))

;the y -> a y
;the x -> an x
;the values -> values

(defun make-a-phrase (name-list)
  (do ((l name-list (cdr l))
       (result nil))
      ((null l) (nreverse result))
    (cond ((not (eq (car l) 'the)) (push (string-downcase (car l)) result))
          ((or (null (cdr l)) (equal (select-is-or-are (cadr l)) "are")))
          (T (push (select-a-or-an (cadr l)) result)))))

(defun possible-embedded-code (role-segcase max-length)
  (if role-segcase
      (let* ((@seg (~01 (??& role-segcase @seg)))
             (code (or (~01 ~varname (??& (&* role-segcase output)
                                          @createvar))
                       (~01 ~name (ignore-errors
                                    (??& (&* role-segcase output * 'ret)
                                         @io 'free-var))) ;kludge
                       (and (eq (~kind @seg) 'constant)
                            (cadr (~name @seg)))
                       (car (~code @seg))))
             (good-code (doc-decode-code code))
             (code-string
               (with-output-to-string (str)
                 (print-code good-code str))))
        (cond ((and good-code
                    (not (?empty-slot (~seg role-segcase)))
                    (< (string-length code-string) max-length))
               (cond ((stringp good-code) code-string)
                     ((not (listp good-code)) good-code)
                     ((multiple-value-bind (ignore last)
                          (read-from-string code-string)
                        (< last (string-length code-string)))
                      (string-append "(" code-string ")"))
                     (T code-string)))
              ((and (null code)
```

```
                    (eq (~kind @seg) 'funcall)
                    (eq (~name @seg) '6noop))
               (let* ((to (~to (car (??& * dflow
                                         (&* role-segcase output))))))
                 (if to (~01 ~name (??& to @io)))))))))

(defun doc-decode-code (code)
  (cond ((not (listp code)) code)
        ((memq (car code) '(*code-is-car* *code-is-cdr*))
         (doc-decode-code (cadr code)))
        ((eq (car code) 'kbe-/#-comment-macro)
         (cond ((null (cadr code)) nil)
               ((and (listp (cadr code)) (eq (cadr (cadr code)) '...)) nil)
               (T (caadr code))))
        (T code)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; PBM-HANDLERS: Definitions of explanation stuff for PBMs.

;;;; Abstraction support

;;; (DEFINE-EXPLANATION-TYPE type)
;;;
;;;   Defines a function DEFINE-type using the given TYPE which will
;;;   work for defining that kind of explanation.

(defmacro define-explanation-type (type)
  (let ((property (intern-keyword type))
        (defname  (symbolconc "DEFINE-" type)))
    '(defmacro ,defname (pbm-name bvl &body body)
       '(defun (:property ,pbm-name ,',property) ,bvl
          ,@bvl ;ignorable
          ,@body))))

;;; Define DEFINE-SUMMARY, etc. These definition forms allow us to
;;; define various kinds of explanations for PBMs.

(define-explanation-type SUMMARY)
(define-explanation-type DETAIL)
```

```
;;;; Constants

;;; CONSTANT

(define-summary CONSTANT (relative-to type thing)
  relative-to ;ignored
  (format nil "~A" (cadr (~name (?& (segment-of thing) @seg)))))

(define-detail CONSTANT (relative-to type thing)
  relative-to ;ignored
  (let ((value (cadr (~name (?& (segment-of thing) @seg)))))
       (selectq type
         (seg (format nil "Produces the constant value ~A" value))
         (output (format nil "The value, ~A, of the constant" value)))))
```

```
;;;; Expr

;;; EXPR

(define-summary EXPR (relative-to type thing)
  (selectq type
    (output (explain (from thing) relative-to ':summary))
    (seg (explain (from (?& thing output)) relative-to ':summary))))
```

```
;;;; New View

;;; NEW-VIEW

(define-summary NEW-VIEW (relative-to type thing)
  (trace-explanation "~&Summarizing (New-View)~
                      ~% Relative to ~S~
                      ~% Type ~S~
                      ~% Thing ~S~%" relative-to type thing)
  (let ((function (~name (?& (segment-of thing) @seg))))
    (selectq type
      ((seg output)
       (let ((my-name (or (get function 'noun) function)))
         (cond
           ((and (eq my-name '5noop)
                 (let ((data (find-inputs (segment-of thing) relative-to)))
                   (and data (not (cdr data)) (text "~A" (car data))))))
           (my-name
            (format nil "the ~A of ~A"
                    my-name
                    (phrase-append "and"
                                   (find-inputs (segment-of thing)
                                                relative-to))))
           (t (explain thing relative-to ':outer)))))
      (input (find-input thing relative-to)))))

(define-detail NEW-VIEW (relative-to type thing)
  (let ((segment (segment-of thing)))
    (let ((function (~name (?& segment @seg))))
      (selectq type
        (seg (let ((input-explanations
                     (find-inputs segment relative-to))
                   (output-explanations
                     (mapcar #'explain-output (??& segment output)))
                   (my-name (get function 'verb))
                   front)
               (setq front
                     (cond (my-name)
                           ((length (??& segment input))
                            (format nil "Takes the ~A of" function))
                           (t (format nil "Applies the function ~A to"
                                      function))))
               (format nil "~A ~A to produce ~A"
                       front
                       (phrase-append "and" input-explanations)
                       (phrase-append "and" output-explanations))))
        (input (format nil "~A of the function ~A"
                       (explain-input thing nil)
                       function))
        (output (format nil "~A of the function ~A"
                        (explain-output thing)
                        function))))))
```

```
;;;; Pred

;;; PRED

(define-summary PRED (relative-to type thing)
  thing relative-to ;ignored
  (selectq type
    (seg (format nil
                 "I'm a predicate."))
    (input "I'm an input to a predicate.")))

(define-detail PRED (relative-to type thing)
  relative-to  thing ;ignored
  (selectq type
    (seg "predicate")
    (input "input to a predicate")))
```

```
;;;; Misc

(defprop + "sum" noun)
(defprop + "adds" verb)
(defprop - "difference" noun)
(defprop - "subtracts" verb)
(defprop // "division" noun)
(defprop // "divides" verb)
(defprop * "product" noun)
(defprop * "multiplies" verb)
(defprop EQ "EQ comparison" noun)
(defprop EQ "compares (using EQ)" verb)
(defprop = "numeric comparison" noun)
(defprop = "compares the numbers" verb)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
```

```
;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
```

```
;;;; INIT: System setup stuff
```

```
;;; Jazzy PA Herald

(defvar *print-PA-herald* t)

(defun print-PA-herald ()
  (when *print-PA-herald*
    (send standard-output ':fresh-line)
    (let ((extra (* 2
                    (+ 5 ; pseudo-VSP
                       3 ; for the :increment-cursorpos below
                       (font-char-height ; 1 line of typeout in this font
                        (send standard-output ':current-font)))))
          (y (cadr
               (multiple-value-list
                 (send standard-output ':read-cursorpos))))
          (line-height (send standard-output ':line-height)))
      (send standard-output ':clear-eof)
      (send standard-output ':increment-cursorpos 3 3)
      (format t "The MIT Artificial Intelligence Laboratory presents...")
      (let ((h (+ extra (font-char-height fonts:43vxms)))
            (w (send standard-output ':inside-size)))
        (send standard-output ':display-x-y-centered-string
              "Programmer's Apprentice" 0 y w (+ y h) fonts:43vxms)
        (let ((b (+ y h)) (l (- w 2)))
          (send standard-output ':draw-line 0 y      l y      tv:alu-xor)
          (send standard-output ':draw-line 0 (1+ y) 0 (1- b) tv:alu-xor)
          (send standard-output ':draw-line 0 b      l b      tv:alu-xor)
          (send standard-output ':draw-line l (1+ y) l (1- b) tv:alu-xor)
          (let* ((tab-stop (+ b 5)) ;pseudo-vsp
                 (delta    (\ tab-stop line-height)))
            (if (not (zerop delta))
                (setq tab-stop (+ tab-stop (- line-height delta))))
            (send standard-output ':set-cursorpos 0 tab-stop)))))))
```

```
;;;; Support Functions

;;; IRRITATING-NOISE
;;;
;;; Two long low-pitched tones. Ideal for errors and such.

; This noise is currently not assigned.

(defun irritating-noise (&optional name &rest ignore)
       name ;ignored
  (sys:%beep 5000. 600000.)
  (sys:%beep 6000. 600000.))

;;; ZIPPY-NOISE-UP
;;; ZIPPY-NOISE-DOWN
;;;
;;; A long slide up/down. Good for something that toggles.

; This noise is currently not assigned.

(defun zippy-noise-internal (&optional (upward?  t)
                                       (start    2700.)
                                       (end      300. )
                                       (step     25.  )
                                       (duration 5000.))
  (cond (upward? (setq step (- step)))
        (t       (setq end (prog1 start (setq start end)))))
  (do ((pitch start (+ pitch step)))
      ((= pitch end))
    (sys:%beep pitch duration)))

(defun zippy-noise-down (&optional name &rest ignore)
  name ;ignored
  (zippy-noise-internal nil))

(defun zippy-noise-up (&optional name &rest ignore)
  name ;ignored
  (zippy-noise-internal t))

;;; PLEASANT-NOISE-1
;;; PLEASANT-NOISE-2
;;;
;;; Short high-pitched noises. #2 is slightly longer than #1.

(defun pleasant-noise-1 (&optional name &rest ignore) ;short
  name ;ignored
  (sys:%beep 290. 100000.)
  (sys:%beep 300. 100000.)
  (sys:%beep 200. 100000.))

(defun pleasant-noise-2 (&optional name &rest ignore) ;slightly longer
  name ;ignored
  (dotimes (i 3)
    (sys:%beep 200. 100000.)
    (sys:%beep 400. 100000.)))
```

```
;;;; Initialization

;;; Noise initializations

#-3600 ;The 3600 makes no noise
(set-noise-for 'code-ready    #'pleasant-noise-1)

#-3600 ;The 3600 makes no noise
(set-noise-for 'display-ready #'pleasant-noise-1)

(define-system-initialization system-keys
  (add-system-key #/A t   "Ada"                       'ada-listener)
  (add-system-key #/B t   "Background Window (KBE)" 'actor-window nil)
  (add-system-key #/E nil "Editor (Zmacs//KBE)"     'edit-frame)
  (add-system-key #/K t   "KBE"                       'simple-frame)
  (add-system-key #/Z t   "Editor (Zmacs)"          'zwei:zmacs-frame))

(define-system-initialization setup-Gprint
  (setq user:Gcheckrecursion nil))

(define-system-initialization setup-pp
  (setf (pp:generalized-print-function-enabled-p 'fact) T)
  (setq prin1 #'pp:top-write))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1984, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; This package sets up things which are probably worthwhile to have
;;; around for debugging but which are totally independent of anything
;;; the KBE depends on to run. Further, it should not rely on any macros
;;; in the KBE so that it doesn't get recompiled a lot...

;;; This takes in a form and macroexpands all of the macros inside
;;; it, until there are no macros left.  Note that it forces all of the
;;; macros to displace whether they do or not.  It then returns the
;;; result to you.  The hard part is that it has to know about fexprs
;;; in order to know what not to macroexpand.
;;;
;;; Note: This function should not be called from code.
;;;       It isn't reliable enough.
```

```
;;;; Utilities

(defun macroexpandall (thing)
  (cond ((eq (typep thing) 'list)
         (let ((new (macroexpand thing)))
           (rplaca thing (car new)) (rplacd thing (cdr new)))
         (cond ((not (symbolp (car thing)))
                (mapc #'macroexpandall thing))
               (t
                (selectq (car thing)
                  ((QUOTE COMMENT DEFPROP)
                     nil)
                  ((LAMBDA PROG SI:DISPLACED)
                     (mapc #'macroexpandall (cddr thing)))
                  ((DEFUN)
                     (mapc #'macroexpandall (cdddr thing)))
                  ((COND)
                     (dolist (clause (cdr thing))
                       (mapc #'macroexpandall clause)))
                  ((DO)
                     (dolist (var (cadr thing))
                       (if (not (atom var)) (mapc #'macroexpandall var)))
                     (mapc #'macroexpandall (caddr thing))
                     (mapc #'macroexpandall (cdddr thing)))
                  (otherwise
                     (mapc #'macroexpandall (cdr thing))))))))
    thing)

(defun ops (x &optional (stream? standard-output) &rest substrings)
  (let ((ops (copylist (if (listp x) x (send x ':which-operations)))))
    (dolist (s substrings)
      (setq ops (mapcan #'(lambda (x)
                            (if (string-search s x) (ncons x)))
                        ops)))
    (setq ops (sort ops #'string-lessp))
    (when stream?
      (let ((fields-for-display
              (let ((package (pkg-find-package "GLOBAL")))
                (letS ((sym (elist ops)))
                  (rlist (cond ((eq (symbol-package sym) package)
                                (text ":~S" sym))
                               (t (text "~S" sym))))))))
        (let* ((field-width (+ 2 (rmax (string-length
                                         (elist fields-for-display)))))
               (n-fields    (// (or (send-if-handles stream?
                                                     ':size-in-characters)
                                    72.)
                                field-width))
               (field-height (// (+ (length fields-for-display)
                                    (1- n-fields))
                                 n-fields)))
          (let ((buckets (do ((i 0 (1+ i))
                              (f fields-for-display (cdr f))
                              (b nil)
                              (bs nil))
                             ((null f)
                              (dotimes (i (- (length (car bs))
                                             (length b)))
                                (push "" b))
                              (lexpr-funcall
                                #'mapcar
                                #'list
                                (nreconc bs (ncons (nreverse b)))))
                           (when (= i field-height)
                             (push (nreverse b) bs)
                             (setq b nil)
                             (setq i 0))
                           (push (car f) b))))
```

```
        (let ((format-string (text "~~&~~{~~{~~~D,' A~~}~%~~}"
                                   field-width)))
          (format stream? format-string buckets))))))
(cond (stream? (values))
      (t ops))))
```

```
;;;; Vocabulary

(defun parse-request ()
  (if (eq (peek-token) 'to) (read-token))
  (cond ((atom (peek-token))
         (unparse-command (parse-sentence)))
        (t
         (parse-name ':expression))))

(define-action tell ((actor (parse-name ':expression))
                     (request (parse-request)))
  "Asks an actor to tell another actor to do something."
  (tell actor task request))

(define-action ask ((actor (parse-name ':expression))
                    (request (parse-request)))
  "Asks an actor to ask another actor to do something."
  (ask actor task request))
```

```
;;;; Debugging

(defun init (&optional (query? (not *debug*)))
  (cond-every ((or (not query?) (fquery '() "Init database? "))
               (db-init)
               (format query-io "~&Database initialized.~%"))
              ((or (not query?) (fquery '() "Init belief system? "))
               (send *beliefs* ':reset)
               (format query-io "~&Beliefs reset.~%"))
              ((or (not query?) (fquery '() "Init processes? "))
               (do-society (actor)
                 (if (send-if-handles actor ':reset)
                     (format query-io "~&~A's process has been reset."
                             actor))))))

(defun debug (&optional (on? t) (query? t))
  (setq *debug* on?)
  (let ((request (text "~:[Disable~;Enable~] debug mode" on?))
        (query-io terminal-io))
    (format query-io "~&~A for...~%" request)
    (do-society (actor)
      (let* ((name (send actor ':short-name)))
        (when (and (send-if-handles actor ':process) ;ignore dummy actors
                   (or (not query?) (fquery '() " ~A? " name)))
          (if (not query?) (format query-io "~&~A: " name))
          (condition-case (cond)
              (ask name task request)
            (error (format query-io " ~A" cond))
            (:no-error (format query-io "  Done.")))))))
  on?)

(defun kbe-1 ()
  (let ((request (prompt-and-read-command)))
    (condition-restart-case (condition-case-if (not *debug*) (cond)
                               (eval-command request)
                             (sys:error
                               (format t "~&<<Request Failed>>~%~A~&"
                                       cond)
                               (values)))
      (((sys:abort sys:error) "Abort this KBE request.") (values)))))

(defun kbe (&optional (delegate "Apprentice 1") (verbose? t))
  (with-delegate (parse-actor delegate)
    (when verbose?
      (format t "~&Entering the KBE command loop.~
                 ~% Type text commands to ~A or ~C to exit.~:[~
                 ~% Note: You are not in debug mode.~;~]~%"
              delegate #\abort *debug*))
    (condition-restart-case-if verbose?
                               (do () (nil) ;indefinitely
                                 (let ((l (multiple-value-list (kbe-1))))
                                   (unless (not l)
                                     (psetq *   (car l)
                                            **  *
                                            *** **))
                                   (format t "~&~@[^Y ~{~S~^,~%  ~}~%~]"
                                           l)))
      (((sys:error null-sentence) "Exit the KBE command loop.")
       (format t "~&Conversation with ~A terminated.~
                  ~%Leaving the KBE command loop.~%"
               delegate)
       (values)))))
```

```
;;;; Debugging window

(comment ;This doesn't work yet. --KMP 3/13/84

(define-system-initialization debug-frame-system-key
  (add-system-key #/D t   "Debug Frame (KBE)"  'debug-frame))

(defflavor debug-pane () (simple-pane-mixin
                          associated-process-mixin
                          actor-window)
  (:default-init-plist
    :delegate-flavor 'standard-actor
    :selectable-p t))

(defmethod (debug-pane :after :activate) (&rest ignore)
  (send self ':set-label
        (text "~A: ~A"
              (send self ':name)
              (send (send self ':delegate) ':short-name))))

(defmethod (debug-pane :process-toplevel) ()
  (kbe (send (send self ':delegate) ':name) nil))

(defflavor debug-frame () (tv:bordered-constraint-frame)
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables
  (:default-init-plist
    :save-bits t
    :panes '((pane-1 debug-pane)
             (pane-2 debug-pane)
             (lisp-pane listener-pane))
    :constraints
       '((main . ((pane-1 pane-2 lisp-pane)
                  ((lisp-pane 0.25))
                  ((pane-1 0.5))
                  ((pane-2 :even)))))
    :selected-pane 'pane-1))

) ;tnemmoc
```

```
;;;  ADA Read-Eval-Print Loop

;;; (ADA-REPL-PROMPT stream char)
;;;
;;;  Called internally by ADA-REPL in order to do prompting.

(define-internal-option *ada-input-marker* "Ada input prompt"
  "" :string)

(defun ada-repl-prompt (stream char)
  char ;ignored
  (format stream "~&~A" *ada-input-marker*))

;;; (ADA-REPL-READ [stream])
;;;
;;;  Reads an ADA expression (from a given stream, if one is provided).
;;;  If possible, does so with rubout-handling enabled.

(define-condition ada-resume ())

(defun ada-repl-read (&optional (stream terminal-io) (break? nil))
  (cond ((send stream ':operation-handled-p ':rubout-handler)
         (ada-repl-prompt stream nil)
         (do () (nil) ;forever
           (cond ((eq (tyipeek) #\c-Z)
                  (tyi) ;Read and echo the char
                  (signal 'sys:abort))
                 ((and break? (= (tyipeek) #\Resume))
                  (send stream ':tyi)
                  (format stream "~&[Resume]~%")
                  (ada-resume)))
           (condition-restart-case
             (return-if
               (send stream ':rubout-handler '((:prompt ada-repl-prompt))
                     #'ada-parse-to-end stream '(#\End #/;)))
             ((sys:abort "Start over with typing this Ada statement.")
              (ada-repl-prompt stream nil)))))
        (t
         (ada-parse-to-end stream '(#\End #/;)))))

;;; (ADA-REPL [show-result? T] [show-result-in-ada? T])
;;;
;;;  Runs an ADA Read-Eval-Print Loop.
;;;  The optional argument SHOW-RESULT? says whether to show return values.
;;;  The optional argument SHOW-RESULT-IN-ADA? says if ADA display is used.

(define-internal-option *ada-result-marker* "Ada result marker"
   "" :string)
;  " ^Y " :string)

(defun ada-repl (&optional (break? nil)
                           (show-result? t)
                           (show-result-in-ada? t))
  (with-user-package
    (condition-case ()
        (let ((ada-supress (not show-result-in-ada?)))
          (do ((-   '-)
               (*   '*)
               (**  '**)
               (*** '***)
               (+   '+)
               (++  '++)
               (+++ '+++))
              (nil)                        ;forever
            (condition-restart-case
              (progn (setq - (ada-repl-read standard-input break?))
                     (let ((temp (prog1 (multiple-value-list (eval -))
                                        (setq +++ ++
```

```
                                        ++  +
                                        +   -))))
                        (when temp
                          (setq *** **
                                ** *
                                *  (car temp))
                          (when show-result?
                            (let ((base 10.) (*nopoint t))
                              (format t "~&~A~:="
                                      *ada-result-marker* *))))))
                ((sys:abort "Return to Ada command level")
                 (format t "~&Back to Ada command level")))))
        ((sys:abort ada-resume) ':resume))))

(defun ada-break-repl (&optional (string nil))
  (format t ">ADA Breakpoint~@[ ~A~]; ~
             Resume to continue, Abort to quit.~2%"
          string)
  (ada-repl t))

(defun (com-mode-break :ada) () (ada-break-repl))

;;; Define an Ada Listener window

(defflavor ada-listener () (associated-process-mixin tv:window)
  (:default-init-plist
     :save-bits t))

(defmethod (ada-listener :process-toplevel) ()
  (with-user-package (ada-repl)))

;;; Define relevant editor commands

(defflavor ada-listener () (associated-process-mixin tv:window)
  (:default-init-plist
     :font-map  '(fonts:bigfnt)
     :save-bits t))

(defmethod (ada-listener :process-toplevel) ()
  (ada-repl))
```